
Cyberboswachters

De beste digitale stropers zijn ook de beste cyberboswachters

Dams Tim



Inhoudsopgave

Introductie	1
Changelog:	1
Het voorbije decennium	2
Voor 2010	2
2010: En dan verscheen Stuxnet	3
2013 : Onze privacy te grabbel gegooid	5
2014: hoe veilig is <i>the cloud</i> ?	6
2015: Stuxnet bewaarheid	9
2015: Scheidingen en zelfmoord	9
2016: Internet-of-horrors	11
2017: Ransomware wordt gemeengoed	12
En nu: de geopolitiek macht van sociale media en grote datalekken	12
En de toekomst?	13
Het is erger, maar...	14
GDPR	15
Wat omvat GDPR	15
Wat beschermd GDPR	16
Meldplicht datalekken	16
Uitzonderingen meldplicht individu	17
Boetes	18
Cryptografie	21
Waarom cryptografie?	21
CIA en het security model	21
Encryptie	22
Kerckhoffs principe	23
De eerste algoritmes	24
Substitutie: Caesar encryptie	25
Transpositie: scytale encryptie	26
Combinatie	27

Cryptanalyse	28
Symmetrische encryptie	29
Sleuteloverdracht	30
Block en Stream cipers	30
Streamciphers	30
Blockciphers	36
Asymmetrische encryptie	46
Het probleem met symmetrische encryptie	46
Publieke cryptografie	47
Dualiteit van publieke cryptografie	48
Diffie-Hellman sleutel uitwisseling	49
RSA	50
Boodschappen ondertekenen	52
Digitale certificaten	55
Wifi security	61
De problemen van Wifi	61
Onbeveiligde managementframes	64
The 802.11 standard and its security	66
Joining a network	66
WEP	69
How WEP failed	73
WPA 1	82
TKIP	86
Putting it all together	89
WPA 2	89
Encryption and MIC creation	90
There goes the neighbourhood	91
WPA 3 (“Wifi 6”)	92
Dankwoord	93

Introductie

Nu dat Zie Scherp en Zie Scherper ingeblikt zijn, wordt het tijd voor een nieuw project. De teksten in deze PDF zijn pre-alpha. Dit wil concreet zeggen dat:

- De teksten nog niet zijn nagelezen qua inhoud én spelling.
- De teksten nog niet compleet zijn.
- Veel afbeeldingen nog niet gecleared zijn om gebruikt te worden qua rechten.

Alle feedback is in deze fase zéér welkom:

- Schrijffouten.
- Zinnen die onduidelijk zijn.
- Paragrafen die je niet begrijpt.
- Zaken die ZEKER niet meer aangepast mogen worden omdat ze geniaal zijn.

Changelog:

Dit is versie 0.0.8 (3/5/21)

- 0.0.8
 - Uitleg en tekeningen certificaten toegevoegd. Hiermee kan hoofdstuk crypto (voorlopig) als afgerond beschouwd worden.
- 0.0.7
 - Aantal tekeningen omgezet
- 0.0.6
 - Wifi hoofdstuk vervolledigd (deels Engelstalig) Afbeeldingen moeten nog goed gezet worden.
- 0.0.5
 - Hoofdstuk 1 en 2 aangevuld en op schrijffouten verbeterd. # Wordt het erger?

De laatste 10 jaar is de (cyber)security wereld erg veranderd. Ze doet dit niet omdat ze daar zin in heeft, maar wel als antwoord op wat er gebeurt in de wereld in zake cyberaanvallen, geopolitieke situaties, etc.

Het is altijd een kat en muisspel, waarbij de boswachters helaas by definition steeds zullen achterlopen op de stropers. De kwaadwillige hackers hoeven maar 1 klein gaatje te vinden in je peperdure beveiliging en ze zijn binnen. Terwijl jij wel aan alles moet (proberen te) denken. Dat het erger wordt is eigenlijk daarom bijna automatisch een evidentie. De wereld van de beveiliging gebeurt per opbod en hoe beter de boswachters het bos kunnen verdedigen, hoe complexer de technieken zullen worden die de stropers hanteren.

In de volgende secties gaan we een klein historisch overzicht geven van enkele belangrijke, interessante of spectaculaire gebeurtenissen die hebben plaatsgevonden in de cyberwereld de voorbije jaren. Dit laat ons toe om enerzijds enkele begrippen te duiden, anders om de vraag “wordt het erger?” te beantwoorden.



We gebruiken de term cyber om alles aan te duiden dat zich afspeelt op de digitale snelweg, namelijk het internet, de cloud, het www, en alle synoniemen en aanverwanten.

Het voorbije decennium

We gaan geregeld terug in de tijd gaan om te bekijken hoe bepaalde cyberverdedigings- en aanvalstechnieken zijn ontstaan, maar in dit hoofdstuk gaan we enkel terug tot 2010. Het jaar waarin Mack Zuckerberg, CEO van Facebook, door Time Magazine tot persoon van het jaar werd uitgeroepen en ook waarin Apple de eerste iPad tablet aan het publiek toonde. Het was helaas ook het jaar waarin het boorplatform, Deepwater Horizon, voor één van de ergste milieurampen ooit zorgde, alsook het jaar dat 33 mijnwerkers anderhalve maand lang 700 meter diep opgesloten zaten in een mijn. Voor ons, als toekomstige cyberboswachters, is echter de meest cruciale gebeurtenis het ontdekken van **Stuxnet**.

Voor 2010

Voor het ontdekken van Stuxnet in 2010 leek de cyberbeveiligingswereld wel een wereld van (relatieve) peis en vree. Het ergste dat kon gebeuren was dat je computer een virusje opdeed waardoor je mogelijk wat data, en vooral veel werkuren, kwijt raakte. Virussen, wormen en spam waren alomtegenwoordig maar eigenlijk niet meer dan luizen in de pels van de eindgebruikers. Tuurlijk, er werd een grondig gevloekt als een virus je foto's verwijderde of wanneer een worm zichzelf verspreidde naar je contacten - denk maar aan het *ILOVEYOU* virus dat als een soort *social engineer* mensen deed geloven dat ze een potentiële liefdesmatch hadden waarop ze vol spanning de bijlage openden en zo de worm *in*

huis haalden. Of wat te denken van de *Conficker* worm die in 2008 meer dan 3 miljoen systemen kon besmetten en telkens de update-services van het Windows toestel uitschakelde. Uiteraard, dat was irritant, maar menig mens zou maar al te graag terug naar die tijd willen gaan als daarmee ransomware, botnets en door overheden gesponsorde cyberaanvallen onbestaande zouden zijn.

//TODO: jan g vragen voor beschrijving over hoe het op bedrijven was



De termen worm, virus en malware zal geregeld door elkaar worden gebruikt in deze cursus. Alle drie betekenen net niet hetzelfde, maar toch:

- Een **virus** is een kwaadaardig programma dat, eens het op een computer of apparaat staat, digitale schade kan aanbrengen.
- Een **worm** daarentegen is ook een virus, maar eentje dat zichzelf kan *voortplanten* naar andere systemen, iets wat een virus niet kan. Een worm kan zichzelf dus verspreiden zonder menselijke hulp.
- **Malware** is een overkoepelende term voor alle programma's en code die digitale schade aan een systeem of netwerk brengen. Virussen en wormen behoren dus tot deze groep.

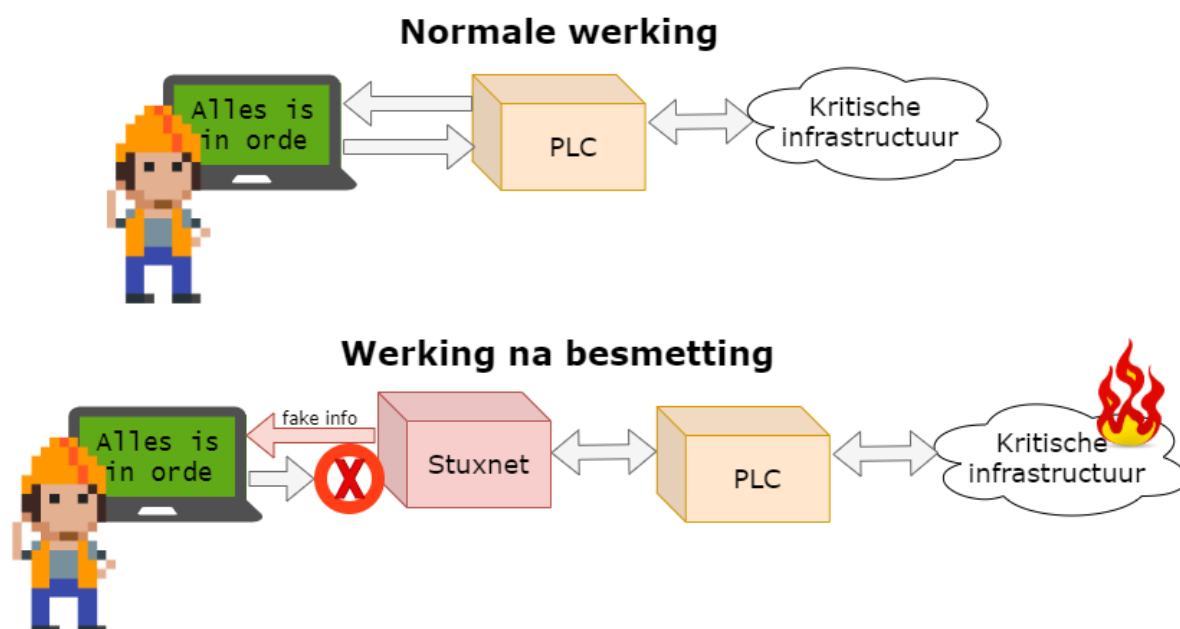
Naast deze 3 termen zullen we ook nog enkele andere malware types tegenkomen zoals Adware, spyware, spam, trojans, backdoors en rootkit. Wanneer nodig zullen we deze termen toelichten. Onthoud nu alvast dat malware de algemene naam is voor alle malafide software.

2010: En dan verscheen Stuxnet

Virussen konden computers (tijdelijk) onbruikbaar maken. OK, lastig, maar niet het einde van de wereld. De Stuxnet worm die in 2010 plots op de radar verscheen kon dat potentieel wel, de wereld eindigen. Het was een worm die op maat was gemaakt om zogenaamde *Programmable Logic Computers (PLC)* te controleren. Héél veel van onze kritische systemen zijn geautomatiseerd met behulp van deze PLCs, krachtige apparaten die machines kunnen aansturen zoals liften, robotarmen aan assemblagelijnen, zuiveringsinstallaties en zelfs kernreactors. PLCs vormen de interface tussen machines (hardware) en de computers (met software op) die de operatoren gebruiken om deze machines opdrachten te geven. Operators kunnen op hun systemen de machines bedienen en ten allen tijde controleren of deze naar behoren werken.

Wat Stuxnet deed was zich tussen de hardware en de software nestelen. Als een virus besmette het laptops en computers en zocht het of er PLC-bedieningssoftware aanwezig was op het toestel. Als dat het geval was dan plaatste het zichzelf er tussen in zodat het:

1. de machines opdrachten kon geven zonder dat de operator dit wist.
2. aan de operator kon vertellen dat alles in orde was, terwijl in realiteit de PLC mogelijk desastreuze opdrachten aan de hardware gaf.



Figuur 0.1: Werking stuxnet

We moeten er geen tekeningetje bij maken wat de effecten kunnen zijn indien een Stuxnet variant bewust werd ingezet om bijvoorbeeld de machinerie in een kerncentrale of waterdam te saboteren. Zonder in details van Stuxnet in te gaan is het duidelijk dat het verschijnen van dit virus een ommekeer betekende in wat de gevolgen van cyberterrorisme konden betekenen: mensenlevens stonden plots op het spel, niet enkel onze dierbare e-mails en digitale vakantiefoto's.



Stuxnet bleek een bewust ontworpen virus te zijn dat probeerde een specifiek soort centrifuge te saboteren. Namelijk de centrifuges die Iran gebruikte om uranium te verrijken. De inlichtingendiensten van de Verenigde Staten en Israël zouden klaarblijkelijk Stuxnet hebben ontworpen om zo dit verrijkkingsproces van uranium van Iran te dwarsbomen.



Wens je meer te weten over stuxnet? Dan raden we je de uitstekende, in 2006 verschenen, documentaire “Zero Days” door Alex Gibney aan. Deze documentaire geeft een zeer ontluisterend én boeiend beeld over de zoektocht naar de oorsprong van het Stuxnet virus.

2013 : Onze privacy te grabbel gegooid

Surfen op het internet was altijd een risico. We wisten in 2013 al dat onze datapakketjes over tientallen apparaten doorheen het internet wordt getransporteerd om zo tot bij hun doel te geraken. Cookies volgden al geregeld onze stappen en ook de aanbevelingen van Amazon waren soms akelig accuraat. Dat was de prijs die we moesten betalen om de ontelbare bronnen van het internet te kunnen gebruiken. En als je echt wat meer privacy nodig had, dan schakelde je de “incognito” modus in van je browser. Maar ook dan was je je ervan bewust dat zelfs je ISP (*internet service provider*) en je doel konden meekijken. Hoe erg kan dat zijn?

Die gedachte leefde bij veel mensen tot dan: het Internet was een nuttige tool en je wist dat er kon meegekeken worden door *derden* als ze dat echt wilden. Maar zou dit nu echt consequent en op grote schaal gebeuren? Neen toch?!

Dat glazen huisje werd in 2013 hardhandig aan diggelen geslagen door twee belangrijke personen:

- **Edward Snowden:** als (contractueel ingehuurd) systeembeheerder bij de NSA, de Amerikaanse inlichtingendienst gericht op elektronische spionage, had hij toegang tot het doen en laten van de dienst. Snowden lekte een grote hoeveelheid documenten naar de bevolking die onder andere het *PRISM*-programma uit de doeken deed. Hieruit bleek dat de NSA complexe samenwerking had met enkele grote Amerikaanse internetbedrijven (o.a. Microsoft, Google, Facebook, Apple, etc.) die zogenaamde *taps* op hun systemen hebben staan zodat de NSA “kan meeluisteren” op de netwerken (dit programma heette *PRISM*).
- **Julian Assange:** Er zijn altijd klokkenluiders (*whistleblowers*) geweest die om persoonlijke overtuigingen vonden dat bepaalde informatie met het publiek moesten worden gedeeld. Zeker in dictatoriale middens kan dit levensgevaarlijk zijn. Aan het eind van de 20e eeuw kregen deze klokkenluiders echter een erg krachtig apparaat om hun nieuw te verkondigen: het Internet. Julian Assange beseftte dit en richtte daarom in 2006 Wikileaks op. Een site waar klokkenluiders op een anonieme manier documenten konden *leken*. Tot 2010 zijn zo enkele erg controversiële documenten met de wereld gedeeld die soms verregaande diplomatieke of geopolitieke gevolgen hadden.



Het is niet evident om over mensen als Assange en Snowden te praten zonder in een controversiële modderpoel te geraken. Voor de één is Snowden een held, voor de ander een verrader. In dit boek trachten we een objectief beeld te geven, zonder waardeoordelen te geven. Wat wel buiten kijf staat is dat zowel Snowden als Assange de wereld getoond hebben dat er in de duistere wandelgangen van de veiligheidsdiensten zaken gebeuren waar “normale stervelingen” zelden weet van hebben.

Het waren uiteraard vooral de onthullingen van Snowden die bij velen de oogkleppen deed afvallen. Op het Internet ben je hoegenaamd niét anoniem. Grote (en kleine) mogendheden en privéfirma's kunnen ons doen en laten op het internet bekijken, bewaren en analyseren. Privacy en het internet zijn een *oxymoron*: een contradictorische term zoals zwarte sneeuw. Wanneer je je op het internet begeeft, op welke manier ook, gooi je een deel van je privacy te grabbel. En dat is iets dat helaas de voorbije 10 jaar er niet op verbeterd is (alhoewel het **Tor** netwerk met z'n *onion routing* toch wel een stevige extra privacy laag kan aanbieden).



Een interessant debat dat altijd opduikt bij deze problematiek is de “Ik heb toch niets te verbergen”-houding. In het kleine maar fijne boekje “Je hebt wél iets te verbergen” van onderzoeksjournalisten Maurits Martijn en Dimitri Tokmetzis (ISBN 9789082821611) wordt onherroepelijk brandhout gemaakt met deze stelling. Finaal zijn we volledig afhankelijk van de diensten die we gebruiken wat ze met onze data nu én belangrijker, in de toekomst zullen doen. Privé-informatie over jou die nu ogenschijnlijk ongevaarlijk lijkt, kan dat potentieel in de toekomst wel zijn wanneer normen, waarden of wetten veranderen.



Alhoewel het **HTTPS** al sinds 1995 bestond, werd het in 2013 amper door websites aangeboden. Nochtans geeft https een extra defensielaag tijdens de communicatie van jouw computer met die waar een website op *gehost* staat. Htps zal namelijk je communicatie versleutelen (zie hoofdstuk crypto) zodat enkel zender en ontvanger kunnen lezen wat er gezegd wordt. Met http is dat niet: al je communicatie kan door eender wie gelezen worden die zich tussen jouw computer en je eindbestemming nestelt. Pas in 2017 boden meer dan de helft van de website wereldwijd https aan. In 2021 gebruikt ongeveer 70% van alle website https als standaard communicatiemiddel aan (vroeger waren er al websites met https, maar http was de standaard oplossing).

[//https://transparencyreport.google.com/https/overview](https://transparencyreport.google.com/https/overview)

2014: hoe veilig is *the cloud* ?

In 2014 vond er een controversale plaats die vooral de puberende tiener zich zal herinneren: “*The fapping*”. Deze naam laat weinig aan de verbeelding over en helaas de lading goed. In het jaar dat Oekraïne Russische legers zag binnenrollen om de Krim (terug) in te palmen, verschenen er duizenden privéfoto's van een honderdtal *celebrities* op het internet. Deze privéfoto's hadden kwaadwillige hackers van de Apple iCloud accounts van de slachtoffers gestolen en vervolgens gepubliceerd via Reddit, 4chan, etc. De foto's verspreidden zich als een lopend vuurtje en de slachtoffers konden enkel

toezien hoe hun in de privésfeer opgenomen beelden door miljoenen mensen werden gedownload.

De beroemdheden hadden hun foto's in de cloud-opslag van Apple bewaard zoals op dat moment duizenden gebruikers ook al deden. Deze dienst zorgt ervoor dat je je als eindgebruiker van eender waar aan je foto's kan, daar ze "in the cloud" staan, wat door de spectaculaire groei van de smartphones (en iPhones in dit geval) een populair concept was geworden. Ook diensten zoals Dropbox, Onedrive (toen nog SkyDrive) toonden aan dat er een grote vraag was naar online opslag van informatie.

Om dergelijke diensten te gebruiken dien je natuurlijk een veilig paswoord te hebben, daar eender wie, van eender waar, kan proberen zich een weg naar je data te verkrijgen door jouw paswoord te raden. In 2014 waren concepten zoals 2-factor-authentication (2FA) en biometrische beveiliging (meer daarover later) nog niet zo populair en dus was je geheime paswoord je enige bescherming tegen onrechtmatige toegang tot je data.

De diefstal van de foto's werd echter vergemakkelijkt om 2 redenen:

- Sommige slachtoffers gebruikten makkelijk te raden, of snel te bruteforcen paswoorden.
- Andere hadden de beveiligingsvragen ter goeder trouw ingediend. Veel diensten stellen een extra beveiligingsvraag (zoals "Wat is de naam van je eerste huisdier" of "Op welke school zat je vader") die ze kunnen gebruiken om jouw identiteit te verifiëren indien je je paswoord vergeten bent en je dit wenst te resetten. Wanneer jij of ik dit soort vragen invullen dan is dit 90% van de tijd informatie die nergens te vinden valt, enkel in de grijze massa in je hoofd en misschien in een gênant dagboekje uit je jeugd. Bij de Amerikaanse beroemdheden wiens iCloud werd gehackt, is dat niet zo. Hun leven kan volledig gereconstrueerd worden aan de hand van de ontelbare interviews die ze al hebben gegeven. Gegarandeerd dat ooit een interviewer al heeft gevraagd wat de kleur van zijn of haar eerste auto was, of in welk dorp hij of zij is opgegroeid.



Het is een verkeerde reflex om in dit soort zaken ogenblikkelijk aan *victim shaming* te doen (kijk maar naar het recentere voorval waarbij enkele Bekende Vlamingen het slachtoffer waren van catfishing). Ieder doet en laat wat hij wenst in z'n privésfeer - daarom heet het ook privé- maar het is belangrijk te beseffen dat als je zaken *in the cloud* bewaard, de kans bestaande is dat iets of iemand er ooit onrechtmatige toegang tot zal krijgen. U weze gewaarschuwd.



Niemand verplicht je om op de beveiligingsvragen een eerlijk antwoord te geven. Er bestaat geen leugendetector in die systemen of een mama die op je vingers komt tikken. Lieg er dus op los, maar onthoudt uiteraard je antwoord. Dankzij die beveiligingsvragen hou je echter een stok achter de hand moest je je paswoord vergeten zijn.

Ook in 2014: als landen vechten

Het was te verwachten dat ook op cyberniveau er ooit een cyber-wapenwedloop zou starten zoals, helaas, ook in de echte wereld plaatsvindt. Daar waar landen voorheen nog vooral defensieve cybermogelijkheden hadden, begon halverwege het vorige decennium dit arsenaal meer en meer uitgebreid te worden met offensieve cyberwapens. Het was met andere woorden wachten tot de eerste cyberaanvallen door soevereine staten op andere staten.

Het probleem met cyberaanvallen is dat het als aangevallen mogendheid ongelooflijk moeilijk is om

1. De bron van de aanval te identificeren. Wie weet lijkt de aanval te komen vanuit land X, terwijl het eigenlijk land Y is dat gewoon de aanval via land X routeert. En het laatste dat je natuurlijk wil doen is het verkeerde land beschuldigen.
2. En wat als een hacker, zonder staatsinmengingen, beslist om op eigen houtje een cyberaanval te initiëren. Is het land van herkomst van die aanvaller dan verantwoordelijk voor de schade?
3. De “schade” van een cyberaanval is niet altijd duidelijk. Wanneer een raket op een stad wordt afgestuurd is dat een duidelijk *act of aggression* en is er grote kans op een militair conflict (indien de diplomatieke weg geen soelaas brengt). Maar is een cyberaanval, zoals een denial-of-service (DoS, zie later), genoeg reden om de oorlog buiten het cyberdomein te escaleren? Zijn er eigenlijk *rules of engagement*? Is er een conventie van Genève? Neen op dit alles. Daar cyberaanvallen zo moeilijk te traceren en identificeren zijn, blijft het allemaal het betere nattevingerwerk werk.

Een bewuste cyberaanval op een soevereine staat was al een enkele keer gebeurt (denk maar aan de cyberaanvallen in 2007 van Rusland op Estland) maar in 2014 was er helaas een nieuwe primeur. Ook nu moeten we naar Hollywood keren, niet om de beroemdheden en hun privé-kiekjes, maar omwille van een film waar een andere staat niet om kon lachen.

In 2014 zou Sony een nieuwe film uitbrengen, getiteld “The Interview”, met James Franco en Seth Rogen. In deze komedie worden 2 journalisten gevraagd om tijdens hun interview van een *fictieve* Noord-Koreaanse dictator hem te doden. De vergelijkingen met de Noord-Koreaanse leider Kim Jong-un waren voor iedereen duidelijk. Dat Noord-Korea op de tenen zou zijn getrapt stond dan ook in de sterren geschreven.

De daaropvolgende gebeurtenissen zijn nog steeds niet uitgeklaard, maar vast staat wel dat een selecte groep hackers toegang had gekregen tot confidentiële data op de servers van Sony en deze vervolgens op het internet lekte. De financiële schade voor Sony was enorm. De daders wisten onuitgebrachte films, e-mails, salarissen en andere privé-informatie te stelen. Heel lang werd gedacht dat de groeperingen die achter de aanval stond door Noord-Korea was aangestuurd. Echter, nu nog steeds heeft men dat niet kunnen bevestigen. Zou zijn er ook experts die denken in het bewijsmateriaal de hand van Rusland en/of China te zien. Kortom, zoals eerder gezegd, cyberaanvallen zijn verdomd lastig om in kaart te brengen.

Als het toch Noord-Korea zou zijn geweest - wat nog steeds meer dan mogelijk is- dan hebben ze hiermee dus een dubieuze primeur: een soeverein land voert een cyberaanval uit op een bedrijf in een ander land. En de vraag die vervolgens veel experts zich stelden was: “vanaf wanneer is dit een *act of aggression*?”, en ook: “moet een land op dit soort aanval reageren namens het bedrijf, of namens de hele natie waar het bedrijf zich bevindt?” Wie zal het zeggen. Dit boekje helaas niet, maar het geeft wel voer voor discussie.

2015: Stuxnet bewaarheid

Keer op keer zullen we dit moeten herhalen: we kunnen nooit met 100% zekerheid de origine van een cyberaanval vaststellen. Heel af en toe, met dank aan klokkenluiders zoals Snowden, of verregaand (al dan niet journalistiek) onderzoek worden specifiek aanvallen volledig uit de doeken gedaan (maar helaas vaak jaren na datum).

In 2015, iets meer dan een jaar na de Sony aanval, werd dat waar we voor vreesden bewaarheid: hackers waren er in geslaagd om de elektriciteitsinfrastructuur van Oekraïne deels plat te leggen op 23 december, vlak voor kerstavond. Een groepering slaagde er zo in om bijna een kwart miljoen mensen gedurende meerdere uren zonder stroom te zetten. Iedereen keek uiteraard ogenblikkelijk in de richting van Rusland - Oekraïne maakte vroeger deel uit van de voormalige Sovjetrepubliek- daar de aanvallen kwamen van IP-adressen die waren toegewezen aan Rusland. Maar zelfs als dat zou zijn, zonder harde bewijzen dat de hackers ook handelden in naam van de Russische overheid blijft het koffiedik kijken wie *op de vingers getikt* moet worden voor de aanval.

Wie het ook waren, één ding staat wel vast: in 2015 waren hackers er voor het eerst in geslaagd om met één welgemikte aanval honderdduizenden mensen in problemen te brengen die verder gingen dan het verwijderen van enkele bestanden.

Enkele jaren later, in februari 2021, zou het trouwens weer bijna prijs zijn. Een hacker kreeg toegang tot de systemen die de waterzuiveringsinstallaties van Oldsmar (Florida, VS) bediende. Een operator zag op tijd dat z'n muis plots bewoog en de maximum toegelaten hoeveelheid natriumhydroxide waarde van de filters op een dodelijk niveau zette. Er werd gelukkig tijdig ingegrepen (en detectoren verderop in het systeem hadden het vergiftigde water sowieso gedetecteerd), maar het deed wederom de vraag rijzen of onze kritische systemen wel voldoende beveiligd zijn tegen dit soort *lone wolves*.

2015: Scheidingen en zelfmoord

In 2015 wordt de schaal van cyberaanvallen steeds groter. De hoeveelheden informatie die hackers van bedrijven kunnen bemachtigen kunnen al lang niet meer op een A4'tje afgedrukt worden. Wanneer hackers toegang krijgen tot de privé-servers van hun doelwit kunnen ze vlotjes ettelijke gigabytes, tot

zelfs terabytes, aan privé informatie stelen. Tegenwoordig hebben we in Europa de GDPR wetgeving die probeert bedrijven duidelijk te maken (en te straffen) dat zij verantwoordelijk zijn om onze data op een veilige manier te bewaren (zie volgende hoofdstuk). In 2015 was dat veel minder. De berichten van datalekken in die tijd spraken boekdelen: van zodra cybercriminelen toegang hadden tot de privéservers konden ze de data zonder problemen lezen. Paswoorden, kredietkaartgegevens, rijksregisternummers, alles stond vaak onbeveiligd (*ongeëncrypteerd*) op de systemen.

De website Ashley Madison was Tinder voor mensen die een relatie wilden naast hun “officiële relatie”. De site hielp kortom mensen aan een affaire. Hun leuze, *“Life is short. Have an affair”*, wond er geen doekjes rond. En met hun meer dan 20 miljoen “klanten” was het duidelijk dat ze een lucratief idee hadden. Dat ze tegenwind zouden krijgen was te verwachten. Maar een wind van 12 beaufort? Dat niet.

In juli 2015 plaatste een groep hackers een ultimatum op een website aan het adres van de eigenaars van Ashley Madison: *“Haal jullie moreel dubieuze website van het internet, of wij plaatsen meer dan 60 gigabyte aan gestolen data online”*. De site werd niet offline gehaald en de hackers “hielden woord.” De gevolgen waren immens, niet zo zeer voor Ashley Madison, wel voor z’n klanten. Plotsklaps kreeg de wereld een lijst te zien waarin honderdduizenden klanten open en bloot aan de schandpaal werden genageld. Mensen werden publiekelijk vernederd. Er is sprake van minstens 2 of 3 zelfmoorden rechtstreeks als gevolg van de lek. Mensen werden ontslagen. Kortom, het lekken van wat uiteindelijk maar een hoop binaire data- nullen en eentjes- was had gevolgen voor relaties, mensenlevens en carrières.

Als positieve noot in dit verhaal halen we hier uit dat dit soort gigantische lekken mensen heeft doen inzien dat ze twee maal moeten nadenken voor ze hun persoonlijke informatie weggeven aan één of andere internet-gigant (het is helaas een les die we jaarlijkse lijken te vergeten, kijk maar naar de populariteit van Tik Tok, Facebook, etc.).



Een stelregel die bedrijven nu hanteren is de volgende: vraag je niet af **of** je gaat gehackt worden maar vraag je af **wanneer** je zal gehackt worden. Dit is een heel andere manier van tegen je beveiligingsprobleem aankijken. Vergelijk het met het in huis halen van een koffer met daarin 10 miljoen euro aan diamanten. Als je je afvraagt of er ooit inbrekers zullen binnen geraken en daar naar beveiligt -waakhonden, videocamera's rond het huis, dubbel slot- dan heb je daar niets aan als ze vervolgens toch binnen geraken en je koffertje stelen. Veel beter kan je én je huis beveiligen, én ervan uitgaan dat ze de koffer gaan bemachtigen: en je dus maar beter ook ervoor zorgt dat de dieven niets met de diamanten kunnen doen (door bijvoorbeeld je naam er in te graveren).

Kortom: bedrijven moeten data die ze van ons opslaan minstens encrypteren en systemen inbouwen die de data onbruikbaar maken als ze toch gelekt zou worden.

2016: Internet-of-horrors

In onze huizen verschenen steeds meer apparaatjes die via het, meestal draadloze, netwerk met elkaar en het internet konden communiceren. De term Internet-of-Things (IoT) bracht een weelde aan nieuwe oplossingen in onze levens. Domotica, wearables, slimme thermostaten, beveiligingssystemen, weegschalen, alles werd én slimmer gemaakt én aan het internet gehangen.

Een inherent probleem met veel van deze kleine toestellen is dat ze op het gebied van beveiliging ondermaats presteren. Dergelijke IoT-apparaten werken vaak op batterijen en de makers willen natuurlijk de batterijduur zo lang mogelijk houden. Iedere extra feature die de designers in het apparaat willen steken heeft een kost op die levensduur. Een aspect zoals beveiliging werd dan ook vaak achteraan de lijst van potentiële features geplaatst. Komt daarbij dat IoT-apparaten updaten (met bijvoorbeeld nieuwe security patches) soms onmogelijk of tenminste omslachtig is. Als er dus een beveiligingslek wordt gevonden in een apparaat dan is de kans bestaande dat dit lek voor altijd aanwezig zal blijven én dus ook kan misbruikt worden.

Het was dus wachten tot de eerste aanvallen, specifiek gericht op IoT apparaten, zouden plaatsvinden. De meest opvallende aanval gebeurde eind 2016. Malware, genaamd Mirai, verspreide zich als een lopend vuurtje over IoT-apparaten waarvan de malware de standaard paswoord en username kende. Gebruikers die vergeten waren het paswoord aan te passen dat het apparaat heeft wanneer je het uit de doos haalde zaten zo plotseling met een ogenschijnlijk perfect werkend apparaat. Echter, de malware nestelde zich onzichtbaar op het apparaat en wachtte op commando's van de Mirai makers. De malware creëerde met andere woorden een zogenaamde *botnet*, een groot netwerk van besmette apparaten die allemaal commando's kunnen uitvoeren van de *botnet herder* (i.e. degene die de malware in de eerste plaats is beginnen verspreiden). De Mirai-makers hadden zo een leger minicomputers onder hun bevel die ze konden zeggen "surf nu allemaal naar die website". Een website die plots tienduizenden gebruikers onverwacht extra te verwerken krijgt zal vaak onder de druk bezwijken en crashen. Kortom, dit Mirai botnet kon zo grote distributed denial-of-service (DDOS) aanvallen uitvoeren, allemaal omdat gebruikers de paswoorden van hun gloednieuwe apparaatjes niet hadden aangepast. In hun verdediging, het is vaak een erg omslachtig, technisch, proces om het paswoord van een IoT-apparaat aan te passen.



De wildgroei van Internet-Of-Things apparaten heeft ervoor gezorgd dat hackers een grote hoeveelheid extra mogelijkheden hebben bijgekregen om op huis en bedrijfsnetwerken te infiltreren.



Een dubieuze (deels betalende) site, **shodan.io**, heeft als enige doel alle internet-of-things apparaten in kaart te brengen die zichtbaar zijn vanop het internet. Deze “Google voor IoT” toont zelfs om wat voor apparaten het gaat en welke bijvoorbeeld nog steeds het standaard (default) paswoord hebben.

2017: Ransomware wordt gemeengoed

De virussen in de vorige eeuw durfden al eens je data te verwijderen. Lastig, maar erg duidelijk: je data was je kwijt, tenzij je ergens een backup had liggen. De nieuwe virussen gingen echter een stapje verder: ze versleutelden al je data (vaak ook je backups als je zo dom was geweest deze op het zelfde apparaat te hebben) én vroegen vervolgens losgeld in ruil voor je data. De naam *ransomware* kon, helaas, niet beter gekozen zijn. Menig particulier betaalde ogenblikkelijk om de eenvoudige reden dat de ransomware de gebruiker nog een uitweg aanbood daar ransomware vaak werd “binnengehaald” door een gênante actie (bv illegale software downloaden, op reclame voor vage pornosites klikken, etc). Voor particulieren was ransomware vooralsnog irritant. Maar wat als de ransomware bedrijf kritische data begon te encrypteren? Dit is exact wat er gebeurde in 2017 met de WannaCry en Petya ransomwares. Deze malwares sloegen er in om banken, hospitalen, havenbedrijven en menig ander groot (en klein) bedrijf te besmetten. De economische schade werd geschat op bijna 4 miljard dollar vanwege het lamleggen van de productie (of in het geval van enkele ziekenhuizen: het redden van mensenlevens).

De schade én de losgeldbedragen worden ook steeds groter. Zo was er het voorval met Garmin in de zomer van 2020 dat niet alleen de populaire sport-tracking diensten gedurende meerdere dagen uit de lucht haalde, maar er ook voor zorgde dat menig vliegtuig niet mocht vliegen omdat de Garmin Pilot apps niet werkten waardoor de piloten geen up-to-date aeronautische plannen voorhanden hadden. Het is nooit geweten of Garmin het losgeld (10 miljoen dollar!) wel of niet heeft betaald, vast staat wel dat ransomware tegenwoordig een, helaas, erg lucratieve handel is geworden voor cybercriminelen.

En nu: de geopolitiek macht van sociale media en grote datalekken

2016 ging er een schokgolf doorheen de wereld. Tegen alle verwachtingen in won Donald J. Trump de presidentsverkiezingen na een bitsige strijd tegen Hillary Clinton. Dat social media een belangrijke rol zouden spelen dit decennium was al lang voorspeld. Facebook, Twitter, Google en konsoorten hadden miljarden gebruikers die met plezier hun privacy te grabbel gooiden in ruil voor dagelijkse dopamine-shots dankzij *likes* en *retweets*. Met dank aan gigantische *troll farms* (organisaties die duizenden fake social media accounts aanmaken en zo mee de social media algoritmes beïnvloeden om bepaalde

informatie te *nudgen* in de gewenste politieke richting) kon Trump honderdduizenden potentiële twijfelaars doen inzien dat hij het juiste antwoord was tijdens de verkiezingen.

De inmenging van Rusland in een buitenlandse verkiezing (een daad waar menig land zich reeds schuldig aan heeft gemaakt) was ontluisterend vanwege de schaal waarop het gebeurde. Het toonde de almacht van de grote Silicon Valley bedrijven en hoe zij op geopolitiek niveau een belangrijke speler zijn geworden. Iets dat vervolgens nogmaals bevestigd werd door het simultaan plaatsgevonden Cambridge Analytica schandaal, dat niet alleen mede ervoor gezorgd heeft dat Trump president werd, maar dat hoogstwaarschijnlijk ook een grote groep twijfelaars heeft kunnen overhalen om een pro-Brexit stem uit te brengen.

Drie jaar later zagen onderzoekers een soortgelijk fenomeen tijdens de verkiezingen van de Europese Unie in 2019. Een rapport toonde aan dat Rusland actieve misinformatie campagnes organiseerde om zo de verkiezingen te beïnvloeden. Ze gebruiken hierbij zogenaamde *bad actors*, fake social media accounts die (al dan niet fake) nieuws verspreiden dat “in de winkel van Rusland past”. Uit het onderzoek bleek dat de toenmalige belangrijkste EU-mandatarissen (Juncker, King, Tajani, etc.) soms tot 20% volgers op Twitter hadden die eigenlijk *bad actors* waren.

De voorbije jaren is er een (lichte) kentering bezig inzake de macht van de social media bedrijven, maar het blijft een feit dat momenteel wij alleen grotendeels afhankelijk zijn van door artificiële intelligentie aangedreven algoritmes die bepalen welke informatie wij willen/zouden/moeten consumeren, ieder uur van de dag. Zolang echter data het nieuwe goud is en bedrijven dit vrij kunnen bewaren (GDPR poogt dit in te perken) zullen zij hun algoritmes kunnen blijven voeden en trainen om nog griezeliger/knapper te maken.

Het gevolg van die *datahoarding* is echter ook dat datalekken ook steeds nefastere gevolgen hebben. Daar waar het bij Ashley Madison nog ging om een dikke 20 miljoen user accounts, medio 2021 zijn het aantal accounts dat bij een datalek betrokken zijn soms vertienvoudigd - de MGM Grand Hotels keten zag in de zomer van 2020 plots 142 miljoen van z’n gast-accounts te koop staan voor een schamele 3000 dollar in bitcoin.

En de toekomst?

Een glazen bol hebben we niet, maar vast staat dat het er niet op zal verbeteren. Zoals gezegd gaan bedrijven uit van *when* in plaats van *if* als het gaat over de vraag of ze al dan niet ooit gehackt zullen worden. Daarnaast zien we dat hoogtechnologische producten meer en meer ingeburgerd geraken in het cybercrimemilieu. Deep fakes video van bekenden zijn nu nog “grappig”, maar de realistische beelden (afbeeldingen, video én nu zelfs ook spraak) die ze kunnen produceren zijn al even niet meer te onderscheiden van het echte en zullen dus meer en meer kunnen gebruikt worden voor sextortion en soortgelijke schandalen. Dit soort trends zullen nog jaren *fake news* hoogtij laten vieren waardoor

ook toekomstige verkiezingen interessante doelen blijven voor andere mogendheden om hun stempel te drukken op geopolitieke tegenstanders.

Het is erger, maar...

Dus ja, het wordt helaas erger. De wapenwedloop in de cyberwereld gaat beangstigend snel vooruit en het wordt moeilijker en moeilijker om als “normale sterveling” er een antwoord op te geven. Als een IoT botnet van 10 miljoen apparaten morgen beslist om de infrastructuur van jouw KMO plat te leggen, dan zullen ze daar in slagen, ongeacht de miljoenen euro's die je hebt geïnvesteerd in *firewalls, intrusion detection systems, virusscanners en honeypots*.

Aan de andere kant heeft de wapenwedloop er wel voor gezorgd dat onze infrastructuur ook steeds complexere aanvallen kan weerstaan. Hierdoor wordt het voor huis-tuin-en-keuken malware een pak moeilijker om nog computers van thuisgebruikers te besmetten.

Maar laten we deze cyberstropers geen vrij spel geven! Laten we leren van hun technieken om zo zelf onze systemen en personeel te *hardenen* en te beschermen tegen wat niet anders dan het “wilde westen van het Internet” (dixit komiek Steven Wright) kan genoemd worden.

GDPR

De General Data Protection Regulation (2018), of in het Nederlands de *Algemene Verordening Gegevensbescherming* is een Europese richtlijn om de privacy van Europese burgers, eender waar in de wereld, te beschermen. Of zoals de EU zelf op haar website beschrijft “the toughest privacy and security law in the world” (wat ook effectief zo is: het is de enige wet momenteel die er in slaagt om wereldwijd de regels ervan af te dwingen). De wet zelf bestaat uit 99 verschillende artikels die de rechten, personen en verplichtingen van bedrijven die onder de verordening vallen, worden uitgelegd. Ieder bedrijf, eender waar in de wereld, die data bijhoudt van EU-burgers dient zich aan deze wet te houden op risico van een boete als ze dit niet doen.



We gaan in dit hoofdstuk enkel een high-level overzicht geven van GDPR opdat we niet verzanden in de taal der rechters en advocaten, het *legalese*.

Wat omvat GDPR

Het recht om persoonlijke gegevens te wissen of het recht om vergeten te worden. GDPR laat je toe, als EU-burger om eender welke site die ‘jou kent’ te verplichten alle, of specifieke, informatie over je te verwijderen. Als je dus vindt dat Google niet moet weten waar je huisadres is, dan heb je het recht google te verplichten deze informatie te verwijderen.



Google kreeg een tijd terug een zeer stevige boete omdat het niet inging op de vraag van een EU-burger om *vergeten te worden*. Sindsdien zijn ze een pak behulpzamer in het naleven van de GDPR-wetgeving.

Een ander belangrijk aspect van GDPR is het zogenaamde **recht op overdraagbaarheid van gegevens**. Voor GDPR bestond was het soms een helse opdracht om bijvoorbeeld van internetprovider te veranderen. Iedere provider werkte met eigen dataformaten waarin de klantgegevens werden bewaard. Bij een overdracht moest dit dan altijd omgezet worden naar het formaat van de nieuwe provider. GDPR verplicht dat dit soort data overdrachten van persoonsgegevens nu volgens een vast formaat moeten gebeuren. En belangrijker: deze overdracht moet zo transparant én zo geautomatiseerd

mogelijk gebeuren. Hierdoor kan de data quasi ogenblikkelijk in de database van de ontvanger geïntegreerd worden en zal jij als klant dus veel sneller de overstap kunnen maken zonder alle bijhorende administratieve rompslomp en vertragingen.



De voorganger van de GDPR was de DPD (Data Protection Directive). Dit was geen Europese wet (verodering) maar een aanbeveling en was een stap in de goede richting maar nog niet ver genoeg. GDPR heeft ervoor gezorgd dat individuele burgers geen speelbal meer zijn van de grote bedrijven die de data van burgers als het nieuwe goud verzamelen en ermee doen wat ze zelf willen.

Wat beschermd GDPR

De GDPR wet zorgt ervoor dat bedrijven veel bewuster met gebruikersdata omgaan. Ze kunnen namelijk verantwoordelijk gehouden worden indien gebruikersdata gestolen of gelekt wordt. De GDPR is een privacy-wetgeving in de eerste plaats: het wil de privacy van het individu beschermen. Volgende data valt dan ook onder de GDPR-wetgeving:

- Persoonlijke identificeerbare informatie: namen, adressen, geboortedatum en rijksregisternummer.
- Op internet gebaseerde gegevens: zoals gebruikerslocatie, IP-adres en cookies.
- Gezondheids-en genetische gegevens.
- Biometrische gegevens (denk maar aan je irisscan of vingerafdruk).
- Raciale en/of etnische gegevens.
- Politieke meningen.
- Seksuele geaardheid.

Meldplicht datalekken

Bedrijven die onder de GDPR-wetgeving vallen (alle bedrijven die data van Europese burgers bewaren vallen hier onder, ongeacht hun locatie in de wereld) hebben een meldplicht indien zij een datalek hebben.



Onder een datalek verstaan we *“Een inbreuk op de beveiliging die per ongeluk, of op onrechtmatige wijze leidt tot de vernietiging, het verlies, de wijziging of de ongeoorloofde verstrekking van of de ongeoorloofde toegang tot doorgezonden, opgeslagen of anderszins verwerkte gegevens.”*

Van zodra het bedrijf weet heeft van een (mogelijk) datalek dienen zij dit ogenblikkelijk te melden:

- Aan de **toezichthoudende autoriteit**: Ieder land heeft z'n eigen autoriteit en in België is dat de **CBPL**, de *Commissie voor de bescherming van de persoonlijke levenssfeer*. Deze commissie zal iedere datalek *case-by-case* beoordelen om te zien of het bedrijf een GDPR-overtreding heeft begaan of niet.
- Aan **ieder individu waar de data betrekking op had**: als de gelekte data ook maar op één manier kan gelinkt worden aan een individu, dan dient deze van de lek op de hoogte gebracht te worden.
- Dit dient **binnen de 72 uur na ontdekking van het datalek te gebeuren**.

Merk op dat tegenwoordig een datalek zelden zo snel wordt ontdekt, dat beseft GDPR ook. Daarom is de regel dat 72 uur ingaan vanaf het moment dat het bedrijf weet heeft van het datalek. Het is dus perfect mogelijk dat de datalek reeds maanden eerder plaatsvond maar dat het bedrijf het nu pas ontdekt.

Uitzonderingen meldplicht individu

Bij een datalek zijn er enkele mogelijke uitzonderingen waarom het bedrijf niet noodzakelijk het individu op de hoogte moet stellen van de datalek:

- Indien het bedrijf kan aantonen dat de gestolen data zodanig beveiligd is dat deze onbruikbaar is. Als dus de data bijvoorbeeld geëncrypteerd is dan zou het bedrijf dit als argument kunnen gebruiken om de meldplicht te verzaken. Een kanttekening is hier natuurlijk op z'n plaats: er bestaat altijd nog de mogelijkheid dat de data finaal nog kan gedecrypteerd zal worden en er dus alsnog private gegevens in verkeerde handen komen.
- Stel dat het bedrijf de persoonlijke voorkeuren van de gebruikers in een database heeft staan en de identificeerbare persoonsgegevens in een andere. Als nu blijkt dat enkel de database met persoonlijke voorkeuren werd gehackt, dan kan deze data nooit gelinkt worden aan personen en is er dus ook geen sprake van een privacy schending.



Het is sowieso altijd een goede gewoonte om het adagio “*don't put all your eggs in one basket*” te hanteren. Het is wijzer om de data over verschillende databases te bewaren. Zo voorkom je dat bij een datalek automatisch steeds alle data wordt gestolen.



Indien er een erg grote hoeveelheid mensen moeten ingelicht worden na een datalek, dan mag het bedrijf ook beslissen om in de plaats daarvan een publieke aankondiging te doen. Uiteraard zijn de bedrijven hier minder happig op omdat dit sowieso een PR-nachtmerrie is (iedere aankondiging van een datalek is dat trouwens).

Boetes

Als er uiteindelijk toch een inbreuk op de GDPR-wetgeving wordt vastgesteld dan zal er dus een boete opgelegd worden afhankelijk van de ernst van de inbreuk:

- Minder ernstige inbreuken: boete tot € 10 miljoen, of 2% van de wereldwijde jaaromzet van het bedrijf uit het voorgaande boekjaar, afhankelijk van welk bedrag hoger is.
- Meer ernstige inbreuken (inbreuken die ingaan tegen de principes van het recht op privacy en het recht om te worden vergeten, die de kern vormen van GDPR): boete van maximaal € 20 miljoen, of 4% van de wereldwijde jaaromzet, afhankelijk van welk bedrag hoger is.

Of en hoe groot de boete zal zijn is gebaseerd op een aantal criteria:

- Ernst en aard van de overtreding.
- Intentie: is de datalek het gevolg van een slordigheid of opzettelijk gebrek aan beveiliging?
- Schadebeperkende omstandigheden.
- Genomen voorzorgen.
- Geschiedenis van overtredingen.
- Medewerking: in hoeverre werkt het bedrijf mee met de toezichthoudende autoriteiten?
- Gegevenscategorie
- Kennisgeving: heeft men zich aan de meldplicht gehouden?
- Certificering: heeft het bedrijf in het verleden reeds via een externe audit aangetoond aan de GDPR wetgeving te voldoen?
- Verzwarende of verzachtende factoren.



Merk dus op dat het schenden van de GDPR wetgeving niet automatisch in een boete resulteert. Als het bedrijf kan aantonen dat ze echt alles in het werk hebben gesteld om de data te bewaren zoals een goede huisvader betaamt, dan kan het zijn er geen boete zal volgen.



De voorbije jaren (mei 2018 tot eind 2020) werden reeds 272 miljoen euro aan boetes geïnd ten gevolge van GDPR overtredingen. De wet is zo effectief, dat ze ook ondertussen dient als inspiratie wereldwijd wanneer landen of groeperingen nieuwe privacy-wetgevingen willen construeren.

Cryptografie

Waarom cryptografie?

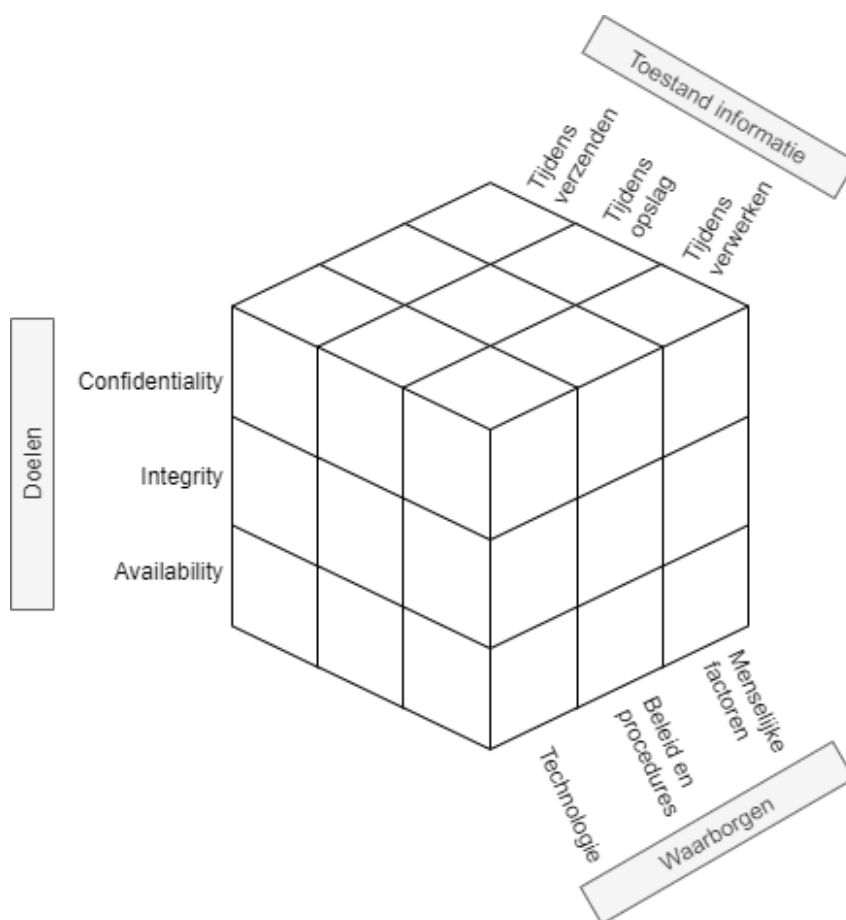
CIA en het security model

Data (of informatie), in welke vorm dan ook (berichten over een netwerk, bestanden op een harde schijf, tekst in een database), moet beschermd worden, dat beseffen we nu. Het doel van onze data is dat deze voldoet aan het acroniem **C.I.A** wat staat voor:

- **C** voor “**confidentiality**”: vertrouwelijkheid. De data kan enkel door zij die er recht toe hebben gebruikt worden. We gaan dit onder andere oplossen met behulp van encryptie en paswoorden.
- **I** voor “**integrity**”: integriteit. We moeten weten of onze data onbeschadigd is en niet werd aangepast door derden (of storingen). Bij bestanden gaan we bijvoorbeeld werken met zogenaamde (secure) hashes.
- **A** voor “**availability**”: beschikbaarheid. Data die niet door rechtmatige gebruikers kan bereikt worden is onbestaande data. Zogenaamde “denial-of-service” (dos) aanvallen hebben als doel deze pijler van CIA aan te vallen. availability is een breed veld en wordt onder andere opgelost door backups, redundante servers enerzijds, en preventieve maatregelen anderzijds zoals firewalls, load balancers, etc.

De zogenaamde McCumber kubus, ontwikkeld door John McCumber in 1991, geeft een goed beeld weer waarom het steeds belangrijk is goed te beseffen binnen welke context we praten. Deze kubus stelt een model voor dat kan gebruikt worden om je ervan te vergewissen dat je aan alles denkt wanneer je je informatie wilt beschermen. De kubus bestaat uit 3 dimensies en iedere dimensie bestaat uit een aantal aspecten. **Enkel wanneer we alle aspecten van alle dimensies in onze beveiligingsaanpak voorzien kunnen we hopen dat we onze C.I.A. doelen hebben bereikt.**

Om ons doel te bereiken (C.I.A.) moeten we ervoor zorgen dat we dit toepassen op alle vormen die onze data kan hebben (opslag, verzenden, verwerken). Dit kunnen we tewerkstellingen door technologische oplossingen (zoals encryptie wat zo meteen wordt uitgespit), maar een niet onbelangrijke factor zijn ook de mensen die met de data moeten werken. Als zij zich niet aan de afspraken (procedures) houden en hun paswoorden gewoon op post-its aan hun scherm hangen, dan mag je een nog zo’n dure firewall hebben, het zal niet baten.



Figuur 0.1: De McCumber kubus

Encryptie

Een grote pijler van CIA, confidentiality, wordt opgelost met behulp van encryptie, namelijk het versleutelen van onze data met behulp van een geheime sleutel. Door deze te versleutelen wordt deze onleesbaar voor personen die de geheime sleutel niet hebben (en bijgevolg niet geautoriseerd zijn om de data te mogen lezen). De moeilijkheid van een goed cryptografisch systeem is dat de data op een zodanig manier met versleuteld worden dat het quasi onmogelijk is om zonder sleutel de originele data terug te vinden. We spreken hierbij over de originele data als de *plaintext* en de geëncrypteerde data als *ciphertext*. De ontvanger van een ciphertext moet deze, als hij de juiste sleutel heeft, terug kunnen omzetten naar de originele plaintext.

Er zijn al veel encryptie algoritmes de revue gepasseerd doorheen de geschiedenis van de mens. Al van in de tijd van de Romeinen werd er aan cryptografie gedaan. Mensen hebben altijd gevoelige data gehad waar vertrouwelijk mee moest om gesprongen worden. Naarmate de **cryptanalyse** (dat is het

proberen ontcijferen van een ciphertext zonder dat je de geheime sleutel hebt) evolueerde moesten ook de cryptografische algoritmes verbeteren. Ook hier zien we weer diezelfde wedloop tussen digitale stropers en cyberboswachters. Hoe sterker onze computers worden (met dank aan de wet van Moore) hoe krachtiger onze algoritmes moeten worden. De eenvoudigste vorm van cryptanalyse, bruteforcing, is rechtstreeks afhankelijk van de snelheid van de computer. Hoe meer sleutels per seconde een computer kan testen, hoe sneller de originele sleutel kan gevonden worden.



De volledige geschiedenis van de cryptografie hier vertellen zou ongeveer 1200 pagina's vereisen. Het briljante boek "The Codebreakers" van David Kahn is een aanrader voor eenieder die meer willen weten over deze boeiende geschiedenis. Laat de 1200 pagina's je niet afschrikken, het boek leest als een echte thriller.

Alle bestaande cryptografische systemen kunnen op verschillende manieren gekarakteriseerd worden (bron Network Security Essentials, door William Stallings):

- De **acties** die op de data wordt uitgevoerd om deze te encrypteren:
 - Substitutie: een teken door een ander teken vervangen.
 - Transpositie: een teken naar op een andere plek in de tekst zetten.
 - Product: een combinatie van meerdere substituties en transposities.
- Het **aantal sleutels** dat nodig is:
 - 1 sleutel, ook wel "private encryption" genoemd.
 - 2 sleutels, ook wel "public encryption" genoemd.
- De **manier** waarop de data wordt verwerkt:
 - Als een blok data, blok per blok.
 - Als een stream, teken per teken.

Kerckhoffs principe

We gaan zo meteen bekijken hoe encryptie effectief gebeurt, maar we willen al even een mythe onderuit halen. Binnen encryptie heb je de *principes van Kerckhoff*, 6 regels die in de 19e eeuw werden vastgelegd waarin encryptie-algoritmes moeten voldoen om als veilig beschouwd te worden. Sommige principes zijn, onder ander door onze steeds krachtige computers, niet meer relevant, meer 1 blijft ongelooflijk belangrijk: " het kennen van het gebruikte encryptiealgoritme door de *tegenstander* is geen probleem, het is enkel de geheime sleutel die ten allen tijde uit de handen van de tegenstander moet blijven."

Dit ogenschijnlijk eenvoudige zinnetje is veel zeggen: **je sleutel (of paswoord) is wat je het beste moet beschermen. Zonder kennis van de sleutel zou iemand nooit toegang mogen krijgen tot versleutelde data, ongeacht dat geweten is met welk systeem de data werd gecijferd.**

Security through obscurity is al lang een dubbel snijdend zwaard binnen de security wereld. Enerzijds is het niet aangeraden om met veel tamtam aan te kondigen hoe jij je data beveiligd. Anderzijds geeft het je mogelijk een vals gevoel van veiligheid (*snake's oil*) daar het geheimhouden van je algoritme en systemen geen garantie is dat deze ook effectief veilig zijn. In de 21e eeuw zijn de meest gebruikte encryptie-algoritmes publiek gekende algoritmes die door duizenden experts aan de tand zijn gevoeld. De kans dat er dus (bewuste) fouten in dergelijke standaarden zit is véél kleiner dan wanneer je met een zogenaamd *proprietary* systeem werkt waarvan de werking angstvallig geheim wordt gehouden.

Finaal draait alles op het geheimhouden van je sleutel, iets dat we telkens weer in dit boek zullen herhalen!



Let op encryptiesystemen die zichzelf verkopen met zinnen zoals “10 jaar nodig op een gewone laptop om alle sleutels te testen”. Dit zou kunnen doen vermoeden dat je dus voor minstens 10 jaar goed zit (we gaan er even vanuit dat de gemiddelde cryptanalist maar toegang heeft tot 1 laptop, wat uiteraard in de echte wereld niet zo is). De gemiddelde tijd van voorgaande systeem om te bruteforcen is 5 jaar, de helft.

Stel dat je een sleutel hebt die bestaat uit 8 karakters. Een karakter is een letter van a tot z (geen onderscheid tussen hoofd en kleine letters en geen getallen of speciale tekens). Er zijn 26^8 mogelijke sleutels (we gaan ervan uit dat de sleutel exact 8 karakters moet bevatten). Een computer kan 1 miljoen sleutels per seconde testen. De duur om alle mogelijke sleutels te testen is dus $(26^8)/1\,000\,000$, oftewel 208 827 seconden, pakweg 58 uur. Intuïtief zou je kunnen denken dat je dus meer dan 2 dagen “veilig” zit, wat niet zo is. De kans dat de eerste sleutel die je test reeds de juiste is, is even groot als dat het de laatste sleutel is. Kortom, gemiddeld gezien zal de sleutel in de helft van de maximum tijd gevonden was, oftewel 29 uur.

De eerste algoritmes

Het doel van ieder encryptie-algoritme is dus om data zodanig te versleutelen zodat enkel eigenaars van de gebruikte sleutel de originele tekst kunnen terugvinden. We vertelden net dat encryptiealgoritmes kunnen onderverdeeld volgens de actie die ze uitvoeren: substitutie, transpositie of een combinatie. We tonen van iedere variant nu een historisch voorbeeld.



Volgende tool, speciaal gemaakt om crypto te leren, is een erg handig iets om de verschillende cryptografische systemen te visualiseren én testen: [link](#)

Substitutie: Caesar encryptie

De Caesar encryptie (naar Julius Caesar) bestaat uit een eenvoudige substitutie algoritme. De sleutel is een getal tussen 1 en 25 en geeft aan door welk element uit het alfabet een teken wordt aangepast, als volgt:

- Ieder element wordt voorgesteld als een cijfer. A krijgt de waarde 1, B wordt 2,... Z wordt 26.
- Als de sleutel het getal 3 is, dan zal nu iedere letter A in de tekst vervangen worden door het teken 1+3, dus D. Iedere B wordt een E, enzovoort.
- Indien er een “overflow” is achteraan komen we uiteraard terug naar voor in het alfabet. Iedere Z wordt dus een C, iedere Y een B, enzovoort.



De modulo operator (%) is erg nuttig bij substitutie-algoritmes zoals bij Caesar encryptie. De modulo operator geeft de rest weer wanneer de linkse door de rechtse operator zouden delen. $19\%5$ geeft dus 4 als resultaat. Je kan de operator gebruiken om snel te weten wat de waarde van een teken wordt bij Caesar-encryptie als volgt:

`teken % sleutel => nieuw teken`

Als je dus een sleutel hebt met waarde 7 en je wilt weten wat de waarde van Y (element 25) wordt dan schrijf je:

`25 % 7`

Dit zal dus 4 worden, oftewel een D.

Het Caesercipher wordt ook wel kortweg *Rot* genoemd, naar het woord *rotatie*. Een cijfer erachter geeft dan aan welk de te gebruiken sleutel is. Rot4 wil dus zeggen dat alle elementen 4 plaatsen opgeschoven moeten worden.



Merk op dat Rot13 (ook wel *Caesaralfabet* genoemd) een speciale sleutel is. Als je namelijk 2 maal na elkaar Rot13 toepast op een tekst (eerst op de plaintext, dan op de resulterende cipertext) dan verkrijgt men terug de originele tekst.

Uiteraard kan iedere weldenkende mens in de 21e eeuw een Caesar-encryptie bruteforcen. Het aantal mogelijk sleutels beperkt zich tot 25 mogelijkheden (sleutel 26 zal resulteren in géén encryptie: je plaintext en cipertext zullen identiek zijn) en je kan dit dus snel testen.

Door **frequentieanalyse** op de ciphertext toe te passen kan men ook de plaintext terugvinden zonder te moeten bruteforcen. Indien de plaintext een tekst in ,bijvoorbeeld, het Nederlands is, dan kunnen we gebruik maken van de statistische eigenschappen van een taal. Zo weten we dat bepaalde letters in een standaard Nederlandstalige tekst meer of minder vaak voorkomen. De letter e komt bijvoorbeeld veel vaker voor dan de v. Daar iedere letter in de encryptie door een andere wordt vervangen, is het dus voldoende om te ontdekken (a.d.h.v. frequentieanalyse) welke letter(s) het meest of minst voorkomen om je zo een vermoeden te geven van de originele letter.



Dit verklaart ook waarom je best je te encrypteren berichten zo kort mogelijk houdt. Hoe minder tekens, hoe minder goed je frequentieanalyse zal werken. Een andere veelgebruikte fout (in klassiekere encryptie) was dat de verzender bijvoorbeeld voorspelbare tekst ging encrypteren. Als je weet dat de verzender altijd begint met “Geachte” in z’n berichten, dan is de kans groot dat de eerste 7 tekens in de ciphertext deze plaintext voorstellen.

Het principe van Caesar-encryptie, de substitutie, blijft echter overeind staan en zal je nog zien terugkomen in de komende algoritmes.

Transpositie: scytale encryptie

Bij transpositie-algoritmen gaan we de positie van de karakters veranderen. De sleutel kan hierbij bepalen op welke manier dit moet gebeuren. De Oude Grieken gebruikten een zogenaamde scytale om aan transpositie-encryptie te doen. Een scytale was een lange stok bestaande uit 3 of meerdere lange zijden. De boodschap werd op een lang lint geschreven en dit lint werd dan over de skytale gedraaid. De sleutel gaf aan uit hoeveel vlakken de te gebruiken scytale moest bestaan. Ieder karakter van de plaintext (het lint) kwam op een andere zijde te liggen. Vervolgens werden alle letters op 1 zijde achter elkaar gezet, en dit werd herhaald voor iedere zijde: dit werd de ciphertext die werd doorgestuurd.

Om de nu de cipertext decrypteren werd een onbeschreven lint over de juiste scytale gelegd. Vervolgens werd de verkregen op dit lint, zijde per zijde, beschreven. Als de ontvanger dan het lint ontrolde kreeg hij terug de originele tekst te zien.



Het voordeel van een transpositiecipher is natuurlijk dat een zogenaamde “known plaintext” aanval iets moeilijker wordt. Een veel gebruikte manier die cryptanalisten vroeger gebruikten is de kennis die ze hadden van delen van de plaintext. Als geweten was dat het bericht altijd begon met “Beste dokter” dan was dit al een goede aanzet om van hieruit verder te werken.

Uiteraard zijn er tal van varianten mogelijk om transpositie te doen. Eerst kan je beslissen om je plaintext in een bepaalde vorm te plaatsen: bijvoorbeeld in 10 kolommen. Vervolgens kan je dan, gebaseerd op de sleutel, beslissen in welke volgorde je de kolommen achter elkaar plaatst om de originele tekst te krijgen. Dit is een zogenaamd *route cipher* wat onder andere werd gebruikt tijdens de Amerikaanse Burgeroorlog.

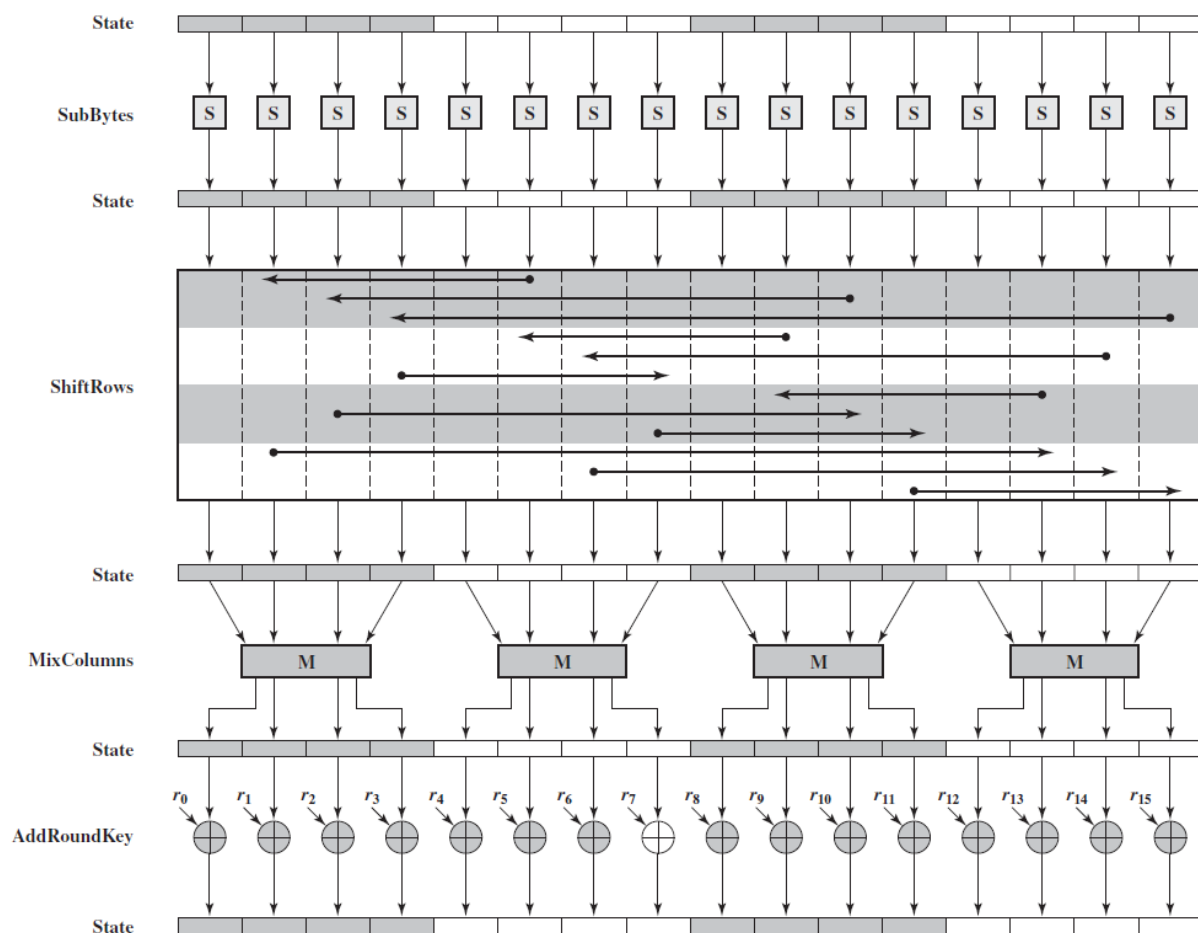


Figuur 0.2: Bron Wikipedia

Combinatie

Het spreekt voor zich dat een combinatie van een transpositiecipher en een substitutiecipher je encryptie nog versterkt. Veel moderne algoritmen kunnen nog steeds herleid worden tot een sequentie van meerdere basisvormen na elkaar.

De **Advanced Encryption Standard (AES)** is in de 21e eeuw zo'n beetje de de facto standaard als het aankomt op symmetrische encryptie (d.w.z. encryptie waar maar 1 sleutel voor nodig is, verder meer hierover). Als we echter eens het algoritme opengooien en een enkele *encryption round* bekijken (AES bestaat uit een sequentie van deze rondes) dan zien we dat de bits die bovenaan binnenkomen (*state*) vervolgens een combinatie van substituties (*sub*) en transposities (*mixcolumns* en *shiftrows*) ondergaat.



Figuur 0.3: Bron Wikipedia

We gaan AES nog terugzien opduiken wanneer we gaan bekijken hoe draadloze netwerken worden beveiligd. Als Belg mogen we trouwens erg fier zijn op deze wereldwijd gebruikte Amerikaanse standaard. Je zal later ontdekken waarom dat zo is!



Doel van dit hoofdstuk is ook aantonen dat je geen wiskundig wondertalent moet zijn om de basisconcepten van cryptografie te begrijpen. Hier en daar neem ik wat vrijheden om bepaalde stappen te vereenvoudigen, maar de essentie van de algoritmen blijft wel bewaard en daarmee, hopelijk, ook de eenvoudigheid (en dus elegantie) ervan.

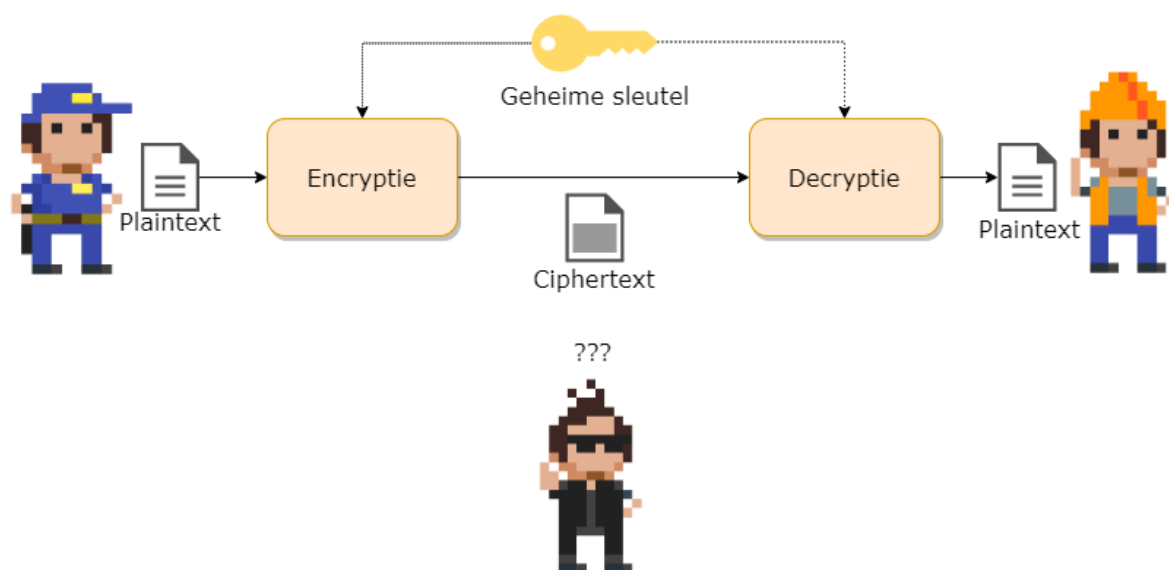
Cryptanalyse

TODO:

- Sleutel lengte en permutaties berekenen afh van alfabetgrootte
- Tijd om te bruteforcen tabel
- cryptanalytics attack
- cryptanalytic attacks vs bruteforce
- Bruteforce vs dictattack
- Noot omtrent quantum computers

Symmetrische encryptie

Er zijn 2 soorten encryptiesystemen als we kijken naar het aantal sleutels. Symmetrische systemen zijn systemen waarbij maar 1 sleutel nodig is: zowel ontvanger als verzender gebruiken dezelfde sleutel. De term symmetrisch verwijst naar het feit dat het algoritme exact hetzelfde doet aan beide zijden. Het enige verschil is dat bij de verzender de plaintext in het systeem wordt gestoken, wat resulteert in een ciphertext. Terwijl de ontvanger de ciphertext in het systeem plaatst om een plaintext te krijgen.



- De symmetrische encryptiesystemen zijn de oudste vorm: alle klassieke algoritmes waren van dit principe. Asymmetrische systemen zijn pas in de 20e eeuw ontwikkeld (circa 1970).
- Voorbeelden van bestaande symmetrische encryptiesystemen zijn: AES, DES, IDEA, RC4, Blowfish, etc.

Dit type encryptie is nog steeds het meest gebruikt en wordt overal gebruikt waar data op een veilige (confidentiality) moet bewaard, verstuurd of verwerkt worden.

Sleuteloverdracht

De moeilijkheid bij symmetrische systemen is de sleuteloverdracht. Daar ontvanger en verzender dezelfde sleutel hanteren is het natuurlijk belangrijk dat deze de sleutel op een veilige manier kunnen uitwisselen. Dit probleem wordt niet opgelost door symmetrische cryptosystemen. Afhankelijk van de context kan deze uitwisseling op verschillende manieren gebeuren:

- Via een asymmetrisch encryptiesysteem dat wél sleutels op een veilige manier kan uitwisselen (zie verder).
- Via een beveiligd kanaal, in eender welke vorm (bijvoorbeeld fysiek de sleutel aan de andere persoon geven of zeggen, deze opsturen via een reeds bestand opgezet symmetrisch encryptie-kanaal, etc.)

Block en Stream cipers

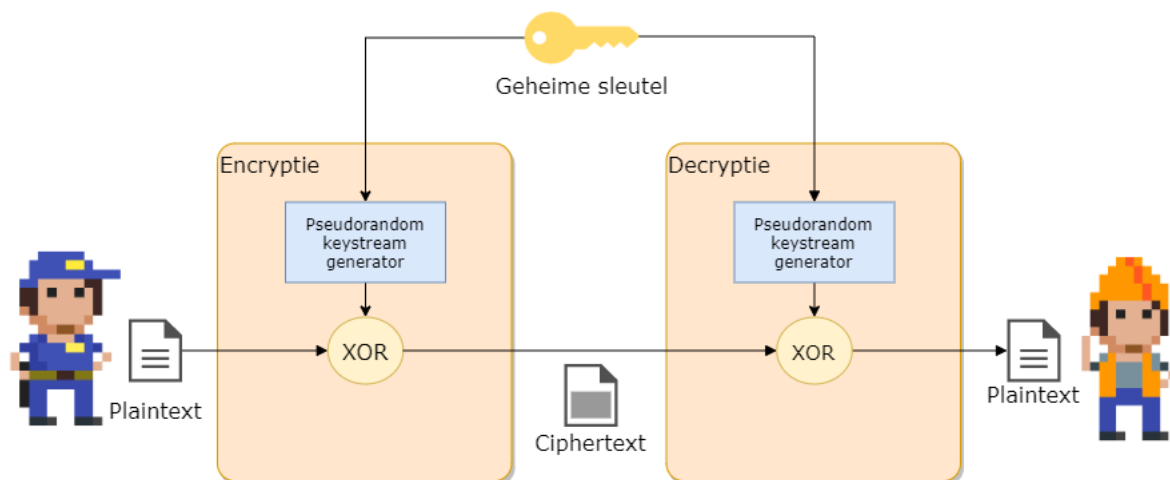
Er zijn 2 soorten symmetrische encryptie-ciphers als we kijken naar de manier waarop ze de te encrypteren data verwerken:

- **Streamciphers:** hierbij wordt de data letterlijk als een stream van tekens, teken per teken geëncrypteerd (RC4, etc.). Voor ieder teken dat verwerkt wordt zal er exact één geëncrypteerd teken gegenereerd worden. Dit soort algoritmes zijn over het algemeen sneller dan de blockciphers.
- **Blockciphers:** de data wordt in blokken (van bijvoorbeeld 128 tekens) verwerkt (AES, DES, 3DES, etc.) Deze vorm komt heden ten dage vaker voor.

Streamciphers

De werking van een symmetrisch stream cipher is verrassend eenvoudig en bestaat uit 2 delen:

- Een pseudorandom keystream generator: deze zal de sleutel als het ware expanderen naar een sleutel met de zelfde lengte als de stream, genaamd een **keystream**. Daar we met een stream werken zal deze generator teken per teken genereren. Hoe dit gebeurt leggen we verderop uit.
- De **xor** of “exclusieve of” functie: deze zal de plaintext naar een ciphertext omzetten door de plaintext met de keystream samen te voegen.



Aan de ontvanger zijde gebeurt exact hetzelfde. **Enkel indien de ontvanger dezelfde sleutel gebruikt, zal deze dezelfde keystream kunnen genereren, en bijgevolg enkel dan de originele plaintext verkrijgen.**

Het hart van een symmetrisch streamcipher is dus enerzijds de xor-functie én, belangrijker, de manier waarop de keystream wordt gemaakt.

De XOR functie

De waarheidstabel van de XOR-functie is de volgende:

Plaintext input	Keystream input	Ciphertext output
1	0	1
0	1	1
0	0	0
1	1	0

De xor-functie wordt in schema's aangeduid door een cirkel met een plusje in: $\oplus$

De XOR-functie heeft de fijne eigenschap dat je deze dus voor encryptie kan gebruiken.

Beeld je in dat we het bericht 1010 willen versleutelen, en we hebben een gegenereerde keystream 1101. Als we deze XOR'n dan geeft dit 0111. Dit is dus de ciphertext. Als de ontvanger dezelfde keystream kan genereren en deze xor'd met de verkregen ciphertext, dan krijgt deze terug de originele plaintext.

De keystream generator

De keystream generator heeft dus als doel om voor iedere karakter dat moet geëncrypteerd worden een bijhorend keystream karakter te maken. Deze karaktergeneratie moet onvoorspelbaar zijn (*random*) tegenover de sleutel die wordt gebruikt en het voorgaande karakter dat werd gemaakt. Echter, dit moet wel PSEUDO (*schijn*)-willekeurig zijn: dezelfde sleutel als begintpunt (*seed*) moet dezelfde reeks genereren.

De kracht (en zwakte) van een symmetrisch streamcipher ligt in de implementatie van de manier waarom deze keystream generator werkt. Mogelijke zwakheden kunnen bijvoorbeeld zijn dat de gegenereerde stroom informatie van de sleutel “lekt” naar de keystream (wat desastreuze gevolgen bleek te hebben bij de originele wifi-security (WEP) waarover later meer) of een voorspelbare “randomiteit” van de keystream?



Om aan encryptie te kunnen doen hebben we systemen nodig die onvoorspelbaar zijn. Als de aanvaller kan voorspellen wat de uitvoer van een onderdeel van de encryptie zal zijn, dan kunnen we geen confidentiality en/integrity voorzien. Kortom, we hebben algoritmes nodig die willekeurige getallen kunnen generen die 100% onvoorspelbaar zijn. Net zoals het werpen van een dobbelsteen niet voorspeld kan worden, zo ook moeten onze algoritmes een (digitale) dobbelsteen hebben.

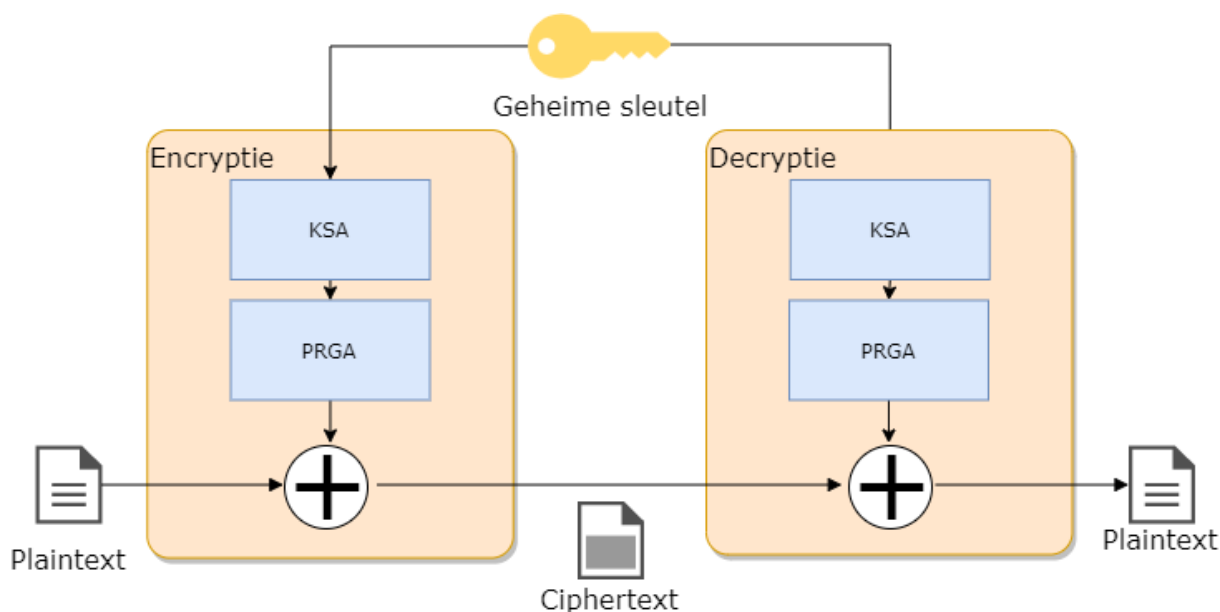
Digitale systemen die perfect willekeurige getallen genereren noemt men **random number generator** (RNG). Uiteraard moet een RNG geprogrammeerd kunnen worden: dat behelst dus een algoritme. Een algoritme is per definitie “voorspelbaar”. Alles hangt daarom af van de invoer die het algoritme gebruikt om random getallen te beginnen genereren. We spreken dan van een **pseudo random number generator** (PRNG), pseudo (**schijnbaar**) omdat de uitvoer afhankelijk is van het startgetal, de zogenaamde **seed**. Die seed kan bijvoorbeeld de encryptiesleutel zijn: enkel met dié sleutel zal het algoritme dezelfde reeks getallen generen. Er zijn echt ook systemen die bijvoorbeeld de staat van een flipflop als startpunt gebruiken: wanneer je een flipflop aanzet kan je niet voorspellen of deze op 1 of 0 zal staan (daar deze staat beïnvloed wordt door de elektromagnetische straling). Uiteraard is een dergelijke seed voor een keystreamgenerator nutteloos, daar zowel verzender én ontvanger dezelfde reeks getallen moeten kunnen genereren.

RC4 tot op het bot

Laten we eens één van de meest gebruikte streamciphers bekijken, het RC4 cipher. Dit algoritme, ontwikkeld Ron Rivest (de afkorting staat trouwens voor *Rons Cipher 4*), wordt gebruikt onder andere om een beveiligde SSL-tunnel (zie later) op te zetten en zit in het hart van veel geëncrypteerde

communicatiekanalen. Het heeft weliswaar enkele zwakheden, maar mits juist toegepast kunnen die grotendeels omzeild worden.

RC4 werkt zoals we eerder verklaarden hoe een stream cipher werkt: het heeft een keystreamgenerator en zal de keystream vervolgens XO'r met de plaintext. Eerst zal de ingevoerde sleutel (die 40 tot 2048 bits lang mag zijn) omgezet worden naar een compatibele werksleutel met behulp van een **Key scheduling algorithm** (KSA). Deze werksleutel zal dan als seed gebruikt worden om een keystream in het **Pseudo-random generator algorithm** (PRGA) te maken.



KSA De **KSA** heeft al doel om, de ingevoerde 40 tot 2k-bit sleutel om te zetten naar een compatibele sleutel voor de PRGA . Het doet dit volgens een eenvoudig algoritme:

Stap 1: plaats de ingevoerde sleutel in een array (**T**) van 256 karakters. Herhaal de sleutel indien nodig.

Stap 2: Maak een array (**S**) aan, ook van 256 karakters, en plaats er de waarden 0 tot en met 255 in.

Stap 3: permuteer (*transpositie*) de elementen in de array T met behulp van de array S als volgt:

```
j = 0
for i = 0 tot 255
{
    j = (j + S[i] + T[i]) % 256
    Verwissel(S[i],S[j])
}
```

Deze stap zal dus de elementen in de sleutelarray S naar nieuwe posities in de T array plaatsen en deze ook de hele tijd van plek wisselen. De index j is afhankelijk van de sleutelwaarde en zorgt er dus voor dat iedere sleutel een unieke array T zal opleveren.

Uitgewerkt voorbeeld van KSA Stel dat onze sleutel “ab” is. De decimale ASCII-waarden van a en b zijn 97 en 98 respectievelijk. Onze array T zal dus bestaan uit 128 keer de waarden 97 en 98 na elkaar aan de start:

Index	Waarde
S[0]	97
S[1]	98
S[2]	97
etc.	

Stap 2 genereert de tabel T:

Index	Waarde
T[0]	0
T[1]	1
T[2]	2
etc.	

Als we dan stap 3 toepassen dan krijgen we na de eerste iteratie van de loop ($i=0$):

$$j = (0 + S[0] + T[0]) \% 256$$

oftewel

$$j = (0 + 0 + 97) \% 256 \Rightarrow j \text{ wordt } 97$$

De eerste wissel die in S zal plaatsvinden is dan `Verwissel(S[0], S[97])`

S ziet er dan als volgt uit na de eerste iteratie:

Index	Waarde
S[0]	97
S[1]	1
S[2]	2
...	
S[97]	0
etc.	

En dit herhalen we nog 255 keer.

PRGA Nu we een compatibele sleutel T hebben kan de keystream generatie van start gaan. Deze bestaat uit een loop die blijft doorgaan telkens een nieuw plaintext karakter binnenkomt, als volgt:

```
i = 0
j = 0
Herhaal telkens keystream karakter nodig is
{
    i = (i+1) % 256
    j = (j + S[i]) % 256
    Verwissel(S[i], S[j])
    k = S[(S[i]+S[j]) % 256]
    output k naar keystream
}
```

Zoals je ziet zal dus de array S de hele tijd van “gedaante” veranderen (hij blijft bestaan uit de getallen 0 tot en 255 maar al lang niet meer in volgorde) en telkens zal het algoritme één specifieke waarden (tussen 0 en 255) uit de array teruggeven als keystream karakter.

Uitgewerkt voorbeeld van PRGA Als we verder werken met de tabel K van het vorige uitgewerkte KSA voorbeeld, dan krijgen we dus bij iteratie 1 van de PRGA



We veronderstellen even dat we de KSA niet verder hebben uitgevoerd en de tabel S dus dezelfde is gebleven als op het einde van het voorbeeld wat in het echt niet zal zijn.

```
i = (0+1) % 256 => 1
j = (0+97) % 256 => 97
Verwissel(S[1],S[97]) => Verwissel(1,0)
k = S[ (S[1]+S[97])%256 ] => k = S[ (1 + 0) % 256 ]
we outputten de waarde die op S[1] staat
```

Finaal zal deze *geoutputte* waarde *k* ge-xor'd worden met het huidige karakter van de plaintext.



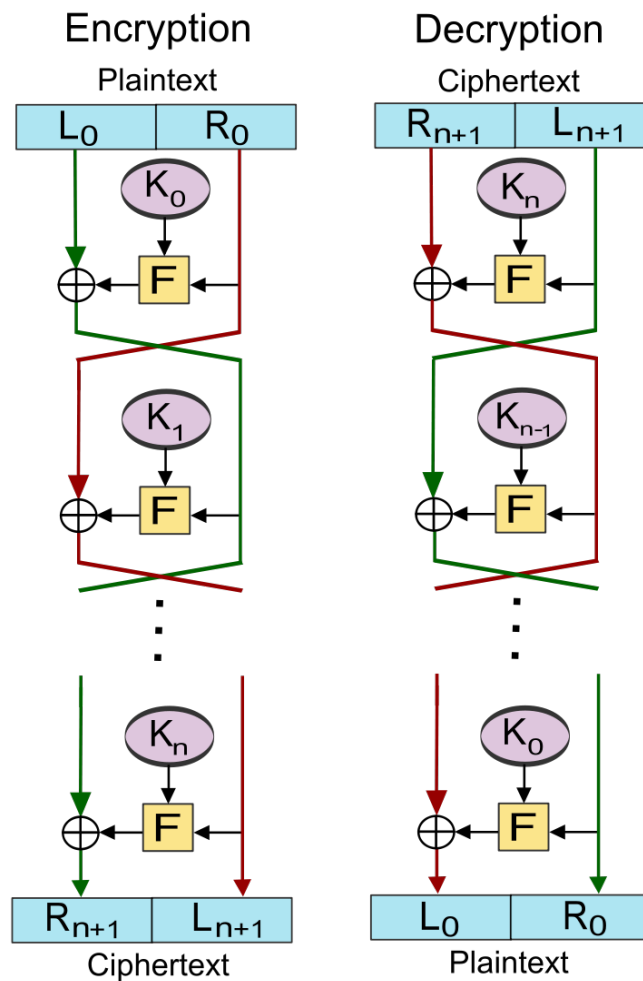
Zo, dat viel nog mee he? Zoals al gezegd, een belangrijke motivatie van dit boek is aantonen dat je niet bang hoeft te zijn van wat er achter de schermen van de cyberwereld gebeurt. De hoeveelheid wiskunde die we bijvoorbeeld nodig hadden is beperkt gebleven tot onze trouwe modulo (%) -operator en meer niet. Wanneer we zo meteen een block ciphers gaan uitkleden zal je ook daar ontdekken dat je best in staat bent schijnbaar complexe technologieën te begrijpen. Hop naar de blockciphers dus!

Blockciphers

Blockciphers, de naam zegt het al, zal eerst de plaintext in blokken karakters opdelen (bv 128 bits) en vervolgens blok per blok encrypteren.

Feistel structuren

Ook hier zullen we dezelfde soorten operaties (XOR, substituties en transposities) zien terugkomen. Echter, ook zogenaamde **Feistel**-structuren worden hier gebruikt: in deze operatie zal steeds de data in 2 helften worden gesplitst en wordt steeds een specifieke operatie (aangeduid met *F* van functie in de figuur), zoals een substitutie, op 1 helft uitgevoerd dat dan wordt ge-xor'd met de andere helft. Dit wordt meerdere keren herhaald, waarbij de linker (*L*) en rechterzijde (*R*) steeds afwisselend door de specifieke encryptie-operatie gaan. Net zoals bij RC4 zullen we ook vaak met een zogenaamd key scheduling algoritme werken zodat de sleutel niet constant doorheen het hele proces dezelfde is en we dus met **subkeys** werken (*K* in onderstaande figuur)



Figuur 0.4: Bron wikipedia

DES tot op het bot

Een van de oudste block ciphers is **DES** oftewel **Data Encryption Standard**. Deze standaard werd in 1977 geboren en vereiste toen blokken van 64 bits data en gebruikte een 56bit sleutel. Ondertussen is dit cipher zeer outdated, maar we gaan hem toch bekijken omdat deze enerzijds erg belangrijk was voor de wereld - grote delen van het bankwezen beschermden er hun financiële transacties mee (en nadien met de opvolger 3DES, zie verder)- en de standaard is erg duidelijk om een goed inzicht in block ciphers te verkrijgen.



Er was veel controverse rond deze standaard (gebaseerd op het door IBM ontwikkelde Lucifer cipher) omdat de originele versie (genaamd Lucifer) werkte met een dubbel zo grootte sleutel (128 bit) en bijgevolg dus veiliger was op lange(re) termijn. De Amerikaanse veiligheidsdienst, NSA, was niet zo happig om een dergelijk goed beveiligd algoritme commercieel te maken en zorgden er daarom voor dat de sleutellengte gehalveerd werd.

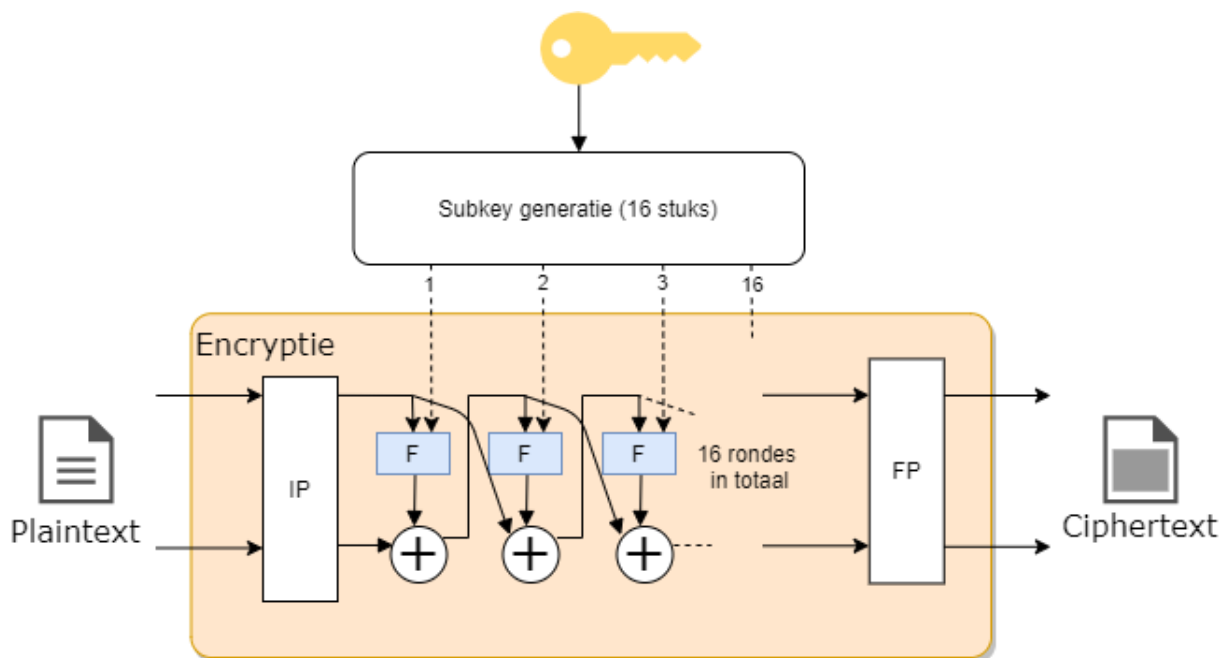
Data met DES encrypteren (en bijgevolg ook decrypteren daar het een symmetrisch cipher is) bestaat uit 2 onderdelen:

1. Versleuteling: Een reeks feistel-structuren na elkaar (**16 rondes**) die de data block per block versleutelen.
2. Subkeys maken: Een key scheduling algoritme dat 16 subkeys genereert (1 voor iedere encryptieronde), gebaseerd op de 56 bit sleutel.



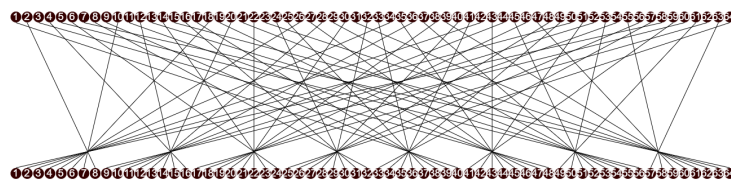
Je kan de DES standaard [hier](#) nalezen en ontdekken dat deze niet zo lang is zoals je zou verwachten van een wereldwijd geadopteerde standaard.

Versleuteling Volgende schema toont de encryptie bestaande uit 16 rondes:



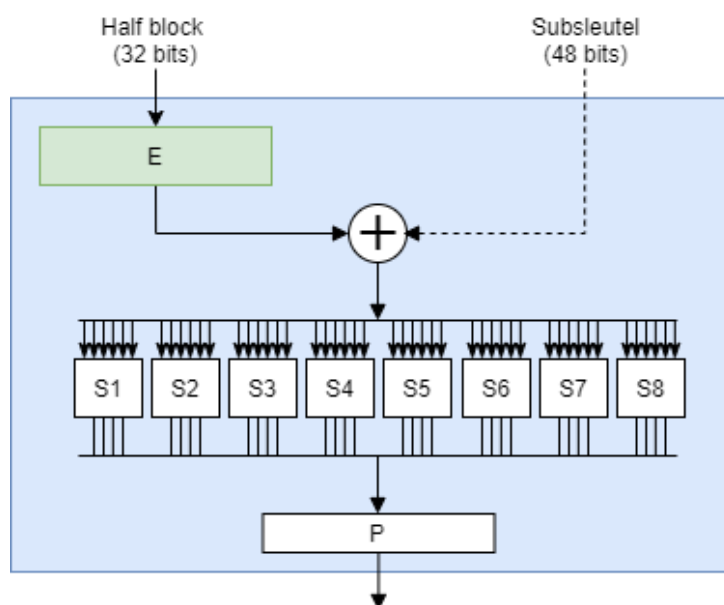
De data wordt blok per blok doorheen dit gedeelte gestuurd. Eerst gebeurt er een zogenaamde *Initiële permutatie* (IP in de figuur) waarbij iedere bit naar een andere plek wordt gestuurd volgens een vast

patroon. Achteraan gebeurt dit nogmaals in een *Finale permutatie (FP)*.



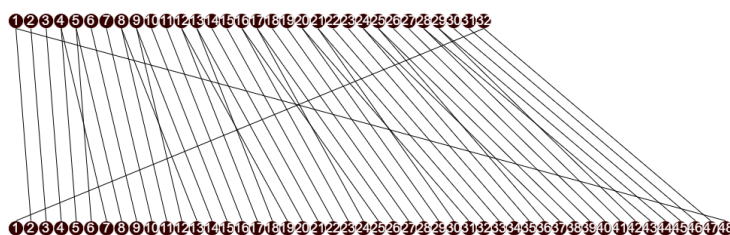
Figuur 0.5: De Initiële permutatie

Na de *IP* gaat de data door 16 feistel structuren die telkens het zelfde doen. De data wordt in 2 helften gesplitst waarbij de rechterzijde door de F-operatie gaat (die we zo meteen toelichten), het resultaat hiervan wordt ge-xor'd met de linkerhelft van de data. Het resultaat van deze XOR, een 32 bit blok, wordt nu het rechterblok in de volgende ronde en omgekeerd.



Figuur 0.6: Binnenin de F-operatie

In het F-blok wordt eerst het 32-bit blok uitgebreid (*E* in de figuur, van expansie) naar een 48 bit blok zodat deze even lang is als de subkey voor deze ronde. De expansie gebeurt, net als de initiële permutatie, volgens een vast patroon:



Figuur 0.7: De expansie van 32 naar 48 bits

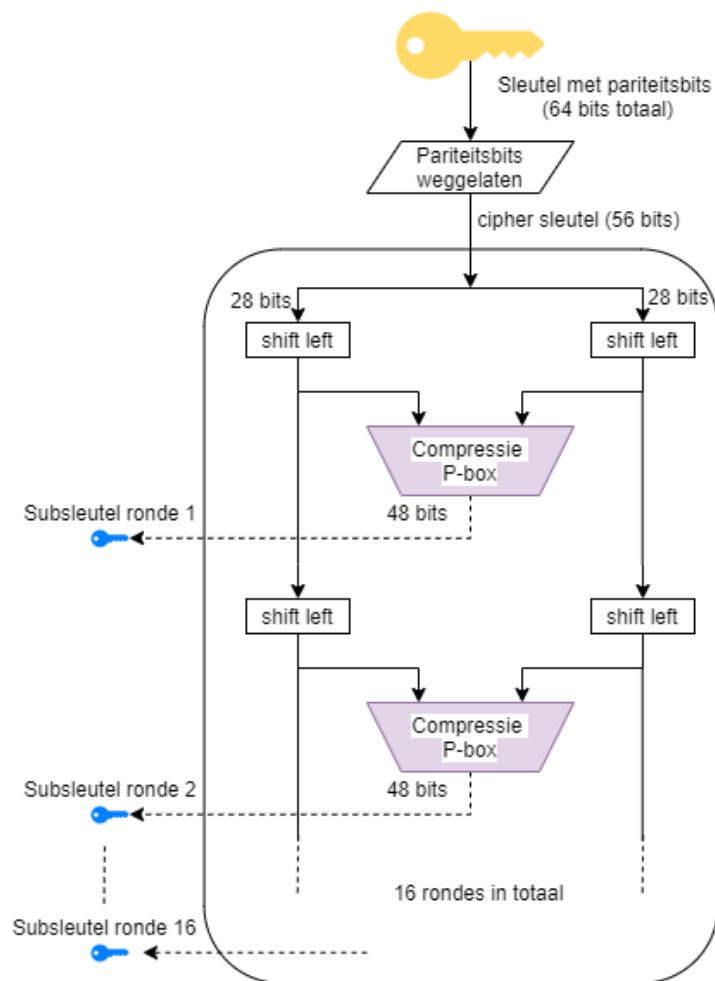
Nu worden deze 48 bits ge-xor'd met de subkey. Het resultaat wordt in blokjes van 6 bits door een *S*-blok gestuurd (zogenaamde *Selection blocks*). In dit blokje wordt 6 bit omgezet naar 4 bit. In de figuur hieronder zien we bijvoorbeeld hoe de omzetting in blok *S5* gebeurt. Ieder blokje heeft een soortgelijke tabel, maar met andere resultaten. De 6 bits bestaan uit de 2 outer bits, namelijk de eerste en de laatste bit, alsook de 4 innerbits. De figuur toont bijvoorbeeld dat de output 1001 zou zijn indien er 011011 in het blok wordt geplaatst.

S ₅		Middle 4 bits of input															
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Outer bits	00	0010	1100	0100	0001	0111	1010	1011	0110	1000	0101	0011	1111	1101	0000	1110	1001
	01	1110	1011	0010	1100	0100	0111	1101	0001	0101	0000	1111	1010	0011	1001	1000	0110
	10	0100	0010	0001	1011	1010	1101	0111	1000	1111	1001	1100	0101	0110	0011	0000	1110
	11	1011	1000	1100	0111	0001	1110	0010	1101	0110	1111	0000	1001	1010	0100	0101	0011

Figuur 0.8: Waarheidstabel van het *S5*-blok (Bron wikipedia)

Finaal krijgen we dus terug een 32 bit blok dat nog een laatste permutatie ondergaat die weer de bits van plaats verandert en de output hiervan is een geëncrypteerd blok data dat kan doorgestuurd worden naar de ontvanger.

Subkeys maken Iedere ronde tijdens de versleuteling vereist een sleutel. Om te voorkomen dat steeds de hoofdsleutel wordt gebruikt (en er zo potentieel dezelfde keystreams worden gemaakt) wordt deze sleutel doorheen een *round-key generator* algoritme gestuurd. Dit algoritme bestaat uit 16 rondes waarbij de 56 bit sleutel (8 van de 64 bits in de originele sleutel zijn zogenaamde pariteits-bits, die dienen om te controleren of de sleutel geen fouten bevat) steeds in 2 helften van 28 bits wordt geknipt.



Figuur 0.9: De 16 rondes die telkens 1 subkey maken

Iedere ronde wordt iedere helft van de sleutel 2 bits *geshift* (in ronde 1,2, 9 en 16 maar 1 bit). Dat wil zeggen dat alle bits 2 plekje opschuiven en de eerste (of laatste) bits komen dan achteraan (of vooraan) te staan. Deze 2 geshifte helften worden dan enerzijds doorgestuurd naar de volgende ronde, anders naar een *compressie P-box* waarvan het resultaat een 48 bits subkey zal zijn van die ronde.

De *compressie P-box* doet al vermoeden wat er gebeurt:

- Compressie: dus een aantal bits zullen wegvallen (er komt 56 bits in, maar we hebben maar 48 bits nodig)
- P-box: een permutatie oftewel transpositie dat alle bits van plek zal veranderen.

Er bestaan verschillende varianten hoe dit in z'n werk zal gaan, maar bij DES wordt volgende tabel gehanteerd:

1	2	3	4	5	6	7	8
14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

Voor zij die graag puzzelen, welke getallen ontbreken?



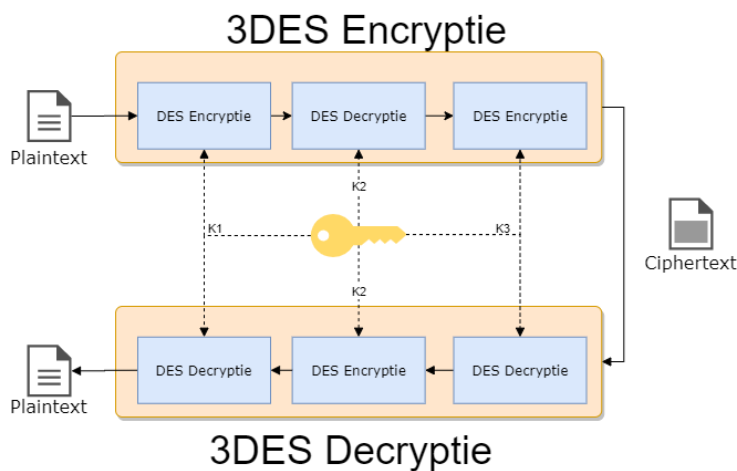
De bits op locaties 9, 18, 22, 25, 35, 43 en 54 worden geblokkeerd. Daar de sleutel steeds geshift wordt wil dit zeggen dat steeds andere bits *achtergelaten* geworden.

En zo hebben we het einde van de werking van DES bereikt. Dat viel al bij al nog mee, niet? Uiteraard hebben we nu vooral getoond *hoe* het werkt, maar niet *waarom* het werkt.

3DES

Al van bij de start gingen er stemmen op dat de originele sleutellengte voor DES (56 bits, 48 in effectiviteit vanwege de pariteitsbits) redelijk snel zou gebruteforced worden. Om die reden werd 3DES in het leven geroepen in 1995. De oplossing was een mooi staaltje compromis: het bood een verhoogde beveiliging doordat het een lange sleutel had (tot 168 bits lang) maar bleef tegelijkertijd compatibel met de bestaande DES hardware en software.

De werking van 3DES (*triple DES*) verrassend eenvoudig: ieder blok data wordt 3 keer doorheen een DES-cipher gestuurd. Hierbij wordt steeds een andere sleutel gebruikt. Om de bestaande DES hardware te gebruiken wordt hierbij de data eerst door de encryptie gestuurd, dan doorheen de decryptie en terug door de encryptie. Daar we een andere sleutel gebruiken in iedere fase heeft dit (dankzij de eigenschappen van symmetrische ciphers) als effect dat we dus effectief 3 maal na elkaar encrypteren met steeds een andere sleutel. Aan de ontvanger zijde gebeurt dan het omgekeerde: decryptie, encryptie, decryptie én dit dus allemaal met de bestaande DES hardware!



3DES laat dus ook (single) DES encryptie toe. Het enige dat je hiervoor moet doen is de subsleutels K2 en K3 gelijkstellen waardoor de 2 en derde fase tijdens de encryptie (en decryptie) eigenlijk niets doet, daar het gewoon de data encrypteerd in ronde 2 en dan ogenblikkelijk in ronde 3 terug gecrypteerd.



Het bankwezen gebruikt 3DES nog steeds (of varianten die erop gebaseerd) zijn om financiële transacties van onder andere Visa en Mastercard te beveiligen.

Block cipher modes

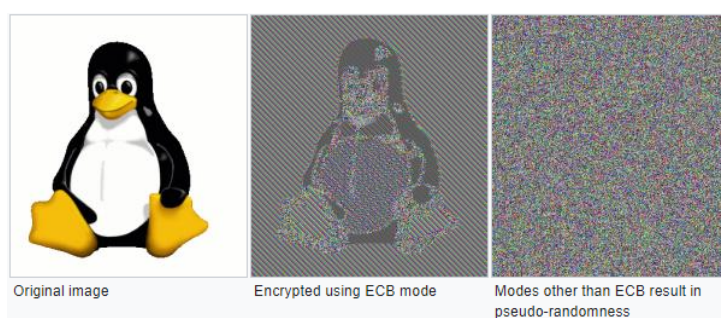
Tot hiertoe gingen we steeds blok per blok in het encryptiecipher sturen en het resultaat ervan doorsturen. Klaar. Oplettende mensen hebben hier mogelijk al een hiaat in gezien: wat als twee blokken exact dezelfde data bevatten? Beide zullen dezelfde ciphertext als resultaat genereren, daar we telkens dezelfde sleutel (en dus subkeys) gebruiken. Twee ciphertexts die identiek zijn willen we vermijden, daar het potentiële informatie over de plaintext zichtbaar maakt. Voorts laat dit soort werking ook replay attacks toe: de aanvaller kan een geëncrypteerd pakket bewaren en op een later moment terug opsturen, zonder dat hij moet weten wat de plaintext bevat.



Door pakketten te sniffen zou de aanvaller kunnen achterhalen wat de mogelijk inhoud van een pakket is. Ieder netwerkprotocol volgt de standaarden die ervoor beschreven zijn en zo kan dus de aanvaller heel veel informatie ontdekken over een ciphertext gewoon ten opzichte van wanneer een pakket wordt verstuurd tegenover de andere. Als het bijvoorbeeld het eerste pakket is dat verstuurd wordt, dan is de kans groot dat dit pakket de typische “*ik wi een verbinding opzetten*”-request is.

Voorgaande modus, waarin we ieder blok onafhankelijk van het vorige encrypteren, noemen we de **Electronic Codebook (ECB)** modus. Alhoewel deze modus dus duidelijk een veiligheidsprobleem met zich mee draagt, heeft deze modus ook één voordeel:

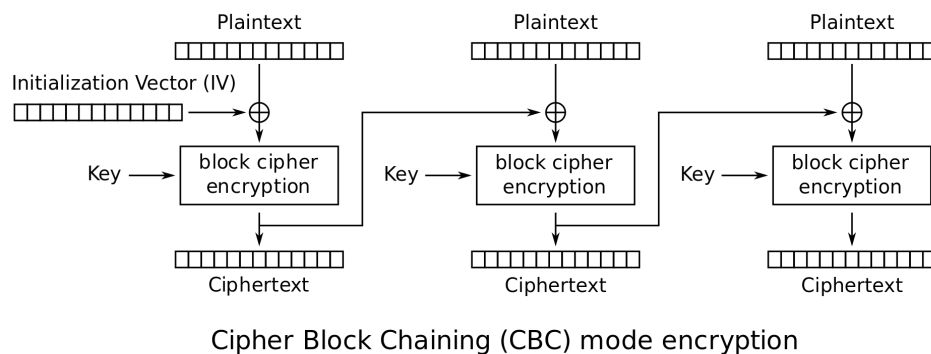
- Ieder blok wordt onafhankelijk van andere blokken gedecrypteerd. Als er dus een blok door wat voor fout niet gedecrypteerd worden, dan heeft dat geen invloed op de daaropvolgende. Dit is dus voor streaming-situaties nuttig: beeld je in dat je decryptie faalt halverwege het binnenkrijgen van een film die je aan het bekijken bent. Je zou helemaal opnieuw moeten beginnen.



Figuur 0.10: Volgende voorbeeld toont een (overdreven) manier waarom ECB minder veilig is dan de modes die we nog gaan behandelen (Bron wikipedia)

ECB is duidelijk een niet zo veilige manier om een block cipher toe te passen. Veel interessanter (veiliger) wordt het wanneer we extra informatie gebruiken om een blok te encrypteren. **Enkel het huidige blok en dezelfde sleutel gebruiken is namelijk níét veilig.** Er zijn verschillende modes om veiliger te encrypteren dan ECB:

- Cipher block chaining (CBC): de output van het vorige block (de ciphertext) wordt mee als input voor de encryptie van het volgende block gebruikt.
- Propagating CBC (PCBC): zelfde als CBC maar bij decryptie van een block zijn ook alle vorige blokken vereist.
- Cipher feedback (CFB): ongeveer zelfde als CBC alleen wordt het vorige block iets later in de encryptie van het volgende block gebruikt.
- Output feedback (OFB): het block cipher wordt als een stream cipher gebruikt.
- Counter-mode (CTR): een extra teller wordt gebruikt als input bij de encryptie van een blok. Deze teller wordt steeds verhoogd. Eén van de meest gebruikte modes (in onder andere WPA2 en IPSEC).



Figuur 0.11: CBC encryptie (Bron wikipedia)

Alle modes uit de doeken doen is hier niet aan de orde maar het moge duidelijk zijn dat ECB de minst veilige mode voorhande is en deze wordt dan ook best vermeden.



Het concept **Initializatie Vector (IV)** zal je veel zien terugkomen in ciphers. Een IV is een getal dat men als extra seed meegeeft tijdens de encryptie, naast de sleutel. Op deze manier voorkomen we dat steeds enkel de sleutel als seed wordt gebruikt en we dus effectief steeds met een *andere* sleutel werken. Uiteraard zal ook de andere zijde over dezelfde IV moeten beschikken en zal deze dus doorgestuurd moeten worden. Dit gebeurt meestal via de header van het bijhorende pakketje en is ongeëncrypteerd. Dit lijkt contra-intuïtief - de IV onbeveiligd doorsturen - maar is geen probleem. Uiteraard is het belangrijk dat er een goed *IV selectie algoritme* wordt gebruikt dat bepaald hoe steeds het volgende IV moet worden berekend (bv steeds met 1 verhogen, een willekeurig, etc.).



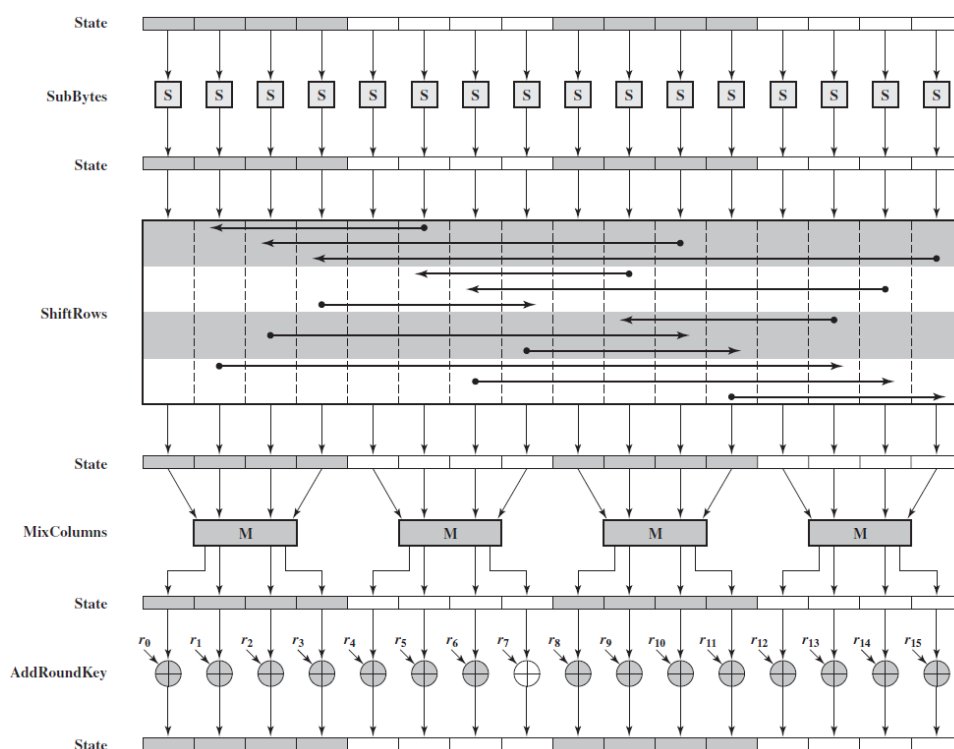
[De wikipedia pagina over block cipher] https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation modes geeft een zeer goed overzicht (én vergelijking).

AES

Alhoewel 3DES een verbetering op DES was, was er toch nood aan een nieuwe encryptie-standaard die langere tijd kon bestaan. In 2001 werd daarom de **Advanced Encryption Standard (AES)** onder het doopvont gehouden als de nieuwe defactor encryptiestandaard wereldwijd. Deze Amerikaanse standaard is gebaseerd op het **Rijndael** algoritme waar we als Belgen fier op mogen zijn: Rijndael is ontwikkeld door 2 Belgische KUL-cryptografen Vincent Rijmen en Joan Daemen.

AES is een symmetrisch block cipher dat data in blocks van 128 bits zal opsplitsen en sleutel van tot 256 bits lang toelaat. De volledige werking van AES gaan we hier niet uit de doeken doen, het voldoet te begrijpen dat in grote lijnen hetzelfde soort stappen worden doorlopen als DES en andere symmetrische ciphers:

- Er is een *key expansie* stap om een unieke sleutel p r ronde te hebben.
- De data wordt doorheen meerdere rondes gestuurd.
- Iedere ronde gebeuren er zaken zoals substituties en transposities, zowel van bytes als van hele rijen of kolommen data.



Figuur 0.12: AES encryptie (Bron wikipedia)

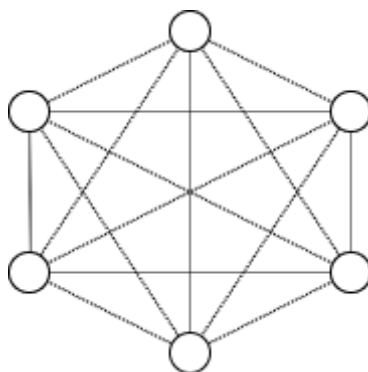
Asymmetrische encryptie

Het probleem met symmetrische encryptie

Wat als je bij symmetrische encryptie met meerdere mensen wilt communiceren zonder dat iedereen elkaars berichten kan zien? Bob kan onmogelijk dezelfde sleutel gebruiken om met Eve te communiceren die hij reeds gebruikte met Alice. Kortom, je hebt per *eindpunt* een aparte sleutel nodig. Het aantal sleutels dat je nodig hebt, zeker als ook alle gebruiker onderling nog eens willen communiceren wordt

erg snel erg groot. Je kan dit berekenen met de formule $n * \frac{(n-1)}{2}$ waarbij n het aantal gebruikers voorstelt:

- 6 gebruikers vereisen 15 sleutels.
- 7 gebruikers vereisen 21 sleutels.
- 10 gebruikers vereisen er al 45.
- 100 gebruikers vereisten er 4950!



Figuur 0.13: Bij 6 gebruikers zijn er al 15 sleutels nodig.

Symmetrische encryptie heeft dus een *key distribution problem* wanneer er encryptie op een grote schaal nodig is. Zeker als we spreken over online communicatie, over het internet, wordt de schaal ogenblikkelijk gigantisch groot en wordt het *sleutelmanagement* problematisch. Een andere oplossing is dus aan de orde.

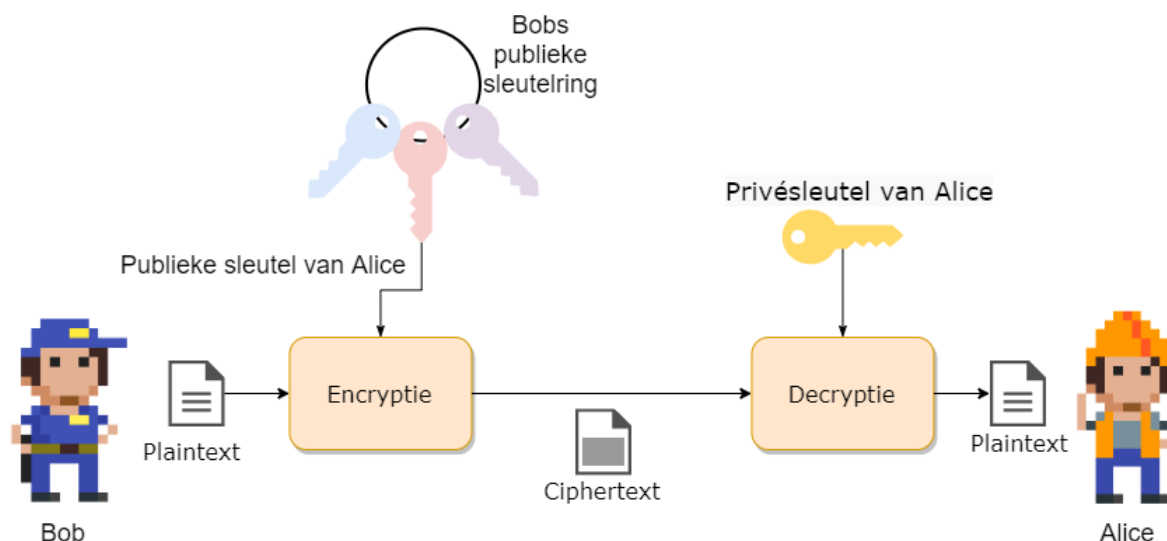
Publieke cryptografie

Publieke crypto oftewel **asymmetrische encryptie** zal het probleem met symmetrische encryptie oplossen doordat er gebruikt wordt gemaakt van 2 sleutels:

- 1 publieke sleutel
- 1 private sleutel

De publieke sleutel kan iedereen, vrij, gebruiken om versleutelde berichten mee aan te maken. Echter, enkel de eigenaar van de bijhorende private sleutel zal deze berichten kunnen decrypteren. Kortom, we lossen nu een deel van het sleutelprobleem op: iedereen heeft z'n eigen private sleutel (**en moet deze geheim houden!**) en kan via z'n publieke sleutel berichten krijgen.

De techniek werd in de jaren 70 door Diffie en Hellman ontwikkeld en had een grote impact op de manier waarop beveiligde communicatie op het internet mogelijk maakt.



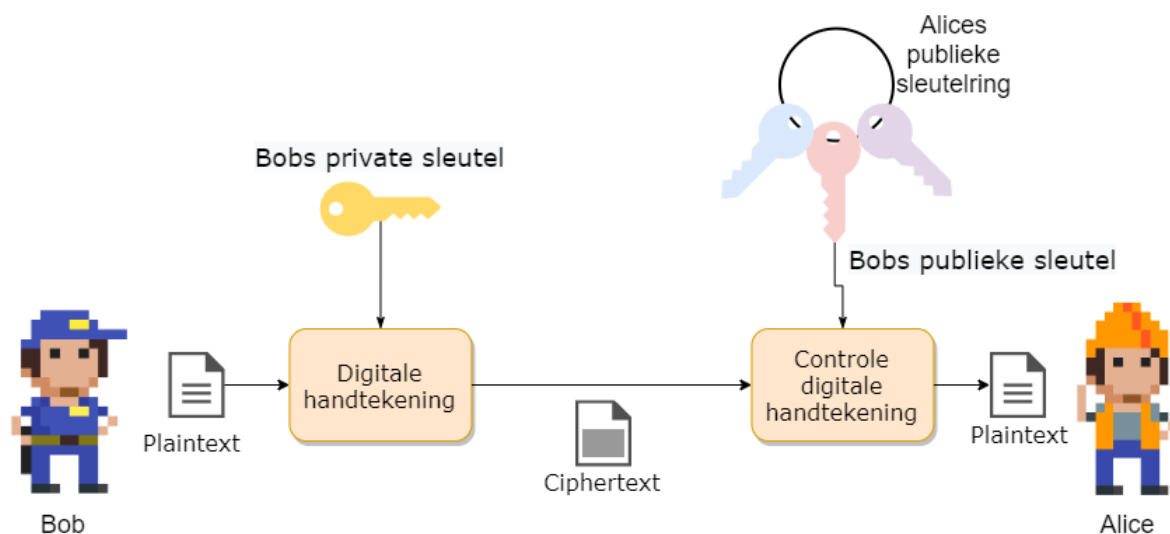
Je kan publieke encryptie beschouwen als volgt. De publieke sleutel, die niet geheim is, is een openstaande kist. Iedereen kan de kist gebruiken om een geheime boodschap in te plaatsen en vervolgens deze op slot te klikken. Enkel de eigenaar van deze kist heeft de bijpassende private sleutel die echter deze kist kan opendoen en de originele boodschap kan lezen (*decrypteren*).



Enkele van de bekendere publieke cryptosystemen zijn onder andere RSA, DSS en El Gamal.

Dualiteit van publieke cryptografie

Asymmetrische crypto zal niet alleen het sleutel-probleem oplossen, het heeft als extra eigenschap dat het publiekprivate sleutelpaar ook dienst kan doen als **digitale handtekening** om te controleren of een boodschap wel degelijk afkomstig is van een specifiek persoon. Hierbij zal een private sleutel gebruikt worden om het digitale bericht te ondertekenen. Daar de ondertekenaar de enige persoon kan zijn die deze private sleutel in z'n bezit heeft, kan men zijn identiteit bevestigen door de bijhorende publieke sleutel te gebruiken. Enkel de bijhorende publieke sleutel zal hiervoor gebruikt kunnen worden en zo hebben we een vorm van integriteit of data authenticatie.



We zullen dit concept verderop uitwerken, maar eerst gaan we bekijken hoe publieke crypto juist werkt.

Diffie-Hellman sleutel uitwisseling

Dankzij public crypto hebben we nu een systeem om sleutels op een veilige manier uit te wisselen. Het is namelijk zo dat symmetrische crypto sneller is én dus voor (realtime) communicatie interessanter is. We weten echter dat het sleutelmanagement bij symmetric crypto een probleem is als we met grote groepen gebruikers zitten. Het Diffie-Hellman sleuteluitwisselingsconcept helpt ons hierbij: het laat toe dat twee gebruikers sleutels over een onveilig kanaal kunnen uitwisselen op een veilige manier.

Zowel Bob als Alice genereren eerst een publiek/privaat sleutelpaar dat ze voor deze sessie wensen gebruiken ter communicatie. Vervolgens stuurt ieder z'n publieke sleutel naar de ander. De ontvanger zal deze publieke sleutel combineren met de eigen private sleutel wat zal resulteren in een nieuw *shared secret* dat beide nu kennen en kunnen gebruiken, bijvoorbeeld, als de symmetrische sleutel om verdere communicatie te bestendigen.

De reden dat dit werkt is met dank aan de modulo operator en de eigenschappen ervan. Een voorbeeld:

	Alice	Bob
Stap 1	Alice kiest een geheim getal A. Ze kiest $A = 3$	Bob kiest ook een geheim getal $B = 6$.
Stap 2	Alice berekent $7^A \% 11 \Rightarrow 343 \% 11 = 2$, genaamd X	Bob berekent $7^B \% 11 \Rightarrow 11764 \% 11 = 4$, genaamd Y

	Alice	Bob
Stap 3	Alice stuurt 2 naar B	Bob stuurt 4 naar Alice
Stap 4	Alice berekent $X^A \% 11 = 2^3 \% 11 = 9$	Bob berekent $Y^B \% 11 = 4^6 \% 11 = 9$

Zoals je merkt kunnen nu Alice en Bob het berekende getal 9 als gedeeld geheim kennen. Enkel zij 2 kennen dit getal.



Uiteraard zullen in de praktijk Bob en Alice véél grotere getallen kiezen dan 3 en 6.

RSA

Eén van de oudste, maar nog steeds populairste, publieke cryptosystemen is het in 1977 ontwikkelde RSA algorithm. RSA, wat staat voor de achternamen van de 3 ontwikkelaars (Rivest, Shamir en Adleman) gebruikt sleutels van 1536 tot 4096 bits lang. Het systeem is vrij traag maar heeft natuurlijk als voordeel dat het veilige sleuteltransmissie toestaat over een onveilig kanaal: we zien daarom vaak RSA gebruikt worden om eerst sessiesleutels uit te wisselen, vervolgens wordt overgeschakeld op een sneller symmetrisch cipher.

De exacte berekeningen die gebeuren tijdens encryptie en decryptie nemen ons iets te ver, maar volgend voorbeeld toont een vereenvoudigde wijze waarop RSA wordt toegepast:

Data encrypteren met behulp van asymmetrische versleuteling gebeurt op bijna dezelfde wijze als de Diffie-Hellman sleutel uitwisseling. Ook nu zullen beide zijde rekenen op de eigenschappen van de modulo-operator om over een onveilig kanaal veilige communicatie te kunnen doen.

Eerst dient een publieke sleutel aangemaakt te worden. Hiertoe dient Bob 2 grote priemgetallen, p en q te kiezen, bijvoorbeeld $p=17$ en $q=11$. Vervolgens berekent Bob N door deze priemgetallen met elkaar te vermenigvuldigen ($p \cdot q$ geeft $17 \cdot 11$, $N=187$). Bob kiest nu nog een priemgetal e , bijvoorbeeld 7. Bob kan nu zijn eigen geheime, private sleutel d maken, namelijk $e \cdot d = 1 \% ((p-1) \cdot (q-1))$ wat dus $7 \cdot d = 1 \% (16 \cdot 10)$ geeft of $7 \cdot d = 1 \% 160$. Om nu d te vinden moeten we een getal vinden zodat $7 \cdot d$ een veelvoud van $1 \% 160$ geeft, dus bijvoorbeeld 1, 161, etc. In dit geval vinden we $d=23$.



Om d te berekenen maken we gebruik van de zogenaamde *Uitgebreid Euclidisch algoritme*, een eeuwenoud algoritme gebaseerd op het *Algoritme van Euclides* dat we in het lager leerden gebruiken om de grootste gemene deler te berekenen van 2 getallen.

Bob heeft dus nu:

- Publieke sleutel bestaande uit $N = 187$ en $e = 7$.
- Private sleutel $d = 23$.

Iedereen die nu naar Bob iets wilt sturen kan dit via z'n publieke sleutel (N en e). Stel dat Alice het ascii-karakter X naar Bob wil sturen. De ascii-waarde van X is 88. De encryptie door Alice gebeurt dan als volgt: $C = data^e \% N$. De te versturen ciphertext C wordt dus: $(88^7) \% 187$ oftewel $C=11$.

Enkel Bob zal deze ciphertext met zijn private sleutel d kunnen decrypteren door $C^d (\% N)$ te doen, oftewel $11^{23} \% 187$ wat terug de plaintext 88 geeft!



de sterkte van publieke crypto stoelt dus op het feit dat ontbinden van (grote) getallen in factoren computationeel veel moeilijker is dan de omgekeerde stap, namelijk 2 getallen met elkaar vermenigvuldigen.

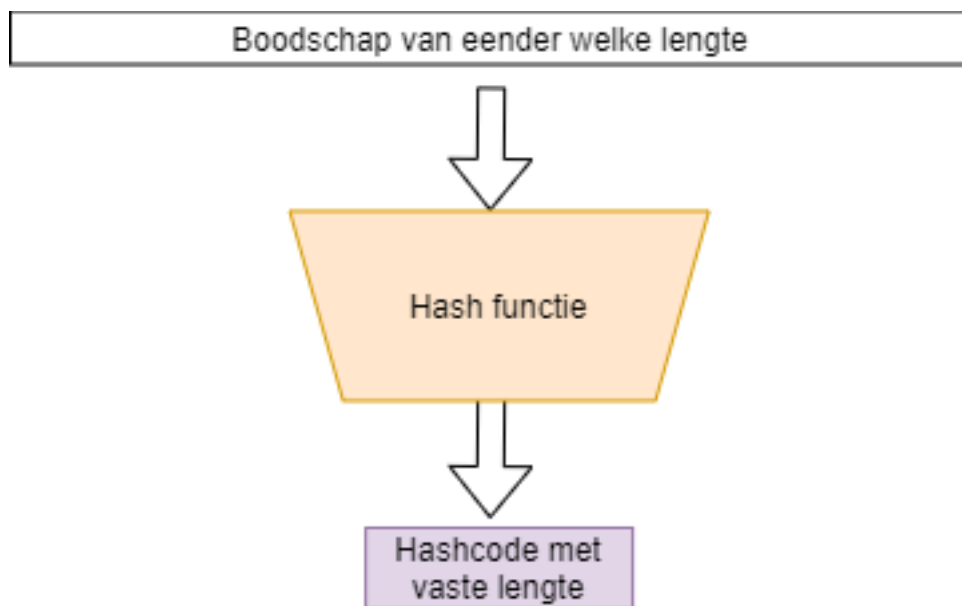
15621 in z'n factoren ontbinden is veel moeilijker dan de getallen 123 en 127 vermenigvuldigen (wat dus ook 15621 zal geven).



Zonder in detail te treden hoe cryptocurrencies en blockchains werken, is het nuttig om te vermelden dat bij cryptocurrencies ook de public crypto concepten worden gebruikt. Ook hier is je private sleutel uiterst belangrijk: enkel de eigenaar van de private sleutel “bezit” de bijhorende cryptocurrencies in de chain. Daarom is het uiterst belangrijk dat je NOOIT je private sleutel aan derden geeft, want zo geef je hen toegang tot jouw coins en kunnen ze vervolgens deze stelen door de private sleutel te vervangen.

Intermezzo: Hashes

Een zogenaamde hash is een concept uit de informatica die we gebruiken om te controleren of een digitaal stuk tekst werd aangepast of niet. Door de tekst in een hashfunctie te steken wordt een hash aangemaakt. Deze hash is een stuk code met een vaste lengte, ongeacht de originele input. Wanneer 1 bit of meer wordt aangepast in de originele boodschap dan zal deze in een totaal andere hash resulteren. Enkel dus wanneer een identiek stuk tekst als invoer (tot op bitniveau identiek) wordt gebruikt zullen twee hashen gelijk zijn.



Voorgaande is uiteraard onmogelijk: daar een hash meestal veel korter is dan de originele boodschap, is het mathematisch mogelijk dat 2 totaal verschillende teksten toch dezelfde hash geven. Het is de opdracht van een goede hashfunctie om dit soort **hash collisions** zo klein mogelijk te houden!

Een hash-functie is niet omkeerbaar: men mag onmogelijk aan de hand van een hash (ook wel *digest* of *hashcode* genoemd) terug de originele tekst kunnen achterhalen. Een hashfunctie is dus een eenrichtingsfunctie, ook wel afbeelding genoemd in wiskundige termen.

Er bestaan veel verschillende hash-functies. Enkele van de bekendere zijn:

- MD5, oftewel Message Digest 5: deze zal een 128-bit hashwaarde genereren.
- SHA-X, oftewel Secure Hash Algorithms. Zo is er SHA-256 wat een 256 bits hash zal genereren.

De sterkte van een hash algoritme zit hem in de grootte van de kans waarop hash collisions kunnen optreden. Zo zijn er bij MD5 al veel meer collisions gevonden dan bijvoorbeeld bij het recenter gepubliceerde SHA3-512 algoritme.

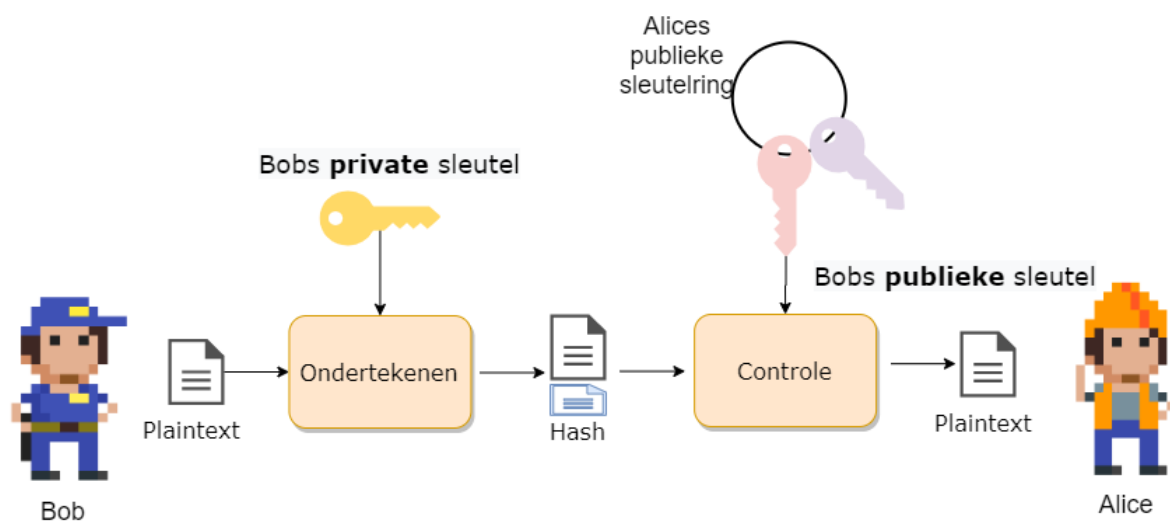
Boodschappen ondertekenen

Een probleem bij online communicatie is dat we geen zekerheid hebben dat het ontvangen bericht wel degelijk van de persoon komt van wie we verwachtten dat deze het bericht had opgesteld. We kunnen daarom het publieke crypto systeem gebruiken om berichten te ondertekenen. Daar enkel Bob de

bijhorende private sleutel kan hebben die bij z'n publieke sleutel hoort, is het bezit van deze private sleutel hebben het bewijs dat hij de rechtmatige eigenaar van een bepaalde publieke sleutel is.

Onder andere RSA laat toe om een digitale handtekening (**digital signature**) te plaatsen bij een bericht om zo te bewijzen dat de verzender de rechtmatige eigenaar van een bijhorende publieke sleutel is.

Een digitale handtekening wordt als extra bericht achteraan de te versturen boodschap geplaatst. De signature is als het ware een hash berekend aan de hand van de private sleutel. De ontvanger zal nu met de bijhorende publieke sleutel van de verzender kunnen verifiëren of de bijhorende private sleutel werd gebruikt om de handtekening te genereren.



Om een digitale handtekening te berekenen moeten we eerst een hash van het bericht berekenen. We gebruiken hier bijvoorbeeld MD5 of één van de SHA-algorithmes voor. Deze hash gaan we nu “verpakken” met de private sleutel.

Stel dat de hash 35 is en we hebben volgende sleutelpaar:

- publieke sleutel: $e=5$ en $n=91$
- private sleutel: $d=29$.

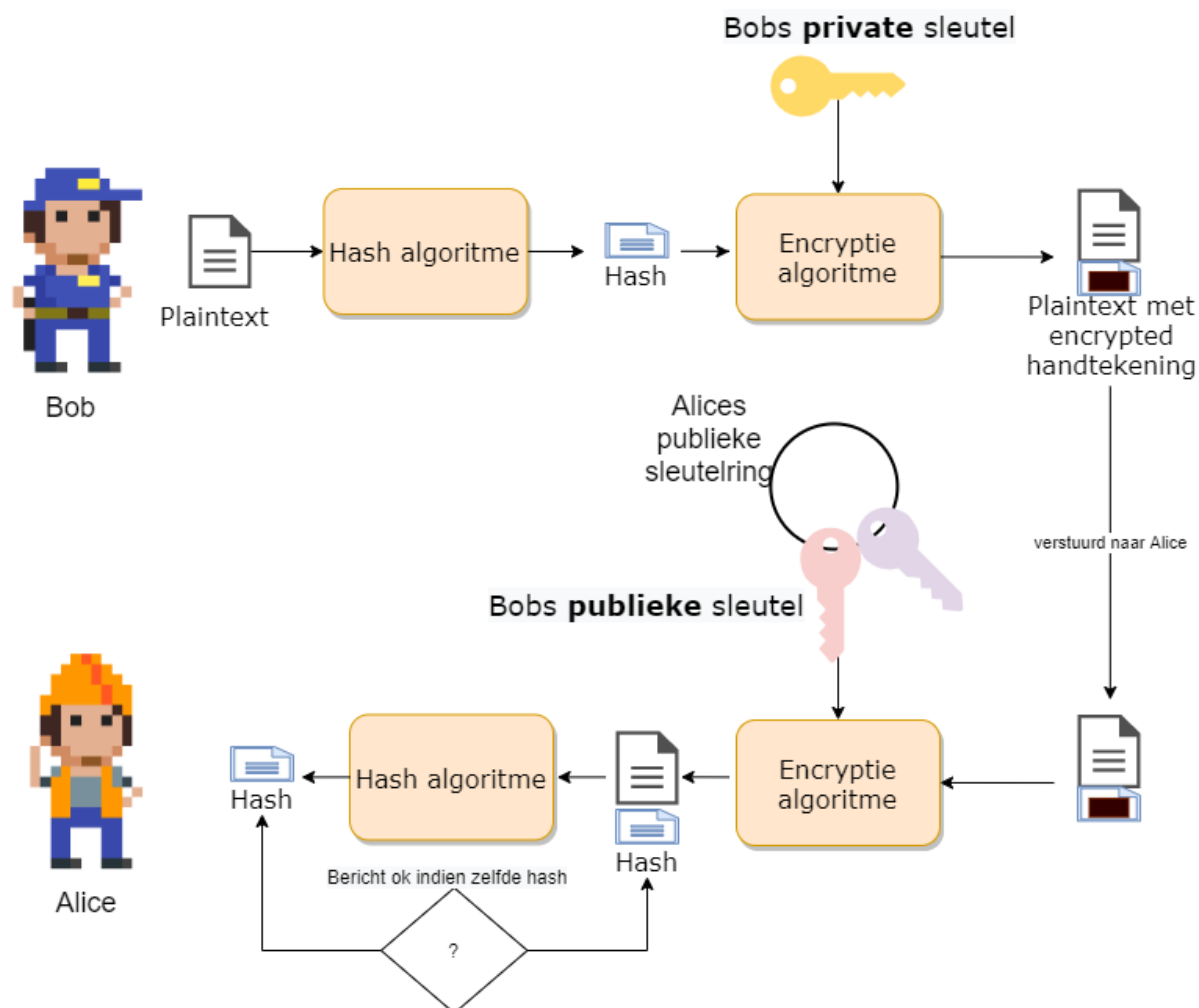
De handtekening wordt berekend door: $s = m^d \% n$ oftewel $s = 35^{29} \% 91$ wat 42 geeft.

We versturen dus naar de ontvanger de boodschap zelf en de bijhorende handtekening: 35, 42.

De ontvanger kan nu controleren of de ontvangen boodschap 35 dezelfde handtekening geeft. Hij gebruikt hiervoor de publieke sleutel e en berekent: $42^e == 35^n$, als dit overeenkomt dan weet de ontvanger dat hij het bericht kan vertrouwen.



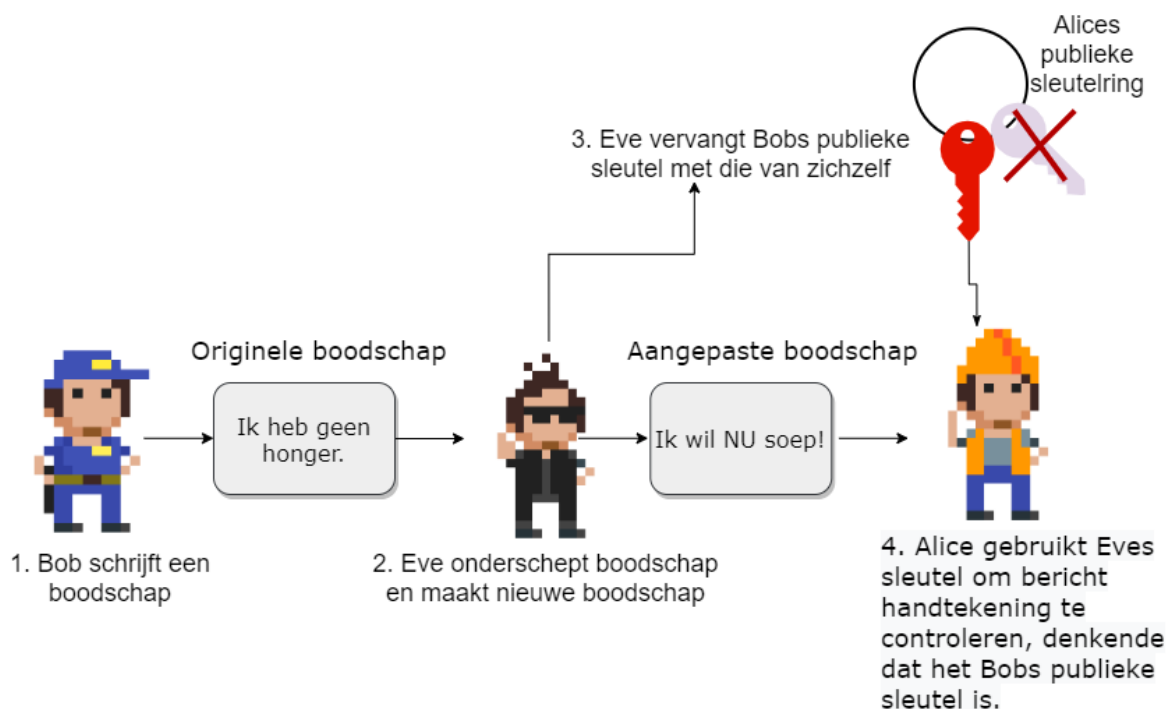
Getallen voorbeeld komt van hier



Het probleem met digitale handtekeningen

We hebben echter een probleem. Hoe weet je eigenlijk dat je wel de juiste publieke sleutel gebruikt. Publieke sleutels zijn, wel, publiek. Iedereen kan jou een publieke sleutel geven en zeggen “*Dit is de sleutel van persoon X*” zonder dat jij kan controleren of dat zo is.

We kunnen daarom als kwaadwillig persoon bijvoorbeeld een legaal bericht onderscheppen, aanpassen en dan vervolgens ondertekenen met onze eigen handtekening. Als we vervolgens aan de ontvanger kunnen wijsmaken dat jouw publieke sleutel zagezegd bij de originele verzender hoort, dan zal de ontvanger jouw aangepaste bericht “geloven”.



Kortom, we hebben een manier nodig om de **identiteit van de eigenaar** van een publieke sleutel te verifiëren. Kom binnen: **certificaten**.

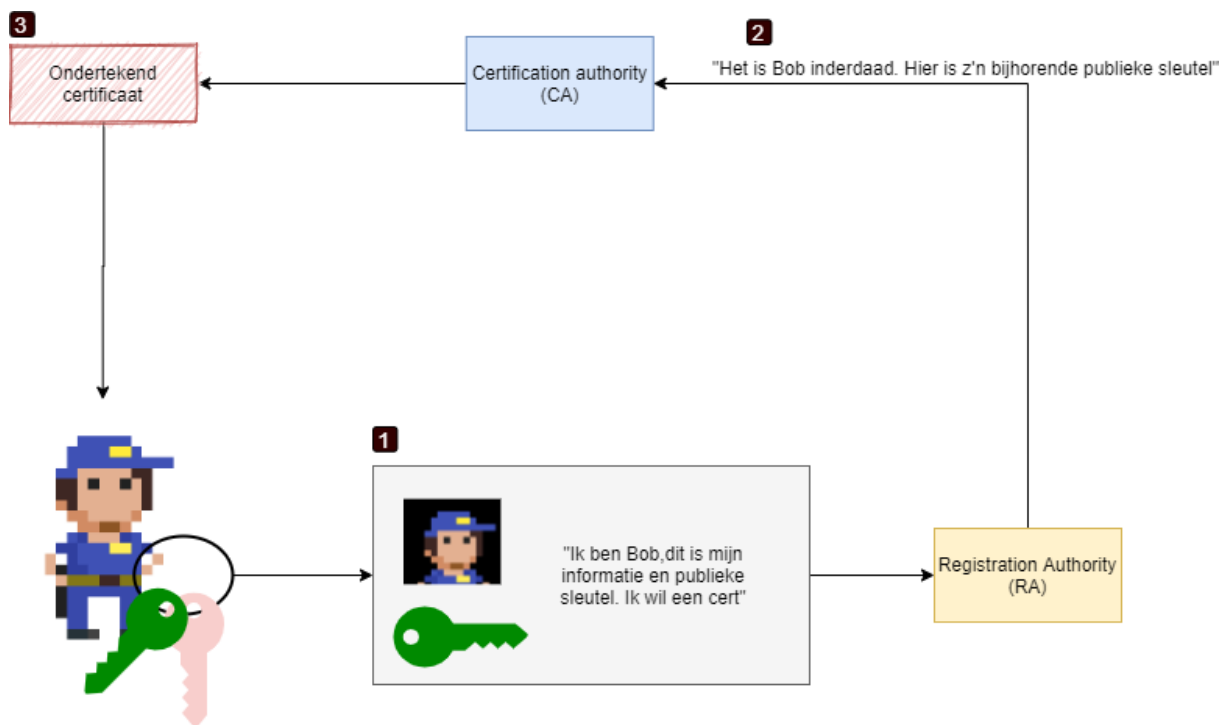
Digitale certificaten

Een certificaat is een (digitaal) document dat de identiteit van een gebruiker bindt aan een publiek sleutel. Dit document werd digitaal ondertekent door een vertrouwde derde partij (**trusted third party**) zodat bij twijfel van de echtheid van het certificaat men altijd bij deze derde partij terecht kan. Uiteraard is het belangrijk dat we deze derde partij kunnen vertrouwen, anders kunnen we ook niet de het certificaat vertrouwen dat zijn onderschrijven.



Certificaten worden beschreven in de **X.509** standaard.

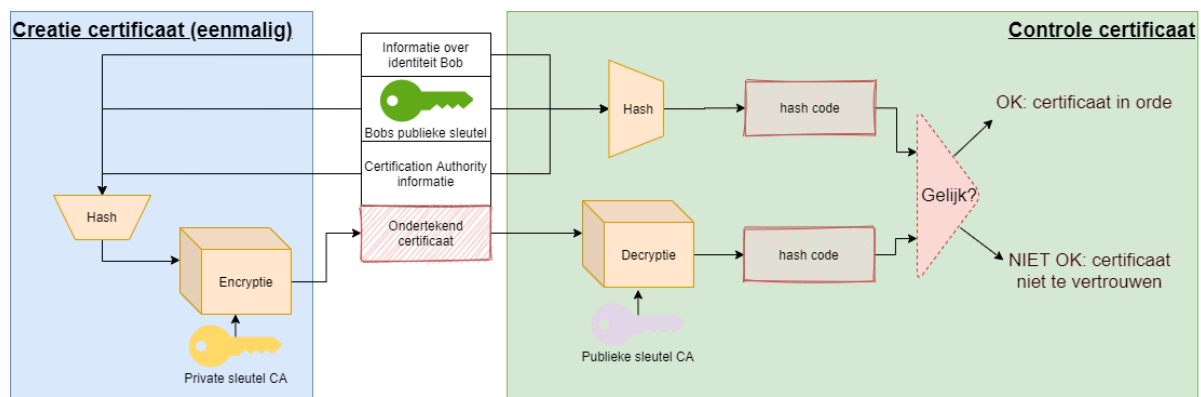
Om een certificaat aan te maken dient Bob naar een **Registration authority** (RA) gaan die zijn identiteit zal verifiëren. Dit gebeurt aan de hand van de typische documenten die ook buiten het Internet worden gebruik om iemands identiteit te bewijzen: identiteitskaart, paspoort, rijbewijs, etc. In sommige gevallen zal de RA zelfs eisen dat Bob zich naar een fysiek kantoor begeeft om daar z'n identiteit *in te flesh* te bewijzen. Indien de RA de identiteit heeft bevestigd zal deze de aanvraag van Bob doorsturen naar een **Certification authority** (CA), inclusief Bobs publieke sleutel.



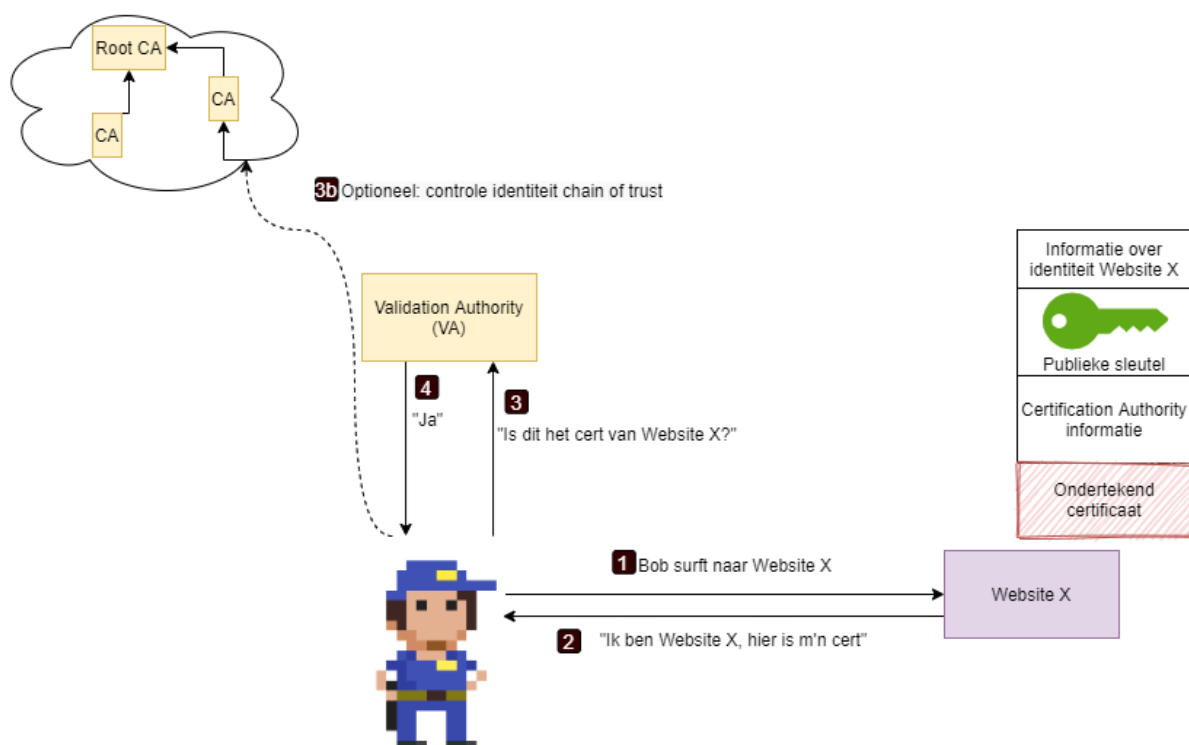
Het gehele systeem van CA's, RA's, etc. dat bestaat om certificaten uit te geven, beheren en bewijzen heet een **public key infrastructure (PKI)**.

De CA zal deze informatie gebruiken om een certificaat, van een bepaalde levensduur, te genereren. Hierbij zal de echtheid van de CA achter af bewezen kunnen worden door de CA:

- Het certificaat is een geëncrypteerde hash van Bobs publieke sleutel, informatie over de CA en over Bob. De encryptie van de hash gebeurt aan de hand van de private sleutel van de CA.
- Om later de echtheid van een certificaat te testen voldoet het om dezelfde hash te genereren (publieke sleutel, info over Bob en CA) en deze te vergelijken met het certificaat na decryptie met de publieke sleutel van de CA. Als deze gelijk zijn weten we dat het certificaat door de gegeven CA werd ondertekend (enkel hun private/publiek sleutel paar zal terug de originele hash geven).

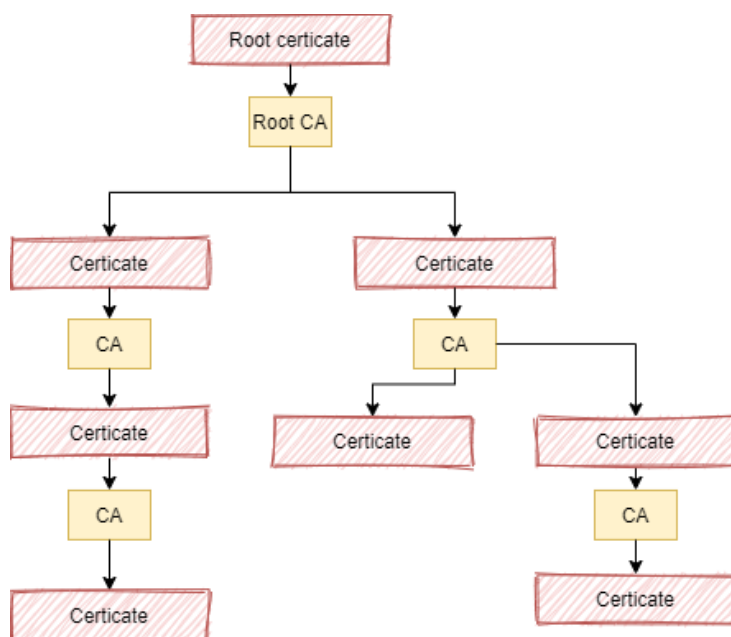


Voorgaande proces zal plaatsvinden wanneer je browser via een **https** verbinding surft naar een website en zo wil controleren of wel degelijk met de website wordt gecommuniceerd en niet met een imposter. Indien de browser (of de gebruiker) twijfelt aan de echtheid van de publieke sleutel van de CA die het certificaat van de website ondertekent, dan zla voorgaande proces zich herhalen, maar deze keer om het certificaat van de CA te controleren met behulp van een bovenliggende CA. Op die manier kan het dus zijn dat een keten van CA's ontstaan die telkens CA's onder zich bewijzen. Uiteraard zal er steeds bovenaan zo'n ketting een **root CA** staan. Als je die vertrouwt, dan kan je al de CA's er onder dus ook vertrouwen... maar ook vice versa!



Het ergste voor een CA dat kan voorvallen is dat de betrouwbaarheid van de CA in het gedrang komt.

Als een CA bijvoorbeeld weet heeft van een potentiële inbraak op hun systemen dan bestaat er de kans dat aanvallers de private sleutel van de de CA hebben bemachtigd en dus zelf certificaten *op naam van de CA* kunnen genereren, met alle gevolgen van dien! Indien dus deze kan bestaat dan is er een *breach of trust* en zullen alle certificaten van deze CA als ongeldig worden bestempeld, inclusief alle certificaten van sub-CA's! Dit kan verregaande gevolgen hebben.

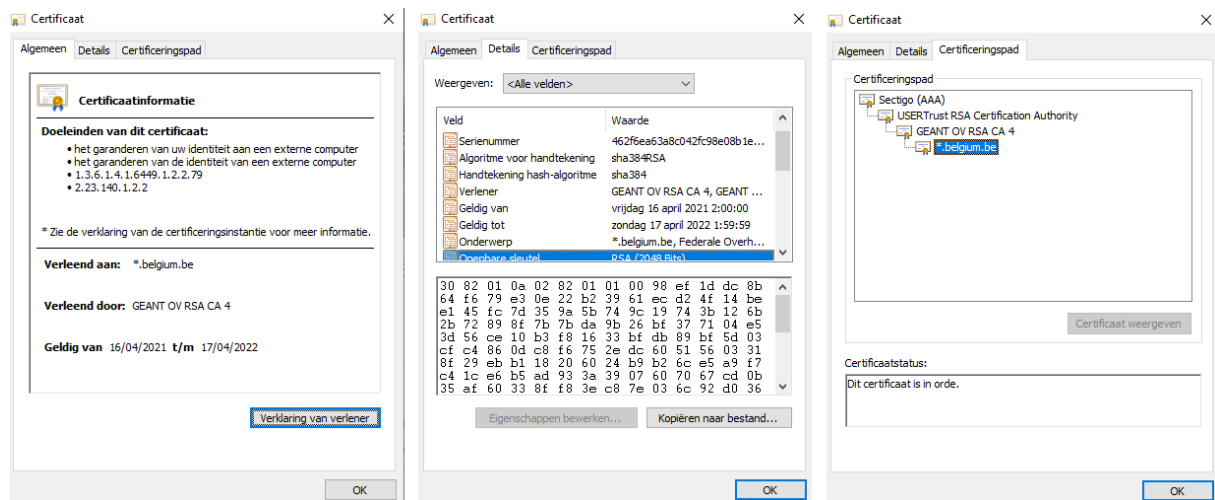


Certificaten bekijken

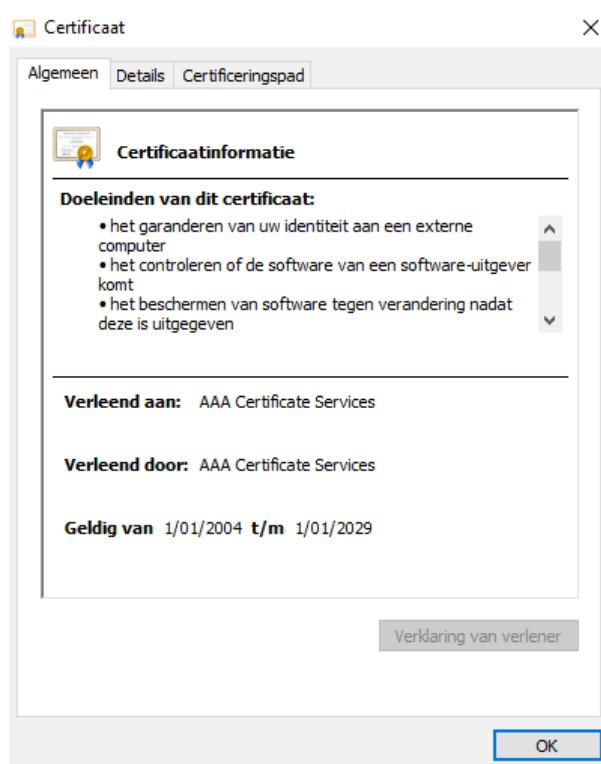
In iedere moderne browser kan je snel bekijken hoe zo'n certificaat er juist uit ziet. Als je via een https verbinding naar een website surft dan op het slotje naast de URL n de adresbalk klikt kan je doorklikken om het certificaat te openen. Als je naar <https://www.belgium.be> surft en dit doet dan krijg je eerst wat samenvattende informatie:

Zo zien we onder andere de geldigheidsduur, alsook de CA die dit certificaat heeft gegenereerd. Onder details kunnen we onder andere de publieke sleutel zien van de website alsook de gebruikte algorithmes voor de hash, e.d.

En op de laatste tab, Certificeringspad, zien we de chain of trust. We kunnen vervolgens hier de bovenliggende certificaten bekijken.



Het certificaat van Sectigo is uiteraard een **selfsigned certificate**, daar zij “bovenaan de hiërarchie staan”. Als we Sectigo niet vertrouwen dan kunnen we ook de communicatie met *belgium.be* niet vertrouwen daar.

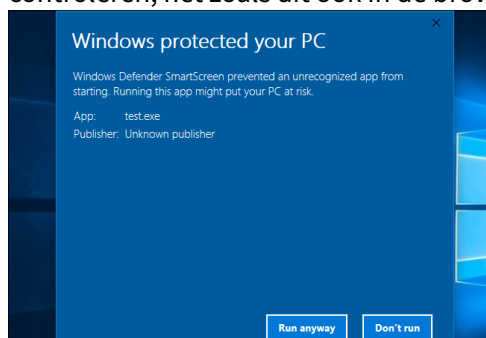


Figuur 0.14: Sectigo heeft een self-signed certificaat wat je herkent aan het feit dat de velden *Verleend aan* en *Verleend door* dezelfde waarde hebben.



Naast certificaten voor webserver (zogenaamde **SSL certificaten**) kan je ook een persoonlijk certificaat aankopen om je eigen identiteit aan derden te bewijzen tijdens bijvoorbeeld email-communicatie. Voorts heb je ook **code signing** certificaten die de echtheid van een applicatie bewijzen zodat je zeker bent dat je geen malware installeert als je programma X hebt gedownload.

Als je in Windows 10 een applicatie of installer probeert uit te voeren dan zal de ingebouwde *SmartScreen* service ogenblikkelijk de echtheid (of ontbreken van) het certificaat controleren, net zoals dit ook in de browser zou gebeuren.



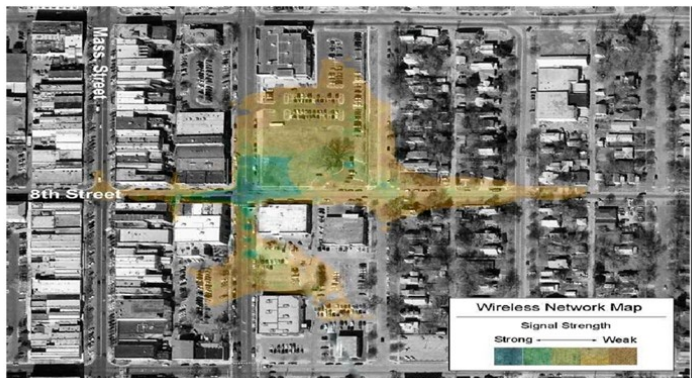
Wifi security

Alhoewel dit hoofdstuk integraal na het crypto hoofdstuk komt, is het toch interessant om dit hoofdstuk geschrinkt met het crypto hoofdstuk door te nemen als volgt:

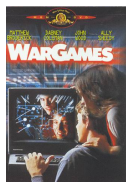
- “Crypto” tot en met “Symmetric stream ciphers”.
- “Wifi security” tot en met “WEP en waarom het faalde”.
- De rest van “Crypto”.
- De rest van “Wifi security”

De problemen van Wifi

We kunnen draadloze netwerken, specifiek wifi-netwerken, niet meer uit ons leven inbeelden. De opkomst van de IEEE 802.11b standaard in 1999 veroorzaakte een kleine revolutie in de manier waarop bedrijven en privégebruikers konden werken. Plots kon je met een laptop van overal in het gebouw - en zelfs er buiten- op het netwerk geraken. Die vrijheid voor de gebruikers betekende wel een nachtmerrie voor de cyberboswachters. Een netwerkkabel heeft een intrinsieke extra beveiliging: enkel daar waar de kabel ligt kunnen gebruikers aan het netwerk geraken. Zolang je dus geen netwerkkabel bijvoorbeeld naar de publieke parking brengt kan niemand van daar illegaal het netwerk benaderen. Met wifi leek het alsof plotseling het hele netwerk in een straal van tientallen meters rond het gebouw beschikbaar was, met alle gevolgen van dien.



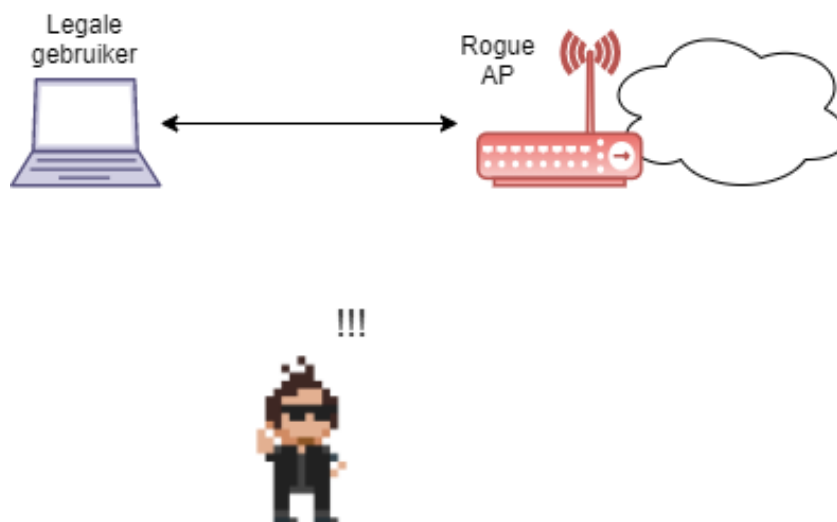
Al gauw werd een nieuwe sport uitgevonden door hobbyist hackers en professionele cybercriminelen: **wardriving**. De idee is eenvoudig: rijdt rond in de stad en laat de laptop naast je in de auto scannen naar alle netwerken, met extra aandacht voor die netwerken die geen of zwakke beveiliging hadden.



De term wardriving komt van de term *wardialing* die op zijn beurt gebaseerd is op de klassieke cyber-cult film “Wargames” uit 1983. Voor de geschiedkundigen onder ons, wardialing was het opbellen van willekeurige telefoonnummers met je modem in de hoop een zogenaamd *bulletin board system* oftewel **BBS** (een pre-internet forum zeg maar) te vinden.

Draadloze netwerken die gevonden worden hebben dan ook een schare aan problemen:

- **Eavesdropping:** iedereen kan “meeluisteren” wat er door de lucht vliegt. Dit heb je niet met een bedrade kabel.
- **Invasion:** je kan eenvoudig verbinden met het netwerk indien er geen beveiliging voorzien werd. Iets dat je dus bij een bedraad netwerk enkel kunt als je fysiek toegang hebt tot een netwerkkabel.
- **Man-in-the-middle aanvallen:** hier gaan we zo meteen dieper op in.
- **Backdoor:** een veelvoorkomend probleem zijn zogenaamde **rogue access points** die worden bijgeplaatst op het netwerk door goedbedoelde werknemers die zou het het bereik van het netwerk wat willen uitbreiden. Vaak zijn echter de beveiligingsinstellingen van zo’n (meestal goedkoop) access point niet zo sterk als die van het bedrijf. Bijgevolg is dit de ideale backdoor voor malafide gebruikers die het netwerk aan het surveilleren zijn. Wanneer ze een netwerkscan doen zullen ze tientallen goed beveiligde access points zien en 1 zwak beveiligd.



Figuur 0.1: Een rogue access point is de ideale manier om binnen te geraken voor hacker.

- **Denial-of-service op fysiek niveau:** om een gebruiker toegang tot een bedraad netwerk te ontfeggen op fysiek niveau dien je de kabel door te knippen. Bij wifi is dit nog eenvoudiger: de “kabel” bij wifi zijn de frequentiebanden in de lucht waarbinnen de apparaten mogen werken (circa 2.4 Ghz bij de oudere wifi apparaten, nu meestal rond de 5 Ghz band). De wifi-apparaten kunnen enkel met elkaar communiceren indien zij een signaal naar elkaar over die frequentieband kunnen sturen op een moment dat niemand anders in de buurt die band gebruikt (zie note hierna). Als een malafide gebruiker dus die frequentieband vult met andere signalen, dan zullen de legale gebruikers nooit iets kunnen uitsturen. Wil je dus een wifi netwerk *Dos’n*, koop dan een signaalgenerator die op de juiste frequentieband de nodige ruist uitzend en klaar is kees.



Bedrade netwerk apparaten werken volgens het CSMA/CD (Carrier Sense Multiple Access / Collision Detection) om met elkaar over de draad te communiceren, hierbij detecteren ze wanneer er ‘botsingen’ tussen pakketen voordoen en de informatie dus opnieuw moet uitgestuurd worden. Bij draadloze netwerken is het echter onmogelijk om *botsingen in de lucht* te detecteren, daarom werken ze met een variant: **CSMA/CA**, oftewel CSMA/ **Collision Avoidance**. Een wifi-apparaat dat iets wil uitsturen zal eerst controleren of de frequentieband waarop ze werken vrij is, enkel dan zal er vervolgens een pakketje de lucht worden ingestuurd.



Sommige gebruikers schakelen het broadcasten van hun netwerknaam (het zogenaamde **SSID**) uit in de hoop dat zo dat burens, voorbijgangers en wardrivers hun netwerk niet kunnen zien. Helaas is dit zogenaamde *snake's oil*: het geeft een vals gevoel van veiligheid. Je kan weliswaar SSID-broadcasting uitzetten (dit is een pakketje dat je access point elke paar seconden uitstuurt om aan iedereen die luistert te zeggen “*Hallo, hier is het netwerk met naam X, en dit zijn de parameters die je nodig hebt om met mij te verbinden*”) maar het SSID wordt ook in een hoop andere pakketjes de lucht in gestuurd. Een beetje wifi hacker kan dus het SSID ogenblikkelijk uit de lucht plukken, ongeacht dat broadcasting werd uitgeschakeld of niet.



Om de huidige, en betere, beveiliging van wifi te appreciëren gaan we wederom even terug in de tijd om te kijken hoe de originele IEEE wifi standaard de beveiliging beschreef. Het zal een ietwat horror-achtige tocht worden waarin we gaan ontdekken dat enkele stevige hiaten ervoor gezorgd hebben dat illegale toegang tot bijna ieder wifi netwerk rond de eeuwwisseling binnen enkele minuten kon gebeuren. Lees verder en huiver mee.

Onbeveiligde managementframes

Veel keuzes die in de IEEE standaard werden gemaakt zijn vermoedelijk het gevolg dat men leentje buur is gaan spelen bij de reeds bestaande IEEE standaarden voor bedrade netwerken. Echter, bij bedrade netwerken had je niet de inherente onveilige omgeving van het draadloze aspect, waardoor op gebied van beveiliging hier weinig tot geen aandacht aan werd besteed.

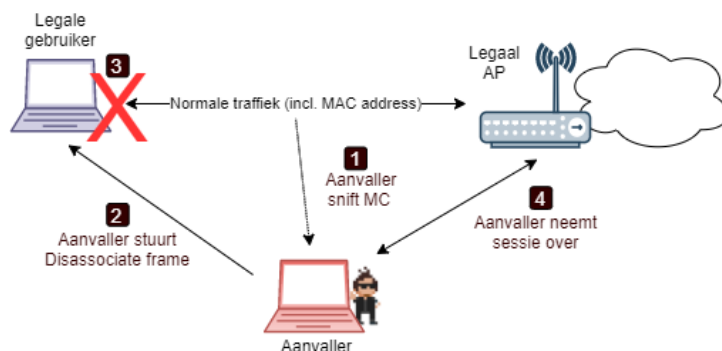
Management frames in wifi zijn frames die ervoor zorgen dat alle aanwezigen op het netwerk op een ordentelijke manier met elkaar kunnen communiceren. Deze frames zorgen bijvoorbeeld voor:

- Om een client verbinding te laten maken met een netwerk.
- Om SSIDs te broadcasten.
- Clients de opdracht te geven het netwerk te verlaten.
- Om te verbinden met een ander access point van hetzelfde netwerk (*handover*).

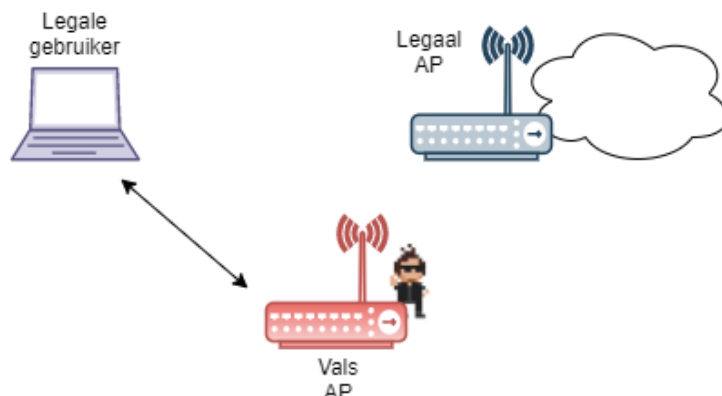
Net zoals bij bedrade netwerken, koos men bij de IEEE wifi standaard om deze managementframes **zonder enige vorm van beveiliging** te gebruiken (er werd geen *CIA* voorzien). Dit zorgde ervoor dat niet legale gebruikers zelfs management frames konden uitsturen op een netwerk, zonder dat de ontvangers ervan konden controleren of de bron wel een legaal access point of client was.

Dit resulteerde in onder andere volgende scenario's:

- Een hacker kan legale gebruikers DoS'n door constant zogenaamde *disassociation* naar hen te sturen. Dit frame, gebruikt door access points, geeft aan clients de opdracht dat ze het netwerk moeten verlaten. De hacker kan zo'n frame uitsturen en daarbij het "source" veld instellen op het MAC-adres van het access points. Dit spoofing kan ongecontroleerd, waardoor legale gebruikers dit frame altijd zullen aanvaarden én vervolgens uitvoeren: de gebruiker kan niet meer verbinden met het netwerk zolang de disassociation frames blijven verstuurd worden (**disassociation flooding**)
- **Identity spoofing** was ook eenvoudig daar de hacker eender welk veld in de frames kan aanpassen. Van zodra hij een legale gebruiker met voorgaande techniek van het netwerk heeft geschopt, kan hij vervolgens zichzelf voordoen als deze gebruiker. Hiervoor moet hij gewoon het MAC-adres spoofing van de legale gebruikers tijdens de communicatie met het access point.



- En last but not least laten de onbeveiligde management frames toe dat we eenvoudig een access point kunt nabootsen (**impersonation**). Vervolgens kunnen we een **man-in-the-middle aanval** uitvoeren daar een hacker zich kan plaatsen tussen de gebruiker en het internet en zo informatie kan ontfutselen.





De linux tool *AirSnarf* laat toe om fake hotspots (publiek wifi netwerk) op te zetten. Hierbij maakt het gebruik van de onbeveiligde management frames. Een scenario in België dat gegarandeerd success heeft (vanuit het standpunt van de hacker) is een fake Telenet Wifree hotspot op te zetten. Hierbij zal je eerst de login pagina van Telenet Wifree kopiëren en via een lokale webserver aanbieden aan de gebruikers die op jouw access point met de naam “Telenet Wi-Free” verbinden. Je kan nu van iedere gebruiker de gebruikersnaam en paswoord stelen telkens deze die informatie op jouw fake pagina invoert.

Volgende Engelstalige teksten komen uit een oudere cursus van dme en zullen (ooit) vertaald worden. Op deze manier kan ik echter focussen op jullie zoveel mogelijk leerstof in cursusvorm aan te bieden.

The 802.11 standard and its security

The original “ANSI/IEEE Std. 802.11” was written in 1999 and had the purpose “to develop a medium access control (MAC) and physical layer (PHY) specification for wireless connectivity for fixed, portable, and moving station within a local area.”

The security chapter in the standard (Chapter 8: “Authentication and Privacy”) was only a mere 10 pages long, compared to the total number of pages (528) this might be seen as a forebode of how little security was originally conceived in the standard.

Before we dive into WEP, the original *security motor* of wifi, we will first have a look how clients actually join a wireless network. As previously noted, this process includes the usage of unprotected management frames.

Joining a network

The moment there is any form of interaction between the attacker and his target he can begin his malicious acts. The same applies to wireless hacking: before being to hack a network the attacker first needs to find one and establish a ‘link’, in this case this can be nothing more but an antenna that captures all passing radio signals. Yet, capturing data passively as described is one thing, actively attacking a network is a whole other business. To be able to do so, an attacker needs to actually join a network, one way or another.

The IEEE 802.11 standard specifies how a wireless LAN can be joined. Several steps need to be undertaken before a user or attacker can be granted actual permission to the network and use it for higher-layer data traffic:

1. **Scanning:** Searching for possible networks to join.
2. **Joining:** Choosing a network you want to access.
3. **Authentication:** Giving the right credentials to be allowed to use the network.
4. **Association:** Making sure the system keeps track of your location in the network. It is important to note that a user or attacker can only begin using a network's resources when he is associated.

Scanning

Any device that wants to use a network first needs to find one, and so it scans the area to discover a compatible network to join. Several parameters are used in the scanning procedure and some of them can be specified by user; many implementations have default values for these parameters in the driver. One of these parameters is the SSID, or Service Set Identifier, which contains the name of the network. With this parameter, the user can choose to scan for a specific network, or scan for any network in the area using the broadcast SSID.

The SSID is a 32byte ASCII character string as described in the 802.11 specifications. In these specifications is also stated that any client setting this string to a 'NULL' string will associate to any access point regardless of the SSID setting on the access points. This is referred to as the broadcast SSID and is normally only used in Probe Request frames when a station attempts to discover all the 802.11 networks in its area.

Joining

After the scan report is made the station can choose to join one of the networks. This joining is not the same as associating; it is analogous to aiming a weapon, you are only planning to use the chosen network and its services, but first you have to be authenticated one way or the other and be associated.

What network is joined depends on whether the user chooses one or not. If the user lets the device choose, the station works with some criteria to make the decision like the power level and signal strength. **When chosen, the station has to synchronize its timing and also match the physical and MAC parameters.** It then tries to authenticate itself.

Authentication

When in a wired network, authentication is almost provided by the mere physical access; if you are close enough to plug in a cable, you most likely were allowed by, for example, the receptionist at the front desk and thus need no extra authentication to use the network.

This is certainly not the fact in a wireless LAN. Therefore authentication was added to the process of joining a network nonetheless. Two major approaches are specified by 802.11 to ensure that a station attempting to associate with a network is allowed to do so:

- **Open-system authentication** (might include WEP)
- **Shared-key authentication** (includes WEP)

802.11 authentication as originally specified as a one-way street. There is no mutual authentication whatsoever, thus allowing a malicious attacker to employ ‘man-in-the-middle’ attacks (see earlier) without the end user knowing it. A rogue access point could send Beacon frames for a network it is not part of and for example attempt to steal authentication credentials.

No other authentication algorithms are defined in the 802.11 standard but the two mentioned before; there exists however a third popular (but not yet standardized) method,

- **MAC Address Authentication using a MAC-ACL:** this is simply a whitelist (ACL stands for **access control list**) containing all the MAC-addresses that are allowed on the network. However, as we’ve seen, this is useless since MAC spoofing in wifi is peanuts (including sniffing the air for a legal MAC-address).

Open-system authentication

Originally the open-system authentication was seen as the ‘no-security’ method of authentication for wireless networks. History however has proven that this mode is now more secure when used in 802.1X since it does not leak any information on the used encryption etc (more about this later). Two frames are exchanged in this setup:

1. From client to access point, requesting access .
2. The other way around, with the access point basically sending an “hello there” frame.

If no encryption is used in the network, any device knowing the SSID of the AP can gain access to the network. With WEP encryption enabled, the WEP key itself becomes a form of access control. If a device does not have the correct WEP key, even though authentication was successful, the device will be unable to transmit data through the AP. And neither can the device decrypt data it receives from the AP. This way however, there is no security against attackers that have stolen/cracked the WEP key; there is no form of real user authentication.



It may seem useless to have an authentication algorithm like this that provides no real security when no other encryption is used. However, there are many wireless devices that simply don't have enough CPU resources to support more complex authentication algorithms. Hand-held devices like bar-code reader, network scanners etc just need quick access to a network without the extra hassle of authentication and therefore could use open-system authentication.

Shared-key authentication

Shared-key authentication, as its name implies, requires that a shared key is distributed to the stations before they attempt to authenticate. This is done using WEP and therefore can only be used with products that implement WEP. Proving that you own the correct WEP key is enough proof to be allowed on the network.

This proof is accomplished through a **challenge-response** system: 1. The access point creates a random string (*the challenge*) and sends this in plaintext to the client 2. The client encrypts this string and sends it back to the access point (*the response*) 3. The access point will try to decrypt the response using his own WEP key: if the original challenge text appears, the access point knows that the response was encrypted with the correct key and so will grant access to the client on the network.



Association

Once authentication is completed, the station is allowed to associate and/or reassociate with any access points in the network, for which he is authenticated.

Once associated, actual data transmissions (i.e. usage of the network) can start. Depending on the settings of the network, it is at this point that actual encryption of the data starts. This encryption is accomplished using WEP, as explained next.

WEP

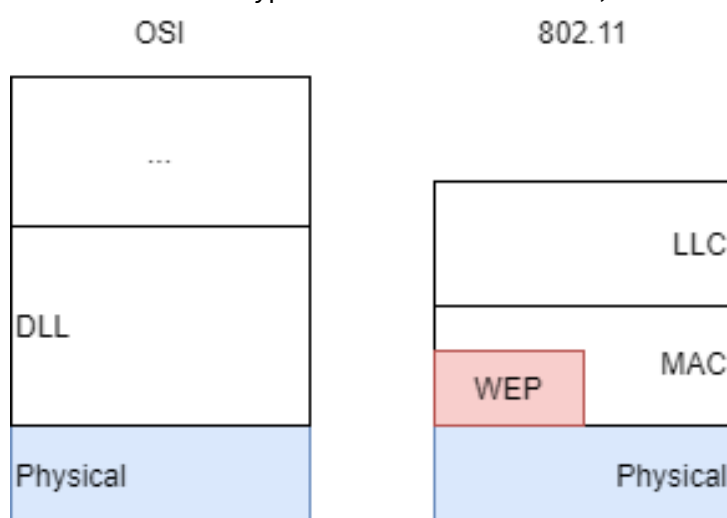
After being associated to a wireless network, the eavesdropping problem (and others) become obvious. Therefore the 802.11 specifications described **WEP** or "**Wired Equivalent Privacy**", a protocol that

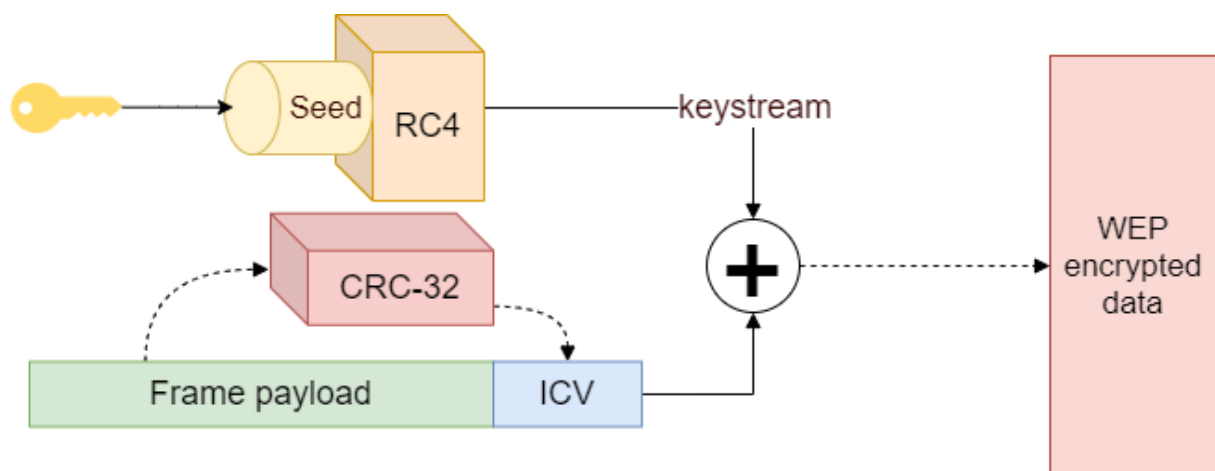
was believed to create the same level of privacy experienced on a wired LAN. WEP is 802.11's optional encryption standard implemented in the MAC Layer that most wireless LAN-cards and access point vendors supported around the time Wifi become immensely popular.

When WEP is enabled, all data is encrypted before being transmitted. Only with the right key can the receiver decrypt the data afterwards. And so, when a user is finally associated with a network, he can rely on WEP to have some form of privacy.

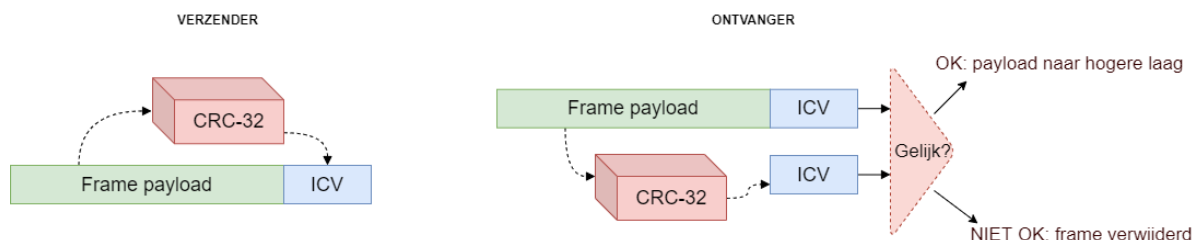


To be exact: the MAC layer receives packets from the LLC layer. If WEP is enabled this packet is sent to WEP where it is fragmented, if needed, into several frames. It is these frames that are encrypted and sent further down, to the PHY layer.



How WEP works**Figuur 0.2:** WEP in full

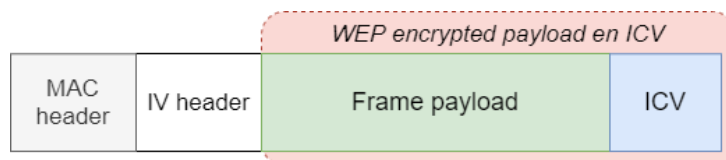
WEP uses RC4, a symmetric stream cipher and a WEP-key to create a pseudo random key stream. It is based on the principle of a one-time pad, which is the only known encryption scheme that is mathematically proven to protect against certain types of attacks. The RC4 algorithm is a set of rules used to expand the key into a key stream as explained earlier. This generated stream is XOR'd with the plain text producing the cipher text. To read this text again, the receiver needs to have the same key. Using this key he then recreates the same pseudorandom stream and XOR's the cipher text.

**Figuur 0.3:** Integrity check

Confidentiality and integrity are handled simultaneously. Before the encryption, the frame is run through an integrity check algorithm (CRC-32), generating a hash called the integrity check value (ICV). This ICV protects the data from being forged during transmission. Both the frame and the ICV are encrypted making the ICV unavailable to casual attackers.

The decapsulation (decryption) is basically the reverse procedure, with that respect that the receiver also makes a CRC-32 hash which is compared to the one received. If both received and self-made hash are equal the data is considered 'clean' (i.e. was not changed due to random noise, etc)

Shown in the following figure is a schematic overview of the resulting frame:

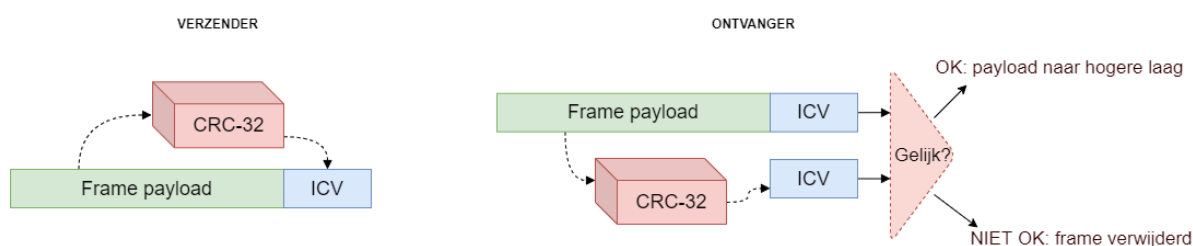


Figuur 0.4: WEP frame layout



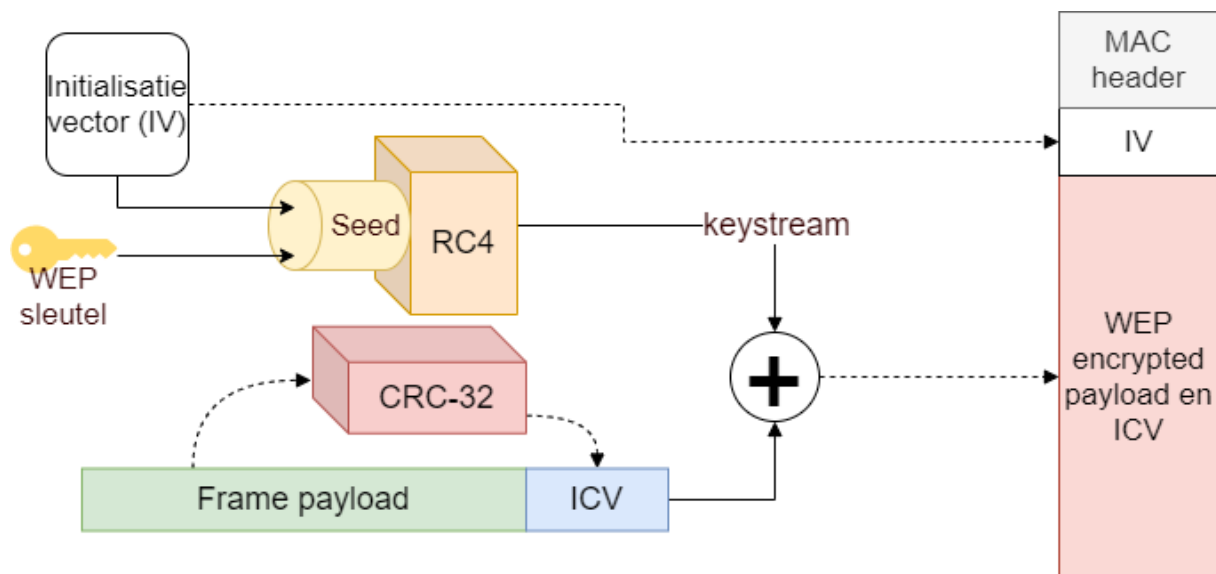
To protect traffic from brute-force decryption attacks, a set of up to four default keys is used. The default keys, identified by a variable keyid (0 to 3), need to be shared among all stations in a service set. Once a station has obtained the default keys for its service set, it can communicate using WEP.

WEP originally specified the use of a 40bit secret key; proprietary versions however also support longer keys (e.g. 104, 128, etc). The secret key is combined with a 24-bit initialisation vector (IV) to create a longer key (64 if 40-bit key was used). The IV is a random generated number; however this was not stated in the original specifications. This new key (WEP-key + IV) is used as the seed for the RC4 key stream generator. The key stream is then XOR'd with the frame body and the ICV.

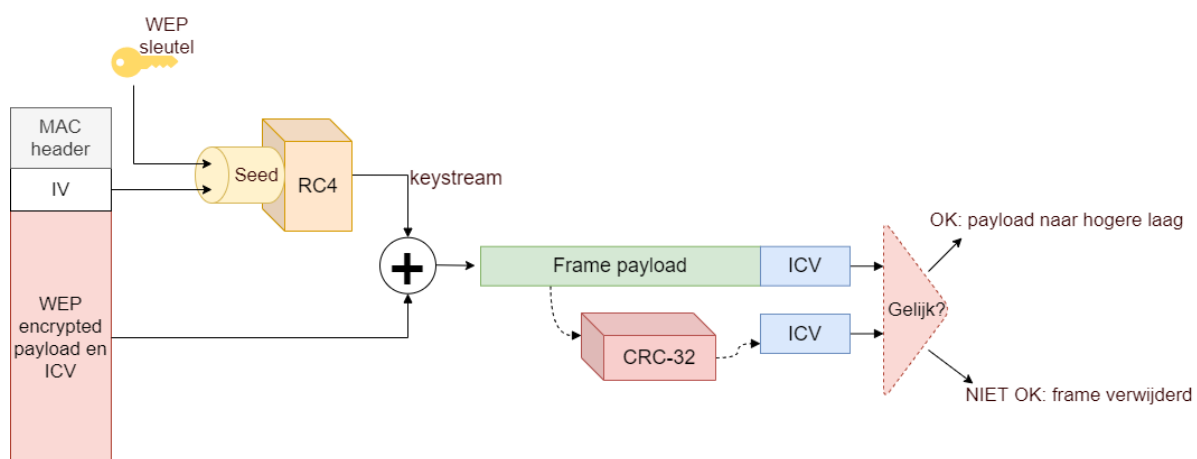


Figuur 0.5: Encryption

To enable the receiver to generate the same random stream, thus enabling him to decrypt the frame, the IV and keyid is placed in the header of the frame.



Figuur 0.6: WEP encryption in full



Figuur 0.7: WEP decryption in full

How WEP failed

Throughout time, more and more papers were released, identifying flaws in the overall WEP security.

The flaws can be summarized to 4 large problems with WEP, each having their own problematic results:

- RC4 cipher was not meant for datagram environments.

- The IV is badly implemented.
- The CRC-32 is not secure enough.
- No real key distribution system.
- There is no replay protection.
- The problem of having no replay protection will not be discussed separately since it is basically a flaw that ‘helps’ to make the four other mentioned problems even bigger.

Problem 1: RC4

Most problems result from a misuse of the RC4 cipher. It is used in a large range of modern security devices, because it provides a high degree of privacy and that for a relatively low performance penalty (consumes very little power since it uses no multiplications).

However, stream ciphers in general, RC4 in particular, are questionable choices in an unreliable datagram environment like that of WEP. In a datagram environment, the same packet is often resent because of a transmission error, which happens as much as 20% of the transmissions. This is normal, but unwanted for a secure stream cipher. Because of this datagram environment, two properties of a stream cipher produce severe privacy gaps that, in conjunction with the IV and other flaws, can be abused by attackers.

RC4 has no random access property Before we actually begin discussing the security related flaws, it should be noted on why RC4 actually was a bad choice by the IEEE 802.11 group when it comes to encryption and decryption speed on the MAC-layer.

The loss of a single bit of a data stream encrypted under RC4 causes the loss of all the data following the lost bit. This is because a data loss desynchronizes the RC4 encryption and decryption engines. A full reset of both the engines is then the only real solution.

Since IEEE 802.11 MAC is neither reliable nor delivers-in order at the level of which WEP operates, WEP requires that the cipher support a random access, “seek” type capability, where it is wanted to instantly and efficiently switch the cipher to any selected point in the key stream and not having to begin from the start after an error.

Instead of selecting a stream cipher with characteristics needed for a datagram environment, the WEP architecture tries to accommodate itself by reinitializing the cipher key schedule on every data frame. Creating an overhead that could have been avoided when a cipher with random access was chosen (one such as AES).

RC4 disallows key re-use Stream ciphers have a second property that is important: it is unsafe to use the same key twice, ever.

Presume you have two plaintext byte sequences $p_1, p_2, p_3, \dots$ and $q_1, q_2, q_3, \dots$ both which you encrypt with the a key stream $k_1, k_2, k_3, \dots$. This encryption, as we have seen, is an XOR of the key and plain text. So we have two new cipher texts:

$$p_1 \oplus k_1, p_2 \oplus k_2, p_3 \oplus k_3$$

$$q_1 \oplus k_1, q_2 \oplus k_2, q_3 \oplus k_3$$

Now, presume an attacker captures these two cipher texts; what then follows is a failure of privacy, since:

$$(p_i \oplus k_i) \oplus (q_i \oplus k_i) = p_i \oplus q_i$$

Or in other words, combining two cipher texts produces a stream, which is not dependant of the used key, and so a large deal of information about the plain texts is revealed. If one of the two plain texts is known, the other can be read, if it has the same length, without the need for a key, a property that will be abused in the following flaws.

As a result, stream ciphers are insecure in a datagram environment without some sort of key management to replace keys before they can be reused. The WEP design attempts to accommodate this lack of key management by introducing the IV. WEP combines the IV with the key to produce a new frame specific encryption key, preventing that any collisions of two or more frames with the same key occur (explained further on).

Problem 2: IV

The WEP designers had some knowledge of the first flaw concerning RC4 and therefore introduced a per-packet key (the IV). From a cryptographic view, this is a good solution.

However a major flaw is the fact that no replay protection is implemented whatsoever which shall soon be explained.

IV gives rise to weak keys The paper by Scott Fluhrer, Itsik Mantin, and Adi Shamir presented in August 2001 investigated the RC4 key schedule when a portion of the RC4 key stream is known. Explaining the paper and its consequences in depth requires some advanced mathematical knowledge, thus only a summary is provided:

The paper showed when a part of the RC4 key stream is known, a class of RC4 weak keys could be identified. When these weak keys are used to generate a pseudo random stream there is a small, but not insignificant, correlation between the input (the WEP key) and the output (the key stream).

In other words: weak IV are the cause of some key leakage to the key stream which, of course, is a very unwanted property of any type of cryptographic cipher.

As a result of these so-called weak keys, if the first two bytes of enough key streams (around 60) can be observed, then the WEP key can be recovered through these weak keys using an FMS attack, named after the authors of the paper.

The FMS attack utilizes the fact that, in some cases, knowledge of the IV and the first output byte leaks information about the key bytes. This attack itself is already a problem but it is even made worse because of another implementation error by WEP: the first couple of bytes of encrypted WEP-data in every packet ARE known. The LLC/SNAP header encapsulating some higher layer protocol is always the first 8 bytes in the encrypted packet, namely 0xAA, in other words: we have a part of the original plain text.

How can this be used? Suppose a plain text p_i , where i denotes the number of blocks, which is encrypted with a key stream k_i . This produces the cipher text c_i :

$$c_i = k_i \oplus p_i$$

An interesting property of all one-time pad ciphers, like RC4, is the following:

$$c_i = k_i \oplus p_i \Leftrightarrow k_i = c_i \oplus p_i$$

Suppose p_i is the first 8 bytes of the known plain text. If these known bytes (the LLC/SNAP header) are XOR'd with the first 8 bytes of the cipher text, the first bytes of the key stream are reproduced. Which of course is exactly the part needed to start an FSM-attack. This part of the key stream can now be used to recover the rest of the RC4 key. An attacker now has all the ingredients to read any cipher text using this key, since both the IV and the PRNG are always known. Making it possible to create the needed key streams to decrypt any captured packet, or vice versa: encrypt any chosen data and sent it to anyone, pretending to be an authorized user.



Two very popular Linux programs utilize the FMS-attack: Aircrack-ng & WEPCrack.

IV collisions occur If the FMS attack itself is not disastrous enough on its own, a whole other breed of flaws results from the fact that the IV is only 24 bit large. The use of a 24-bit IV is inadequate because the same IV, and therefore the same key stream, must be reused within a relative short period of time.

A 24-bit field can contain 2^{24} or 16 777 216 possible values. A short calculation demonstrates the short life of a 24-bit IV.

Given: a slow access point running at 11 Mbps and constantly transmitting 1.500-byte packets:

- 11 Mbps / (1.500 bytes per packet x 8 bits per byte) = 916.67 packets transmitted each second
- 16.777.216 IVs / 916.67 packets per second = 18.302,41745 seconds

That means that after a little bit more than 5 hours all IVs are used up and collisions will start occurring.

This flaw can be abused in two ways, by passively attacking the network, or actively flooding the network with data and retrieving the key streams because of the collisions.

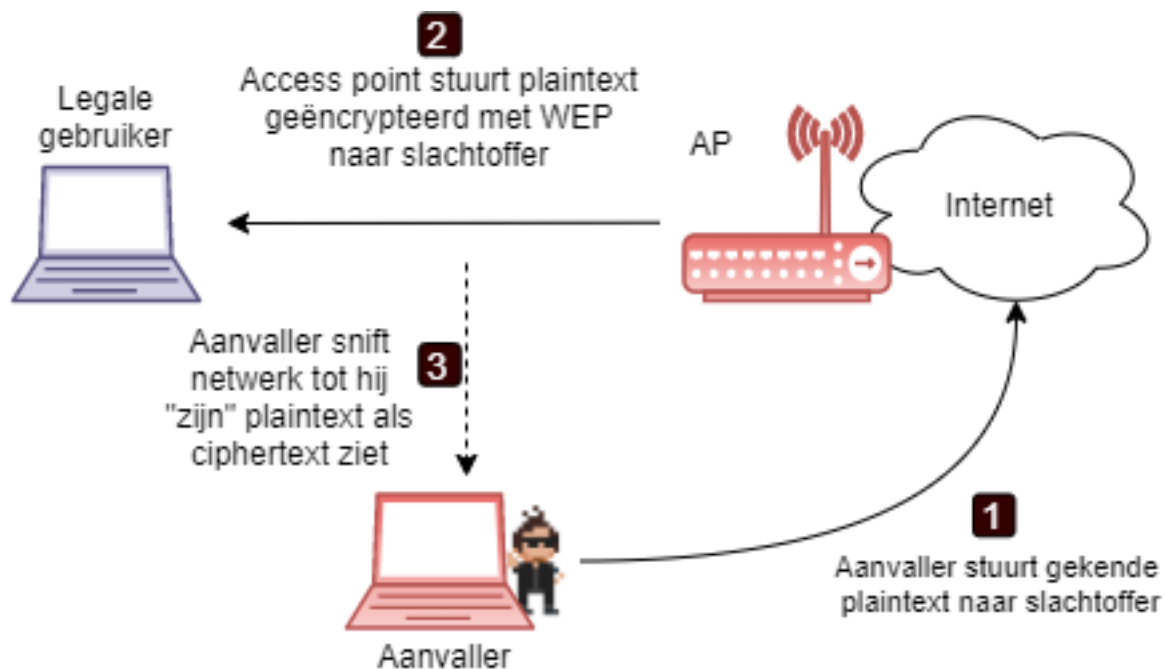
Passive attack A passive eavesdropper can quietly intercept all wireless traffic, until an IV collision occurs. By XOR'ing two packets that use the same IV, the attacker obtains the XOR of the two plaintext messages. The resulting XOR can be used to retrieve information about the contents of the two messages.

IP traffic is often very predictable and includes a lot of redundancy. This redundancy can be used to eliminate many possibilities for the contents of messages. Further educated guesses (through means of cryptanalysis) about the contents of one or both of the messages can be used to statistically reduce the space of possible messages, and in some cases it is possible to determine the exact contents, an example of this was given earlier where the LLC-header was used to launch an FMS attack.

When such statistical analysis is inconclusive based on only two messages, the attacker can look for more collisions of the same IV. With only a small factor in the amount of time necessary, it is possible to recover a modest number of messages encrypted with the same key stream, and the success rate of statistical analysis grows quickly. Once it is possible to recover the entire plaintext for one of the messages, the plaintext for all other messages with the same IV follows directly, since all the pairwise XOR's are known.

Active attack An extension to this attack uses a host somewhere on the Internet to send traffic from the outside to a host on the WLAN installation. The contents of such traffic will be known to the attacker, yielding the known plaintext. When the attacker intercepts the encrypted version of his message sent over 802.11, he will be able to decrypt all packets that use the same initialization vector.

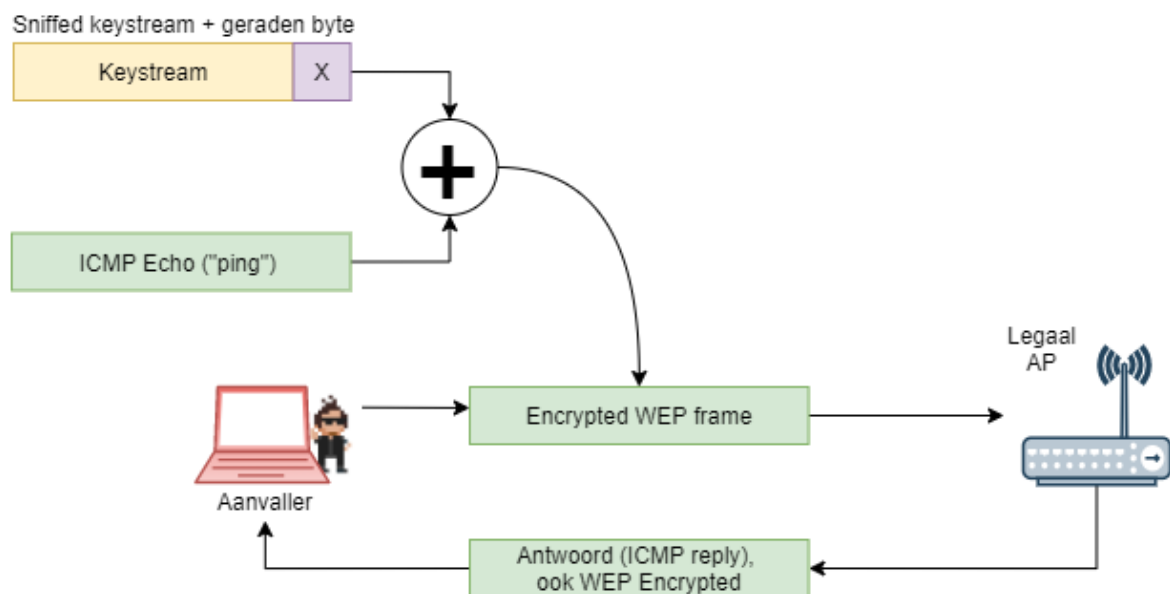
An attacker could use the Internet for this type of attack:



1. A known plain-text message is sent to an observable wireless LAN client (an e-mail message). If the attacker is only capable of utilizing the WLAN he will have to use a bit-flip attack, explained later on.
2. The network attacker will then start sniffing the wireless LAN looking for the predicted cipher-text and eventually find it.
3. The network attacker will find the known frame and derive the key stream using the reverse XOR action: $c_i = k_i \oplus p_i \Leftrightarrow k_i = c_i \oplus p_i$. Where k_i is the desired key stream.

Growing key streams When the attacker has one key stream, the story does not end here. With this key stream, the attack can now create any key stream, of any desired length due to the fact that there is no replay protection. WEP doesn't check if a frame sent is authentic or not, if a frame is encrypted with the right key, WEP considers the user to be valid. Someone can capture a packet and resend it at any given time; it will not be checked and thus be used as if it were a valid packet. An attacker can thus use the WLAN as was it is own private laboratory where he can test as much as desired.

And so the attacker can grow his own key stream of any given length :



1. The network attacker can build a frame one byte larger than the known key stream size; an Internet Control Message Protocol (ICMP) echo frame is ideal because the access point solicits a known response.
2. The network attacker then augments the key stream by one byte.
3. The additional byte is guessed because only 256 possible values are possible and he can 'guess' as many times as he wants.
4. When the network attacker guesses the correct value, the expected response is received: in this example, the ICMP echo reply message. Otherwise he won't get anything since the packet was encrypted with an invalid key stream.
5. The process is repeated until the desired key stream length is obtained.

IV Selection The fourth problem with the IV is how it needs to be selected. The 802.11 standard specifies no rules for IV selection, instead it merely recommends updating "frequently", a pretty undefined term. Each vendor implemented his or her own IV selection strategy, some being smarter than others:

- Fixed IV: Some implementations operate with a fixed IV, employing the same RC4 key to encrypt every packet. Making it necessary that the RC4 key needs to change after each packet, since a collision occurs afterwards with the second packet!
- Random IV: Other vendors selected the IV at random, seemingly the best solution but actually not much better than the former solution, all due to the infamous **birthday paradox**. Because this paradox it only takes 4823 packets to have a 50% chance of collision. And so a key change needs to occur about every three seconds.

- Incremental IV: A third option is to have a circular counter, incrementing the IV after each transmission, starting always from zero upon boot. This strategy guarantees a collision after two different stations transmit a single packet, since it is common to use only default keys (i.e. the same key on every device).

It is clear that 16.777.216 possible values are not enough in a high data-rate environment, such as in a WLAN, and so the IV is a severe weakness.

No matter what IV sequencing formula is used, a strong key management system is required, which is not available. This key management could solve the IV problem by including a system that would change the keys before all the IV possibilities are exhausted, thus creating new key streams.

Problem 3: CRC

WEP uses an integrity checksum field to prevent a packet from being modified during transmission, using a CRC-32 checksum. This wasn't the best choice: CRCs in general are designed to detect random errors in a message (such as sudden line interference, noise, etc), not to detect planned forgeries.

This, and the fact that a stream cipher is used to encrypt the payload, gives rise to 1 more group of vulnerabilities in WEP.

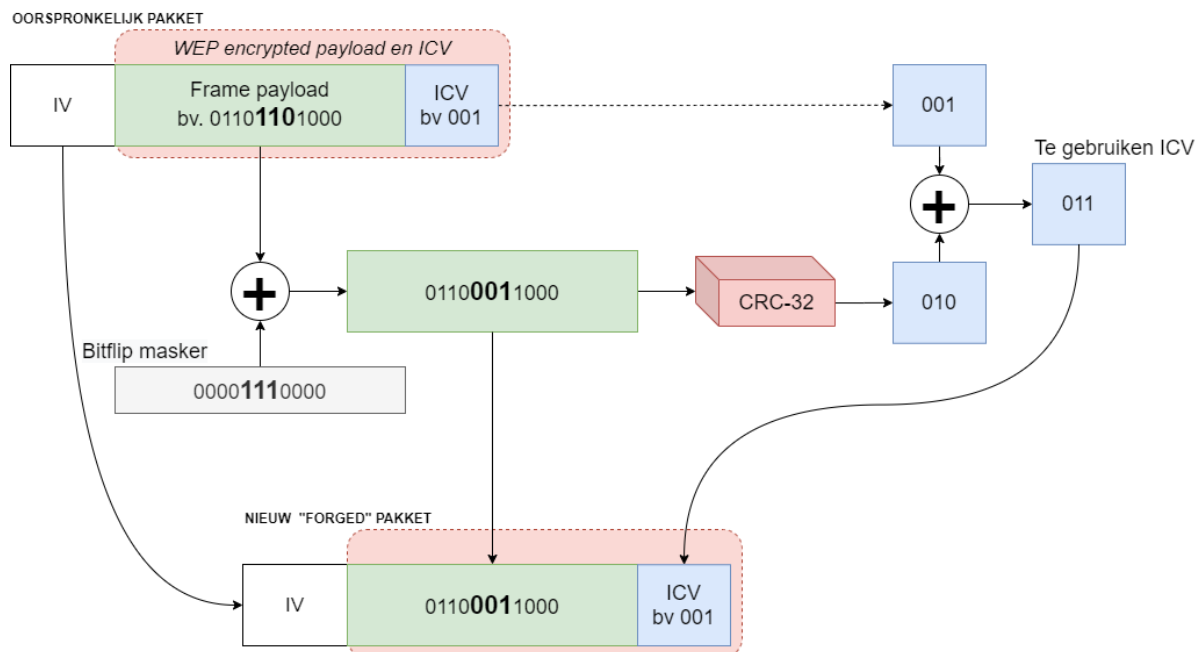
The WEP checksum is a linear function of the message, as is with all CRCs. Meaning that the CRC of one message XOR'd with the CRC of another message, is the same as the CRC of two XOR'd messages:

$$CRC(message_1) \oplus CRC(message_2) = CRC(message_1 \oplus message_2)$$

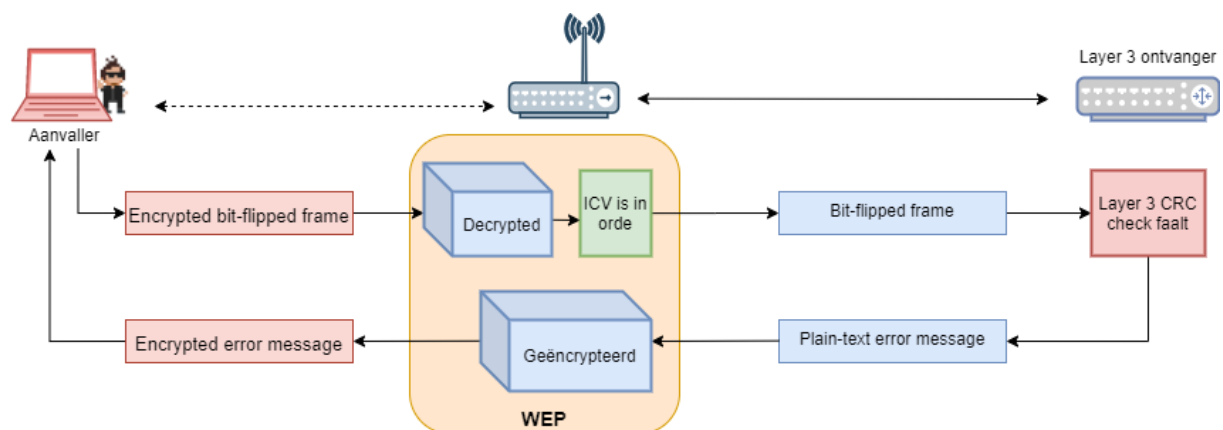
As a consequence it becomes possible to make a controlled forgery without disrupting the checksum. To do this, an attacker employs a 'bit-flip attack'.

Bit-flip attack A bit-flip attacks enables an attacker to inject his own messages without the receiver being able to detect that it is not the original message.

Firstly the attacker captures a valid WEP frame (without needing to know the plaintext context). He then XOR's this frame with a self created stream of bits, where the 1 bits cause the bit to flip (0 to 1, 1 to 0), the so-called bitmask. Usually this stream of bits is randomly. As will be shown, the attacker just needs a valid frame, whatever the data it carries. Lastly the attacker, to have a valid ICV, XOR's both ICVs of the new and original frame. Since XOR'ing is commute (the order of the operations is not relevant) a new valid hash is created.



The attacker can now send a forged frame that is considered genuine by the receiver. The receiver (AP) dutifully decapsulates the bit flipped data and the LLC discovers that the data is 'gibberish'. Yet, since the ICV was valid, the receiver simply thinks some higher layer CRC error has occurred and thus sends an encrypted error-message to the attacker.



It is this error-message that the attacker knows he will receive, so encrypted or not, the attacker can now recreate the key stream by XOR'ing the encrypted error-message with the original error-message as shown as explained before: $c = k \oplus p \Leftrightarrow k = c \oplus p$

Once this key stream k is derived, the attacker can have a key stream of any size by 'growing' one, as described earlier. By doing this, the hacker can create a dictionary of keystreams of all sizes. From then on, the attacker can transmit any message of any size, since he has valid keystreams



Note that the attacker in this scenario doesn't need to have the WEP-key. He simply uses the keystreams to mimick being a valid user.

Problem 4: Keying

A problem, where most symmetric systems suffer from, is the key distribution. The default keys need to be distributed to all the stations that want to participate in the service set secured by WEP. There is, however, no key distribution specified in the 802.11 standard. And so, vendors haven't done anything: most devices just have you type the keys manually in to the device drivers or APs. (Some provide a way of distributing the keys on a small disk with an executable).

It is clear that this manual entry is entirely dependent of the users and system manager, not of the protocols, creating a very insecure key environment:

Keys cannot be considered secret: all keys need to be manually entered, and any local user, with a minor knowledge of his system, can extract the keys (e.g. in Windows OS through the device manager).

Whenever someone, a staff member for example, leaves the organization, all keys should be changed. Knowledge of WEP keys allows a user to setup a station and passively monitor and decrypt traffic using the secret key. WEP thus cannot protect against authorized insiders who also have the key. Organizations with a large number of authorized users must publish the key to this group when needed, which, of course, prevents the keys from being secret.

WPA 1

By 2001, it was clear that an urgent solution was needed. Wifi was booming everywhere, both in private homes as in companies. A task group was created to create a new standard. However, they couldn't just simply start writing a new security standard for the IEEE 802.11 specifications; some constraints existed:

- Already millions of WEP-based devices have been sold. WEP patches operating on already-deployed hardware therefore will have to rely entirely on a firmware upgrade.
- Most AP's are equipped with a cheap, slower processor (i486, ARM7 or PowerPC running at 40 or even 25MHz). The load however generated by normal WLAN traffic consumes 90% or more of this microprocessor. And so there aren't many spare cycles left making that the solutions need to have a limited amount of instructions.
- To support RC4 encryption without consuming too many cycles, additional hardware is installed to take care of the encryption functions. However, these are basically ASICs and these hardwired

encryptions thus create a third constraint. Some functions of the WEP/RC4 combination will always be performed, no matter what. And so WEP will never really leave the security business.

As a result of these constraints, the taskgroup started working on two solutions, one that will work with these constraints, the other one won't. One solution will be entirely new, based on AES and will be used for future devices called "counter with CBC-MAC mode AES protocol" (CCMP, WPA2). To allow existing devices to be upgraded, a lightweight protocol called Temporal Key Integrity Protocol (TKIP, WPA1) is being developed. TKIP can be implemented on existing hardware platforms while still providing acceptable security in the short-term future.

Another standard that is being discussed is IEEE 802.1X standard, which enables authentication and key management for LANs. 802.1X will be used together with CCMP and/or TKIP, the latter two providing the data encapsulation and integrity, the former, 802.1X, providing the key management and authentication.

802.1X

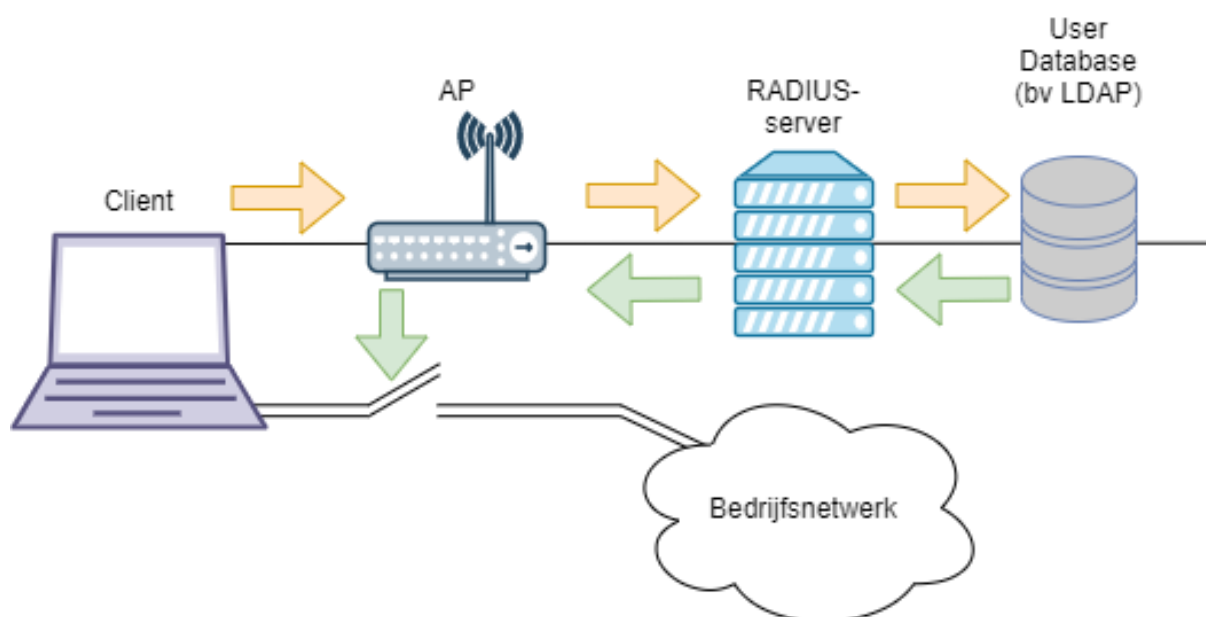
Where TKIP and CCMP provide integrity and encryption, it is 802.1X that provides the following:

- Provide (mutual) **authentication**.
- Have a **central user management** system.
- Have a secure way to **distribute secret keys**.

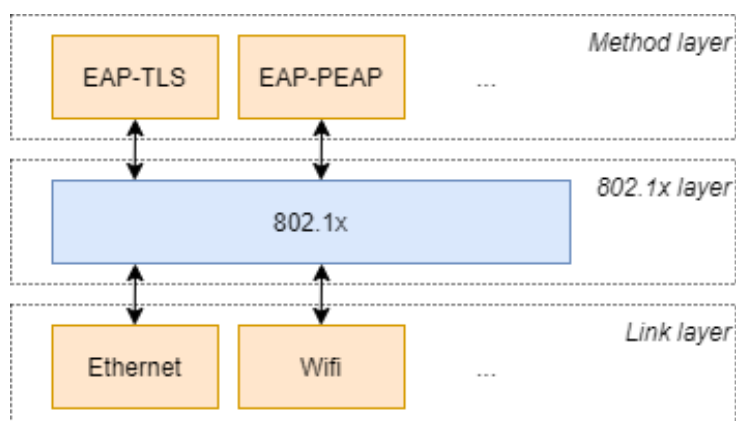


As noted, 802.1X is only used in the enterprise modes of WPA1 and WPA2. If you are a home user, you will probably use WPA-Personal, which is without 802.1X. In personal mode, you simply will have a secret passphrase that is only known by the access points and the legal user.

There are several methods to have one authenticated but the most important thing to remember is that usually the access point won't do the actual authentication (some access points actually do). This is done by a RADIUS server; the access point (authenticator) merely relays the authentication message exchanges between the user (supplicant) and RADIUS (authentication server). While the user isn't authenticated the access point will only allow 802.1x/EAP-based traffic. Once the user is authenticated the access point will open the port for normal traffic.



Make note that 802.1X is not an authentication/authorization algorithm itself; it merely translates messages to and from an authentication/authorization algorithm, using the appropriate frame formats. 802.1X leaves the choices of authentication/authorization algorithm, key management method and accounting profiles up to each EAP authentication type.



EAP (Extensible Authentication Protocol) is extensible meaning that several different authentication methods can be used, depending on the user friendliness and grade of security.

It is important to understand that you have to make sure that both your client and server are compatible with the chosen EAP method. From a customer point-of-view this means that you have to check not only whether the AP is 802.1X-, EAP- or WPA-compatible but that it also supports the wanted EAP method(s).

The most important and usually most-supported EAP (described later on) methods are:

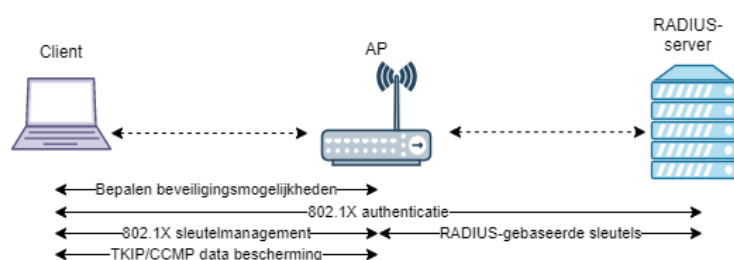
- EAP-TLS
- PEAP
- LEAP
- EAP-TTLS

And to a lesser extent:

1. EAP-SIM

For an 802.1X port-controlled environment to work it is of utmost importance that your access points are connected to a switch (and not to a hub) and more importantly that the switch is 802.1X-compatible. Most switches from the last 10 years will probably be 802.1X-compatible, with older ones this might not be case.

When integrating your wireless network in your wired LAN it is important that you isolate the traffic of the access points in the network (just as you would isolate Internet traffic using a firewall). Using a firewall would give a large management overhead and is not recommended. A good option is to have a Virtual LAN for isolating your wireless network traffic.



The previous figure shows what general steps are performed using 802.1X:

1. Both the client and the access point will exchange messages to discover which EAP-methods are possible to use (being a method they both thus should have).
2. Once a mutual agreement on the EAP-method is reached, the AP will act as a relay between the client and the authentication server. Client and server will then commence the authentication exchange.
3. If the server has enough evidence to authenticate the client it will generate the necessary keys (explained here-after) and relay them to the access point.
4. The access point will further distribute the needed keys and make sure they are updated often.
5. Once the keys are correctly distributed the actual data can be started, which will be encrypted using CCMP or TKIP, depending on the chosen encryption type.

In 802.1X-enabled WLANs, two sets of keys are generated, session keys (also referred to as pairwise keys) and group keys (also referred to as groupwise keys). Group keys are shared amongst all the clients connected to the same AP and are used for multi-cast traffic. Session keys are unique to each

association between an individual client and the AP and create a private virtual port between a client and the AP.

If configured to implement dynamic key exchange, the 802.1X authentication server can return session keys to the access point along with the accept message. The access point uses the session keys to build, sign and encrypt an EAP key message that is sent to the client immediately after sending the success message. The client can then use contents of the key message to define applicable encryption keys. In typical 802.1X implementations, the client can automatically change encryption keys as often as necessary to minimize the possibility of eavesdroppers having enough time to crack the key in current use.

TKIP

Having obtained the needed keys through the 802.1X framework, it is time to encrypt the data. The **Temporal Key Integrity Protocol** (TKIP) is a suite of algorithms wrapping the WEP protocol on old hardware to enhance security, minimizing the threats that exist when using WEP.

TKIP surrounds WEP with new algorithms, namely:

1. A cryptographic message integrity code (MIC), called Michael, to defeat forgeries.
2. A new IV sequencing discipline, to prevent replay attacks.
3. A per-packet key mixing algorithm, to solve the weak keys issue in RC4.
4. A re-keying mechanism, which changes the integrity keys every 10,000 packets or so.

Because an adversary can compromise the TKIP MIC with relatively few messages (it isn't the most secure thing, explained in the next chapter), TKIP also implements countermeasures. In the original WEP specifications, no countermeasures were undertaken when an attack was detected; taken into account that WEP itself could actually not detect a lot.

In TKIP, these countermeasures have the following consequences: 1. They force key updates to be rate limited. 2. They limit the probability of a successful forgery and the amount of information an attacker can learn about a key before it is renewed.

Firstly, we will describe how these 4 algorithms work on their own, afterwards shall be described they all work together under TKIP.

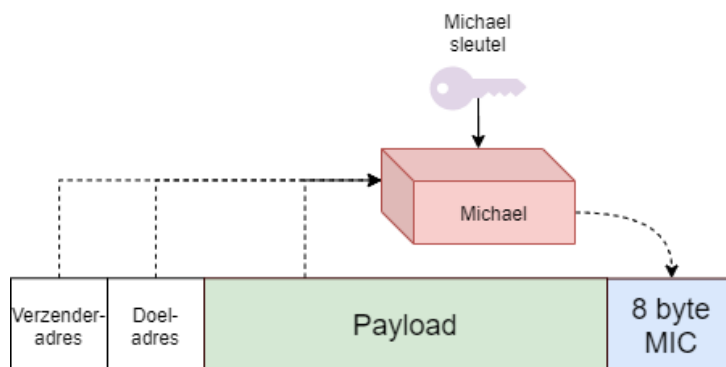
Michael the Mic

A message integrity check (MIC) is a cryptographic device to detect any tampering with data.



The literature calls these MICs usually ‘message authentication codes’ or MACs. However, IEEE 802 already used this acronym for “media access control” and so MIC was chosen instead. A typical MIC is the HMAC used in the IPSec protocol suite.

“Michael” computes a MIC of the payload but it also includes the authentication key, the sender address and the receiver address (in comparison to CRC-32 which only included the payload itself).



When a TKIP implementation detects two failed forgeries in a second, it is assumed there’s an ongoing attack and performs the following steps: 1. The station deletes its keys 2. Disassociates from the AP 3. Waits for a minute and then reassociates

Although this disrupts the communications, it is the only way of thwarting the active attack.

IV Sequence enforcement

WEP did not provide protection against replay attacks since any packet resend later, with a valid key, was considered secure. A simple, yet robust method to prevent replaying packets is to introduce a sequencing number for each packet send. This sequence number is associated with a MIC key (a MIC on its own doesn’t provide detection of replay attacks).

Only a packet which has the next number of the previously send packet is allowed. Of course, this would mean that an infinite sequencing is necessary. Otherwise, in a finite sequencing loop, once all numbers are used replay attacks are possible. The choices available for the transmitter to prevent this sequence number exhaustion are: 1. Halt communication altogether 2. Rekey the MIC with a fresh key 3. Keep sending, but with no protection against replay.

TKIP closely follows this design. To defeat replays, TKIP uses an extended 48-bit IV called the TKIP sequence counter (TSC). The TSC is constructed from the first and second bytes from the original WEP IV and the 4 bytes provided in the extended IV. TKIP extends the length of a WEP encrypted frame by 12 bytes; 4 bytes for the extended IV information and 8 bytes for the MIC.

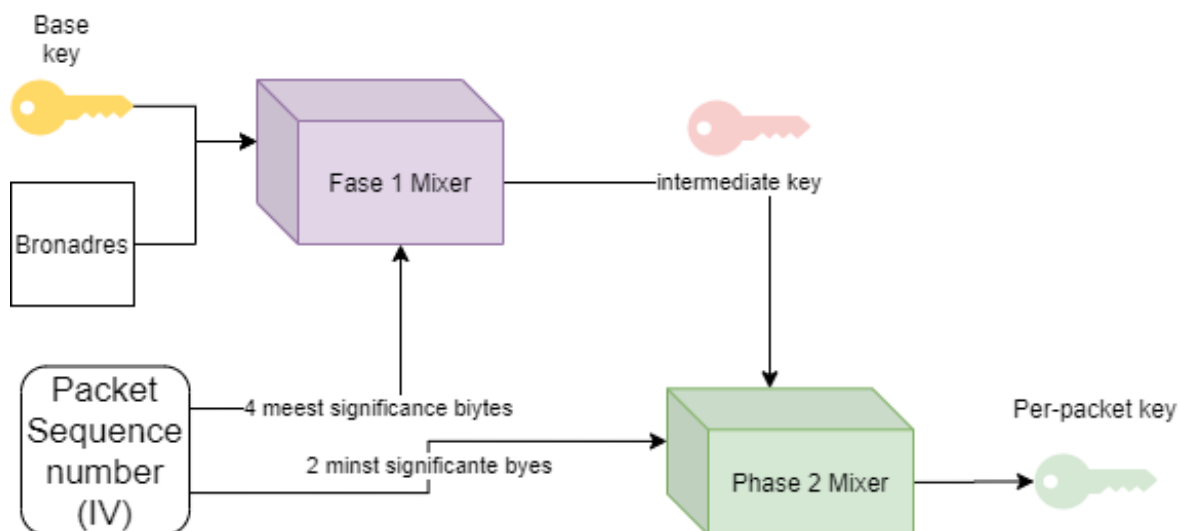
Key mixing

WEP misuses the RC4 algorithm as described earlier. TKIP's per-packet key construction is a feature therefore necessary to correct this flaw. The new per-packet construction, the **TKIP key mixing function, uses a temporal key or per-packet key, which is substituted for the WEP base key**. This key is called temporal because they have a short lifetime and are replaced frequently.

The key mixing works as follows. A key mixing function transforms a temporal key and packet sequence number into a per-packet key and IV.

This is done in two phases, each phase compensating for a particular WEP design flaw:

- Phase 1 eliminates the same key from use by all links by mixing the transmitter address (TA) with the IV and the base key.
- Phase 2 eliminates knowing the per-packet key by viewing the public IV.



Phase 1 mixing

Phase 1 combines the MAC address of the local wireless interface and the temporal key by iteratively XOR'ing each of their bytes to index into an S-Box, producing an intermediate key.

By doing so, using the local MAC address, a different key is generated, even if they begin from the same temporal key (a common situation in ad hoc deployments). And so the stream of generated per-packet encryption keys generated is different at every station. The intermediate keys need only to be computed when the temporal key is updated, so most implementations cache this key (to improve performance).

Phase 2 mixing

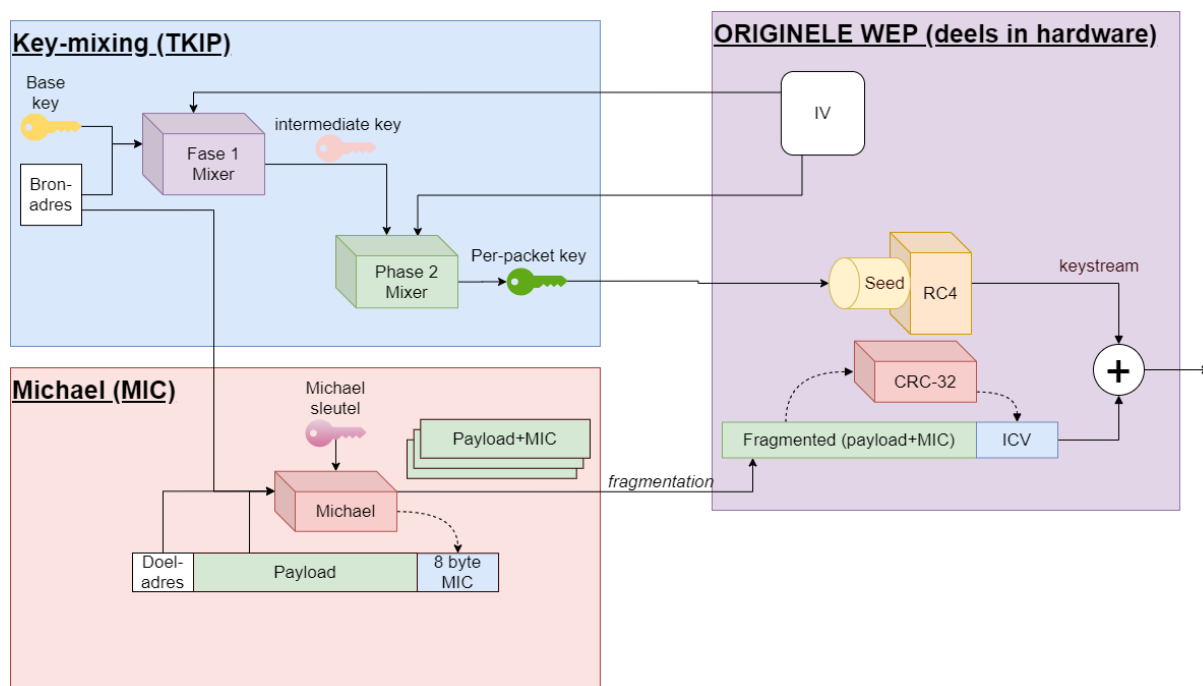
A tiny cipher, a Feistel structure (see crypto chapter), is used in phase 2 to encrypt the packet sequence number, using the intermediate key. A 128-bit per-packet key is produced.

The first 3 bytes of the phase 2 output correspond exactly to the WEP IV, the last 13 to the WEP base key, as existing WEP hardware expects. (Since it uses the base key and IV to form the per-packet key).

Phase 2 assigns the 8 most significant bits of this counter to the first and second bytes of the WEP IV, the least significant counter bits to the third IV byte. The most significant bit of the second IV byte is then masked; ultimately preventing any RC4 weak keys attacks.

Putting it all together

And so finally we have all the pieces ready, which we now can wrap around the broken WEP hardware:



Even though this solution was a nice interim solution, at its core, WEP was still used. In 2009 WPA1-Personal was ready to be retired as several new vulnerabilities were found that allowed attacker to retrieve the WPA passphrase by capturing the handshake client and access point perform when they start communicating.



Check out coWPAtty and Aircrack. Also, have a look at Asleap and THC-LEAPcracker which can hack parts of 802.1X.

WPA 2

The final solution that would ignore the constraints was built around AES.

AES based encryption can be used in a number of different modes or algorithms. The mode that has been chosen for 802.11 is the counter mode with CBC-MAC (CCMP). The counter mode delivers data privacy while the CBC-MAC delivers data integrity and authentication. This was named: **“counter with CBC-mode AES protocol”(CCMP)**.

CBC-MAC, short for “Cipher Block Chaining – Message Authentication Code”, as its name implies will create a hash of the data being encrypted in AES.

As noted, 802.1X is used for key distribution and user authentication.

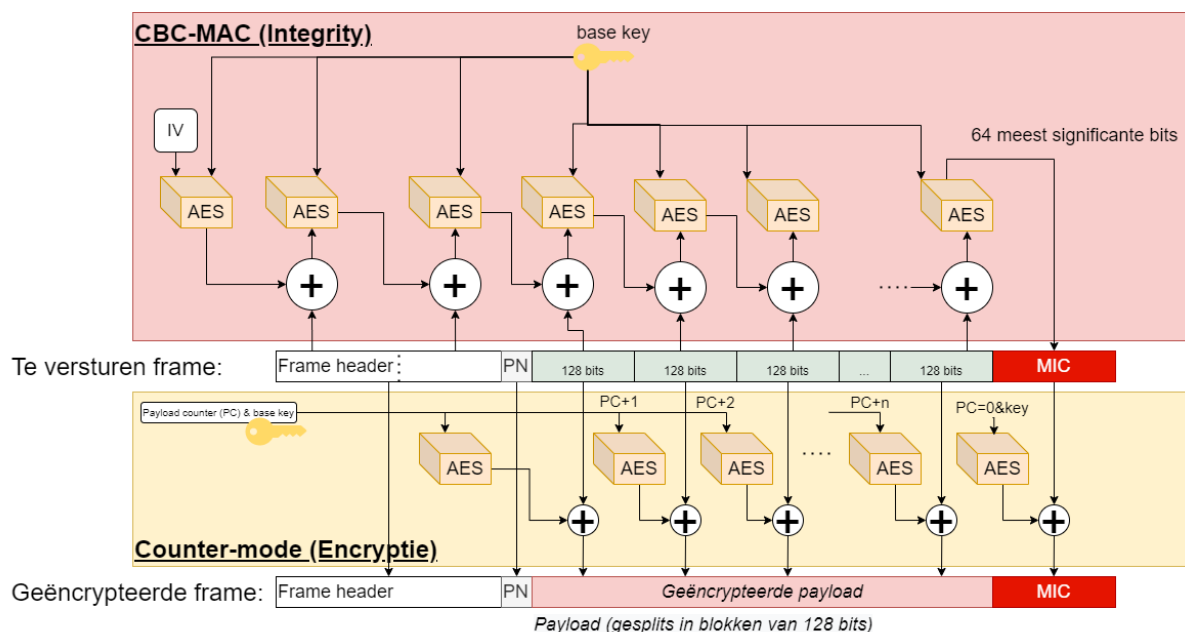
Encryption and MIC creation

Like TKIP, CCMP also uses a 48-bit IV called a packet number (PN). The packet number is used along with other information to initialize the AES cipher, which is a block cipher, for both the MIC calculation and the frame encryption.



Remember that MAC was already reserved and thus CBC-MAC could actually be CBC-MIC.

The AES encryption blocks in both the MIC calculation and the packet encryption use the same temporal encryption key (K in the figure). As with TKIP, the temporal key is derived from the master key that was derived as part of the 802.1X exchange.



The MIC calculation and encryption proceed along parallel paths as shown the figure. The MIC calcu-

lation is seeded with an IV formed by a flag value, the PN, and other data pulled from the header of the frame. This IV is fed into an AES block and its output is XOR'd with select elements from the frame header, which is then fed into the next AES block. This process continues over the remainder of the frame header and down the length of the packet data to compute a final 128-bit CBC-MAC value. The upper 64 bits of this MAC are extracted and used in the final MIC appended to the encrypted frame. The encryption process is seeded by a counter preload also formed from the PN, a flag value, data from the frame header, and a counter value which is initialized to 1.

This preload value is fed to the AES block and its output is XOR'd with 128 bits of clear text from the unencrypted frame. The counter value is incremented by one and this process is repeated for the next block of 128 bits of clear text. This process continues down the length of the frame until the entire frame has been encrypted. The final counter value is set to 0 and input to an AES block whose output is XOR'd with the MIC value computed previously before appending to the end of the encrypted frame for transmission.

There goes the neighbourhood

Until 2017 all seemed well. WPA2 was a well-spoken of standard and everyone was happy with the security provided. However, in May 2017 Belgian researcher Mathy Vanhoef published a paper describing it as follows:

We discovered serious weaknesses in WPA2, a protocol that secures all modern protected Wi-Fi networks. An attacker within range of a victim can exploit these weaknesses using key reinstallation attacks (KRACKs). Concretely, attackers can use this novel attack technique to read information that was previously assumed to be safely encrypted. This can be abused to steal sensitive information such as credit card numbers, passwords, chat messages, emails, photos, and so on. The attack works against all modern protected Wi-Fi networks. Depending on the network configuration, it is also possible to inject and manipulate data. For example, an attacker might be able to inject ransomware or other malware into websites.



We will not go into detail on how this attacks works. Check out krackattacks.com to learn more about it.

As a result, IEEE needed to go back to the drawing board and design a *final final solution*, called WPA3.

WPA 3 (“Wifi 6”)

In 2018 the Wifi Alliance announced the release of WPA3 (also called Wifi 6). The standard provides much improved security, and also copes with the way wireless networks are used nowadays. Modern networks are populated by a plethora of devices, not only laptops. 21th century wireless networks, at work and at home, have mobiles phones, internet-of-thing devices, printers, and whatnot.

As with WPA1 and 2, version 3 also both a personal and enterprise mode. Additionally, it provides several improvements:

- Simultaneous Authentication of Equals (SAE): this cryptographic concept makes the personal passphrase-based authentication much more secure.
- It replaces the passphrase in WPA2-personal with a more secure implementation.
- It is resistant to offline dictionary attacks
- Forward secrecy: even if hacker the finds the wifi key of old captures, they can’t be decrypted.
- Wifi Easy connect: a user-friendly, and secure, methode to connect internet-of-thing devices to the network.
- Wifi Enhanced open: a secure way of connecting to public hotspots. Gone are the days of sniffing a public network for juicy details, now every user will have a secure, encrypted channel with the access point.

It also replaces 802.1X in WP3-Enterprise with more advanced features:

- Authenticated encryption: 256-bit Galois/Counter Mode Protocol (GCMP-256)
- Key derivation and confirmation: 384-bit Hashed Message Authentication Mode (HMAC) with Secure Hash Algorithm (HMAC-SHA384)
- Key establishment and authentication: Elliptic Curve Diffie-Hellman (ECDH) exchange and Elliptic Curve Digital Signature Algorithm (ECDSA) using a 384-bit elliptic curve
- Robust management frame protection: 256-bit Broadcast/Multicast Integrity Protocol Galois Message Authentication Code (BIP-GMAC-256)



We did not discuss these advanced cryptographic features, but it is safe to say that they are state-of-the-art and very secure.

Dankwoord

Een oprechte dank aan Stefaan Somerling (RealDolmen) voor de feedback op het hoofdstuk omtrent GDPR. Geef gerust een sein als u, conform de GDPR wetgeving, liever uw naam niet in dit document ziet staan ;)

